

1 Algorithm

1.1 LIS

```

1 // Longest increasing subsequence
2 int LIS(const vector<int>& s) {
3     // use binary search to update the smallest
4     // element
5     // that ends in a subsequence of length d
6     vector<int> v{};
7     v.push_back(s[0]);
8     for (int i{1}; i < s.size(); ++i)
9         if (s[i] > v.back()) v.push_back(s[i]);
10        else *lower_bound(v.begin(), v.end(),
11                          s[i]) = s[i];
12    return v.size();
13 }

```

1.2 LCS

```

1 // Longest common subsequence
2 int LCS(const vector<int>& a, const vector<
3         int>& b) {
4     vector<vector<int>>> dp(a.size() + 1,
5                             vector<int>(b.size() + 1));
6     for (int i{1}; i <= a.size(); ++i)
7         for (int j{1}; j <= b.size(); ++j) {
8             dp[i][j] = max(dp[i][j], max(dp[i - 1][j], dp[i][j - 1]));
9             if (a[i - 1] == b[j - 1]) dp[i][j] = max(dp[i][j], dp[i - 1][j - 1] + 1);
10        }
11    return dp[a.size()][b.size()];
12 }

```

2 Basic

2.1 mod helper function

```

1 int add(int i, int j) {
2     if ((i += j) >= MOD)
3         i -= MOD;
4     return i;
5 }
6
7 int sub(int i, int j) {
8     if ((i -= j) < 0)
9         i += MOD;
10    return i;
11 }

```

2.2 self-defined-pq-operator

```

1 auto cmp = [](int a, int b) {
2     return a > b;
3 };
4 priority_queue<int, vector<int>, decltype(
5     cmp)> pq(cmp);

```

2.3 generating all subsets

```

1 for (int b = 0; b < (1<<n); b++) {
2     vector<int> subset;
3     for (int i = 0; i < n; i++) {
4         if (b & (1<<i)) subset.push_back(v[i]);
5     }
6 }

```

2.4 memset

```

1 memset(a, 0, sizeof(a)); // 0
2 memset(a, 0x3f3f3f3f, sizeof(a)); // INF

```

2.5 submask enumeration

```

1 // O(3^n)
2
3 int m = 0b10110; // binary: 10110, decimal:
4 22
5 cout << "Mask: " << bitset<5>(m) << " (" <<
6     m << ")\\n";
7
8 // Enumerate all submasks
9 for (int s = m; s; s = (s - 1) & m) {
10    cout << bitset<5>(s) << " (" << s << ")
11    \\n";
12 }
13
14 // Optionally include the empty submask
15 cout << bitset<5>(0) << " (0)\\n";

```

2.6 custom-hash

```

1 struct custom_hash {
2     static uint64_t splitmix64(uint64_t x) {
3         // <http://xorshift.di.unimi.it/splitmix64.c>
4         x += 0x9e3779b97f4a7c15;
5         x = (x ^ (x >> 30)) * 0
6             xbf58476d1ce4e5b9;
7         x = (x ^ (x >> 27)) * 0
8             x94d049bb133111eb;
9     }
10 }

```

```

7     return x ^ (x >> 31);
8 }
9
10 size_t operator()(uint64_t x) const {
11     static const uint64_t FIXED_RANDOM =
12         chrono::steady_clock::now().
13         time_since_epoch().count();
14     return splitmix64(x + FIXED_RANDOM);
15 }
16
17 unordered_map<long long, int, custom_hash>
18     safe_map;
19 gp_hash_table<long long, int, custom_hash>
20     safe_hash_table;

```

2.7 stringstream split by comma

```

1 while (std::getline(ss, segment, ',')) {
2     segments.push_back(segment);
3 }

```

3 DP

3.1 deque

```

1 /*
2 遊戲 DP - O(N^2)
3 A 與 B 將進行以下的遊戲。
4 最初，他們會得到一個序列 a = (a1, a2, ...,
5     aN)。在 a 尚未為空時，兩位玩家輪流進行以
6     下操作，從 A 開始：
7     從 a 的開頭或結尾移除一個元素。玩家會獲得 x
8     分，其中 x 為被移除的元素。
9     設 X 與 Y 分別為遊戲結束時 A 與 B 的總得分。
10    A 會嘗試最大化 X-Y，而 B 會嘗試最小化 X-Y。
11
12    假設兩位玩家都採取最優策略，請求出最後的 X-Y
13    值。
14
15    定義 dp[i][j] 為在區間 [i, j] 上，對於 B 來
16    說的最優分數 (X-Y)。
17 */
18
19 void solve() {
20     int n;
21     cin >> n;
22     vector<int> a(n);
23     vector<vector<int>>> dp(n + 1, vector<int>
24         (n + 1, 0));
25
26     for (int i = 0; i < n; ++i) {
27         cin >> a[i];
28     }
29 }

```

```

23     dp[i][i] = a[i];
24 }
25
26 for (int i = n - 1; i >= 0; --i) {
27     for (int j = i + 1; j < n; ++j) {
28         dp[i][j] = max(a[i] - dp[i + 1][j],
29                         a[j] - dp[i][j - 1]);
30     }
31 }
32 cout << dp[0][n - 1] << "\\n";
33 }

```

3.2 walk

```

1 /*
2 DP on graphs - O(N^3 Log K)
3 給定一個簡單的有向圖 G，具有 N 個頂點，編號
4     為 1, 2, ..., N。
5 對於任意 i, j (1 ≤ i, j ≤ N)，給定整數 a_{i,j}，
6     表示是否存在從頂點 i 指向頂點 j 的有
7     向邊。若 a_{i,j} = 1，則存在邊；若 a_{i,j} = 0，
8     則不存在。
9 求圖中長度為 K 的不同有向路徑數目，對 10^9+7
10    取模。路徑可重複通過相同邊（即允許重複
11    邊）。
12
13 注意：當我們將鄰接矩陣 m 與 m 相乘時，得到的
14    是長度為 2 的路徑數；若取 m 的 p 次方 m^p，
15    則其 (i, j) 元素表示從 i 到 j 的長度
16    為 p 的路徑數。
17 */
18
19 void solve() {
20     int n, k;
21     cin >> n >> k;
22     vector<vector<int>>> m(n, vector<int>(n));
23
24     for (int i = 0; i < n; ++i) {
25         for (int j = 0; j < n; ++j) {
26             cin >> m[i][j];
27         }
28     }
29
30     Matrix<int> mat(m);
31     mat = power(mat, k);
32     int ans = 0;
33     for (int i = 0; i < n; ++i) {
34         for (int j = 0; j < n; ++j) {
35             ans += mat[i][j];
36             ans %= MOD;
37         }
38     }
39     cout << ans << "\\n";
40 }

```

3.3 grouping

```

1  /*
2  狀態  $DP = O(3^N * 2^N * N^2)$ 
3  有  $N$  隻兔子，編號為  $1, 2, \dots, N$ 。
4
5  對於每一對  $i, j$  ( $1 \leq i, j \leq N$ )，兔子  $i$  與  $j$  的相容
   度由整數  $a_{i,j}$  描述。這裡  $a_{i,i} = 0$  對於
   每個  $i$  ( $1 \leq i \leq N$ )，且  $a_{i,j} = a_{j,i}$  對於任
   意  $i$  與  $j$  ( $1 \leq i, j \leq N$ )。
6
7  A 將  $N$  隻兔子分成若干個群組。每隻兔子必須且
   僅屬於一個群組。分群後，對於每一對  $i$  與
    $j$  ( $1 \leq i < j \leq N$ )，若兔子  $i$  與  $j$  屬於同一群
   組，A 即可獲得  $a_{i,j}$  分。
8
9  求 A 能獲得的最大總分。
10
11 令  $cost[S]$  表示將集合  $S$  中的所有兔子放在同一
   群組時所得到的分數。此值可在  $O(2^N * N^2)$ 
   時間內計算。
12
13 接著我們計算  $dp[S]$ ，表示對集合  $S$  中的兔子進
   行分群時所能得到的最大分數。
14
15 */
16 void solve() {
17     int n;
18     cin >> n;
19     vector<vector<int>>> a(n, vector<int>(n));
20     ;
21     vector<int> cost(1<<n, 0);
22     vector<int> dp(1<<n, 0);
23
24     for (int i = 0; i < n; ++i) {
25         for (int j = 0; j < n; ++j) {
26             cin >> a[i][j];
27         }
28     }
29
30     // backtrack all subset
31     for (int b = 0; b < (1<<n); ++b) {
32         vector<int> subset;
33         for (int i = 0; i < n; ++i) {
34             if (b & (1<<i)) {
35                 for (const int& j : subset) {
36                     cost[b] += a[i][j];
37                 }
38             }
39         }
40     }
41
42     // dp
43     for (int i = 0; i < (1<<n); ++i) {
44         int j = ((1<<n) - 1) ^ i;
45         for (int s = j; s != 0; s = (s - 1)
46             & j) {
47             dp[i ^ s] = max(dp[i ^ s], dp[i]
48                 + cost[s]);
49         }
50     }

```

3.4 matching

```

48     }
49     cout << dp[(1<<n) - 1] << "\n";
50 }

```

```

1  /*
2  Bitmask DP -  $O(N * 2^N)$ 
3  有  $N$  個男人和  $N$  個女人，分別編號為  $1, 2, \dots, N$ 。
4
5  對於每個  $i, j$  ( $1 \leq i, j \leq N$ )，男人  $i$  和女人
    $j$  的相容性由整數  $a[i][j]$  給出。
6  如果  $a[i][j] = 1$ ，則男人  $i$  和女人  $j$  是相容
   的；
7  如果  $a[i][j] = 0$ ，則不是。
8
9  A 正在嘗試組成  $N$  對，每對由一個相容的男人和
   女人組成。在這裡，每個男人和每個女人必須
   恰好屬於一對。
10
11 求 A 可以組成  $N$  對的方法數，結果對  $10^9 + 7$ 
   取模。
12
13 定義  $dp[S]$  為將集合  $S$  中的女性與前  $|S|$  個男
   性配對的方法數。
14
15 */
16 const int maxn = 21;
17 const int MOD = 1e9 + 7;
18 int n;
19 int grid[maxn][maxn];
20 int dp[1<<maxn];
21
22 void solve() {
23     cin >> n;
24     memset(dp, 0, sizeof(dp));
25
26     for (int i = 0; i < n; ++i) {
27         for (int j = 0; j < n; ++j) {
28             cin >> grid[i][j];
29         }
30     }
31
32     dp[0] = 1;
33     for (int s = 0; s < (1<<n); ++s) {
34         int ps = __builtin_popcount(s);
35         for (int w = 0; w < n; ++w) {
36             if ((s & (1<<w)) || !grid[ps][w]) {
37                 continue;
38             }
39
40             dp[s | (1<<w)] += dp[s];
41             dp[s | (1<<w)] %= MOD;
42         }
43     }
44     cout << dp[(1<<n) - 1] << "\n";
45 }

```

3.5 projects

```

1  /*
2  LIS DP -  $O(N \log N)$ 
3  有  $n$  個你可以參加的專案。對於每個專案，你知
   道其開始與結束天數以及可獲得的報酬金額。
4  在同一天你最多只能參加一個專案。
5  問：你最多可以賺到多少金額？
6
7   $dp[i]$  = 在第  $i$  天之前我們可以賺到的最大金
   額。
8
9  */
10 void solve() {
11     int n;
12     cin >> n;
13     vector<array<int, 3>> vc(n);
14     map<int, int> days;
15
16     for (int i = 0; i < n; ++i) {
17         int a, b, p;
18         cin >> a >> b >> p;
19         days[a] = days[b] = 1;
20         vc[i] = {a, b, p};
21     }
22     int idx = 1;
23     for (auto& x : days) {
24         x.second = idx++;
25     }
26     vector<int> dp(idx, 0);
27
28     sort(vc.begin(), vc.end(), [](const
29         array<int, 3>& va, const array<int,
30         3>& vb) {
31         if (va[1] != vb[1]) return va[1] <
32             vb[1];
33         if (va[0] != vb[0]) return va[0] <
34             vb[0];
35         return va[2] > vb[2];
36     });
37
38     int i = 0;
39     for (int d = 1; d < idx; ++d) {
40         dp[d] = dp[d - 1];
41         while (i < n && days[vc[i][1]] == d) {
42             dp[d] = max(dp[d], dp[days[vc[i]
43                 ][0]] - 1 + vc[i][2]);
44             i++;
45         }
46     }
47     cout << dp[idx - 1] << "\n";
48 }

```

3.6 stones

```

1  /*
2  遊戲  $DP = O(N^2)$ 
3  有一個集合  $A = \{a_1, a_2, \dots, a_N\}$ ，包含  $N$  個正整
   數。太郎和次郎將進行以下的遊戲。
4

```

```

5 一開始有一堆  $K$  顆石頭。兩位玩家輪流進行以下
   操作，從太郎開始：
6
7 選擇集合  $A$  中的一個元素  $x$ ，並從石堆中移除恰
   好  $x$  顆石頭。
8 當某位玩家無法進行操作時即輸掉比賽。假設兩位
   玩家都採取最優策略，請判斷誰會獲勝。
9
10 定義  $dp[i]$  表示當剩下  $i$  顆石頭時，是否有可能
   獲勝。
11
12 */
13 void solve() {
14     int n, k;
15     cin >> n >> k;
16     vector<int> a(n);
17     vector<bool> dp(k + 1, 0);
18
19     for (int i = 0; i < n; ++i) {
20         cin >> a[i];
21     }
22
23     for (int i = 1; i <= k; ++i) {
24         for (int x : a) {
25             if (i >= x && !dp[i - x]) {
26                 dp[i] = 1;
27             }
28         }
29     }
30     cout << (dp[k] ? "First" : "Second") <<
31         "\n";
32 }

```

3.7 coins

```

1  /*
2  機率  $DP = O(N^2)$ 
3  給定一個正奇數  $N$ 
4  有  $N$  枚編號為  $1, 2, \dots, N$  的硬幣，第  $i$  枚出現正
   面的機率為  $p$ ，反面為  $1 - p$ 。
5  已經拋擲所有硬幣，求正面數大於反面的機率。
6
7  定義  $dp[i][j]$  為拋完前  $i$  枚硬幣後，得到  $j$  次
   正面的機率。
8
9  */
10 void solve() {
11     int n;
12     cin >> n;
13     vector<double> a(n);
14     vector<vector<double>> dp(n + 1, vector<
15         double>(n + 1, 0.0));
16
17     for (int i = 0; i < n; ++i) {
18         cin >> a[i];
19     }
20
21     for (int i = 0; i <= n; ++i) {
22         dp[i][0] = 1.0;
23     }

```

```

24 int least = n / 2 + 1;
25
26 for (int i = 1; i <= n; ++i) {
27     for (int j = 1; j <= least; ++j) {
28         // head
29         dp[i][j] = dp[i - 1][j - 1] * a[
30             i - 1];
31         // tail
32         dp[i][j] += dp[i - 1][j] * (1 -
33             a[i - 1]);
34     }
35 }
36
37 cout << fixed << setprecision(10) << dp[
38     n][least] << "\n";

```

3.8 elevator rides

```

1 /*
2 狀態 DP -  $O(2^N)$ 
3 有  $n$  個人想要搭電梯到樓頂。建築物只有一部電
4 梯。你知道每個人的體重以及電梯的最大允許
5 載重。最少需要搭乘多少次電梯？
6
7 定義  $dp[S] = \{r, w\}$ 。其中  $r$  是將集合  $S$  中的
8 所有人送到樓頂所需的最少電梯次數。而  $w$  是最
9 後一次電梯所載人的總重量。
10
11 */
12
13 void solve() {
14     int n, x;
15     cin >> n >> x;
16     vector<int> w(n);
17     vector<pii> dp(1<<n, {INF, INF});
18
19     for (int i = 0; i < n; ++i) {
20         cin >> w[i];
21     }
22
23     dp[0] = {1, 0};
24     for (int b = 1; b < (1<<n); ++b) {
25         for (int i = 0; i < n; ++i) {
26             if (b & (1<<i)) {
27                 auto [r_prev, w_prev] = dp[b
28                     ^ (1<<i)];
29                 pii can;
30                 if (w_prev + w[i] <= x) {
31                     can = {r_prev, w_prev +
32                         w[i]};
33                 }
34                 else {
35                     can = {r_prev + 1, w[i]
36                         };
37                 }
38                 dp[b] = min(dp[b], can);
39             }
40         }
41     }
42     cout << dp[(1<<n) - 1].first << "\n";
43 }

```

3.9 slimes

```

1 /*
2 Range DP -  $O(N^3)$ 
3 有  $N$  個史萊姆排成一列。最初，從左邊數來第  $i$ 
4 個史萊姆的大小為  $a_i$ 。
5
6 A 想要把所有史萊姆合併成一個更大的史萊姆。他
7 會重複執行以下操作，直到只剩下一個史萊姆
8 為止：
9 選擇兩個相鄰的史萊姆，將它們合併成一個新的史
10 萊姆。新史萊姆的大小為  $x+y$ ，其中  $x$  和  $y$ 
11 是合併前兩個史萊姆的大小。
12 這時會產生  $x+y$  的花費。合併時，史萊姆的相對
13 位置不會改變。
14 請求出合併所有史萊姆所需的最小總花費。
15
16 令  $dp[i][j]$  表示將第  $i$  個到第  $j$  個史萊姆合併
17 成一個史萊姆的最小花費。
18
19 */
20
21 const int maxn = 401;
22 const int INF = 1e18;
23 int dp[maxn][maxn];
24 int a[maxn];
25 int prefix[maxn + 1];
26
27 int f(int i, int j) {
28     if (i + 1 == j) {
29         return a[i] + a[j];
30     }
31     if (i == j) {
32         return 0;
33     }
34     if (dp[i][j] != INF) {
35         return dp[i][j];
36     }
37     //cerr << i << " " << j << "\n";
38
39     int ans = INF;
40     for (int k = i; k < j; ++k) {
41         ans = min(ans, f(i, k) + f(k + 1, j)
42             );
43     }
44     return dp[i][j] = ans + (prefix[j + 1] -
45         prefix[i]);
46 }

```

3.10 digit sum

```

1 /*
2 Digit DP -  $O(|K| * D)$ 
3 計算在  $1$  到  $K$  (含) 之間，滿足其十進位數字和
4 為  $D$  的倍數的整數數量。答案對  $10^9+7$  取
5 模。
6
7 令  $dp[i][j]$  表示在已確定前  $i$  位數字的情況
8 下，構成長度為  $|K|$  的數字且目前數字和

```

```

1 mod  $D$  等於  $j$  的方法數。
2
3 */
4
5 const int MOD = 1e9 + 7;
6 int dp[10001][101][2];
7
8 void solve() {
9     string K;
10     int D;
11     cin >> K >> D;
12     int len = K.size();
13     memset(dp, 0, sizeof(dp));
14     dp[0][0][1] = 1;
15
16     for (int i = 1; i <= len; ++i) {
17         int limit = K[i - 1] - '0';
18         for (int s = 0; s < D; ++s) {
19             for (int flag = 0; flag <= 1; ++
20                 flag) {
21                 int ways = dp[i - 1][s][flag
22                     ];
23                 if (ways == 0) continue;
24                 int max_d = (flag ? limit :
25                     9);
26                 for (int d = 0; d <= max_d;
27                     ++d) {
28                     int rs = (s + d) % D;
29                     int rflag = (flag && d
30                         == max_d ? 1 : 0);
31                     dp[i][rs][rflag] += ways
32                         ;
33                     dp[i][rs][rflag] %= MOD;
34                 }
35             }
36         }
37     }
38
39     int ans = (dp[len][0][0] + dp[len
40         ][0][1]) % MOD;
41     ans = (ans - 1 + MOD) % MOD;
42     cout << ans << "\n";
43 }

```

3.11 sushi

```

1 /*
2 期望值 DP -  $O(N^3)$ 
3 有  $N$  盤壽司，編號從  $1$  到  $N$ 。第  $i$  盤最初有  $a_i$ 
4 ( $1 \leq a_i \leq 3$ ) 塊壽司。
5
6 太郎不斷擲一顆編號  $1$  到  $N$  的骰子。如果結果是
7 第  $i$  盤且該盤還有壽司，他就吃掉一塊；否
8 則什麼也不做。
9
10 請求出吃完所有壽司所需擲骰子的期望次數。
11
12  $dp[x][y][z]$  代表還有
13  $x$  盤剩 1 塊壽司、
14  $y$  盤剩 2 塊壽司、
15  $z$  盤剩 3 塊壽司時的期望擲骰次數。
16
17 */

```

```

14 const int maxn = 301;
15 double dp[maxn][maxn][maxn];
16 int n;
17
18 double dfs(int x, int y, int z) {
19     if (x < 0 || y < 0 || z < 0) return 0;
20     if (x == 0 && y == 0 && z == 0) return
21         0;
22     if (dp[x][y][z] > 0) return dp[x][y][z];
23     double ans = n + x * dfs(x - 1, y, z)
24         + y * dfs(x + 1, y - 1, z)
25         + z * dfs(x, y + 1, z -
26             1);
27     return dp[x][y][z] = ans / (x + y + z);
28 }
29
30 void solve() {
31     cin >> n;
32     vector<int> a(n);
33     memset(dp, -1, sizeof(dp));
34     vector<int> freq(4, 0);
35
36     for (int i = 0; i < n; ++i) {
37         cin >> a[i];
38         freq[a[i]]++;
39     }
40
41     cout << fixed << setprecision(10) << dfs
42         (freq[1], freq[2], freq[3]) << "\n";
43 }

```

3.12 candies

```

1 /*
2 組合 DP -  $O(NK)$ 
3 有  $N$  個小孩，編號為  $1, 2, \dots, N$ 。
4
5 他們決定將  $K$  顆糖果分給自己。對於每個  $i$  ( $1 \leq i \leq N$ )，第  $i$  個小孩最多可以拿到  $a_i$  顆糖果
6 (包含  $0$  顆)。所有糖果都必須分完，不能
7 剩下。
8
9 請問有多少種分配糖果的方法？請將答案對
10  $10^9+7$  取模。若存在某個小孩分到的糖果數
11 不同，則視為不同的分配方式。
12
13 令  $dp[i][j]$  表示將  $j$  顆糖果分給前  $i$  個小孩的
14 方法數。
15
16 */
17
18 void solve() {
19     int n, k;
20     cin >> n >> k;
21     vector<int> a(n);
22     vector<int> dp(k + 1, 0), S(k + 1, 0);
23
24     for (int i = 0; i < n; ++i) {
25         cin >> a[i];
26     }

```

```

21 dp[0] = 1;
22 for (int i = 0; i < n; ++i) {
23     vector<int> new_dp(k + 1, 0);
24     S[0] = dp[0];
25     for (int j = 1; j <= k; ++j) {
26         S[j] = (S[j - 1] + dp[j]) % MOD;
27     }
28     for (int j = 0; j <= k; ++j) {
29         if (j - a[i] - 1 >= 0) {
30             new_dp[j] = (S[j] - S[j - a[i] - 1] + MOD) % MOD;
31         }
32         else {
33             new_dp[j] = S[j] % MOD;
34         }
35     }
36     dp = new_dp;
37 }
38 cout << dp[k] << "\n";
39 }
40

```

3.13 permutation

```

1  /*
2  抽象 DP -  $O(N^2)$ 
3  設  $N$  為正整數。給定一個長度為  $N-1$  的字串  $s$ ，
4  字元僅包含 ' $<$ ' 與 ' $>$ '。
5  求滿足條件的排列 ( $p_1, p_2, \dots, p_N$ ) (即 1 到  $N$ 
6  的排列) 數量。答案對  $10^9+7$  取模：
7  對於每個  $i (1 \leq i \leq N-1)$ ，若  $s$  的第  $i$  個字
8  元為 ' $<$ '，則要求  $p_i < p_{i+1}$ ；若為 ' $>$ '，
9  則要求  $p_i > p_{i+1}$ 。
10
11 令  $dp[i][j]$  表示：在考慮前  $i$  個比較符號 (即
12 構成長度為  $i+1$  的排列) 且最後一個元素為
13  $j$  的有效排列數量。
14
15 */
16 void solve() {
17     int n;
18     string s;
19     cin >> n >> s;
20     vector<vector<int>> dp(n + 1, vector<int>
21         >(n + 1, 0));
22     vector<int> prefix(n + 1, 0);
23     dp[1][0] = 1;
24
25     for (int i = 2; i <= n; ++i) {
26         for (int k = 0; k < n; ++k) {
27             prefix[k + 1] = prefix[k] + dp[i
28                 - 1][k];
29         }
30         for (int j = 0; j < i; ++j) {
31             if (s[i - 2] == '>') {
32                 dp[i][j] += prefix[i - 1] -
33                     prefix[j];
34             }
35             else {
36                 dp[i][j] += prefix[j];
37             }
38             dp[i][j] %= MOD;
39         }
40     }
41 }
42

```

```

29     for (int k = j; k < i - 1;
30         ++k) {
31         dp[i][j] += dp[i - 1][k];
32     }
33 }
34 else {
35     dp[i][j] += prefix[j];
36     dp[i][j] %= MOD;
37 }
38 /*
39     for (int k = 0; k < j; ++k) {
40         dp[i][j] += dp[i - 1][k];
41     }
42 }
43 */
44 }
45 int ans = 0;
46 for (int j = 0; j < n; ++j) {
47     ans += dp[n][j];
48     ans %= MOD;
49 }
50 cout << ans << "\n";
51 }
52

```

3.14 Knasack2

```

1  // 01 背包，背包承重大 ( $1e9$ )，物品價值和較小
2  ( $1e5$ )
3
4  const int maxn = 101;
5  const int maxv = 100001;
6  int weight[maxn];
7  int cost[maxn];
8  int dp[maxv];
9
10 void solve() {
11     int n, w;
12     cin >> n >> w;
13
14     for (int i = 0; i < n; ++i) {
15         cin >> weight[i] >> cost[i];
16     }
17     fill(dp, dp + maxv, 1e18);
18
19     dp[0] = 0;
20     for (int i = 0; i < n; ++i) {
21         for (int j = maxv - 1; j >= 0; --j) {
22             if (dp[j] + weight[i] <= w) {
23                 dp[j + cost[i]] = min(dp[j] +
24                     cost[i], dp[j] +
25                     weight[i]);
26             }
27         }
28     }
29     for (int i = maxv - 1; i >= 0; --i) {
30         if (dp[i] != 1e18) {
31             cout << i << "\n";
32         }
33     }
34 }
35

```

```

30     return;
31 }
32 }
33 }
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
23
```

```

10 long long n;
11 cin >> n;
12 dp[1][0] = 1;
13 dp[1][1] = 1;
14 for(int i = 2; i <= n; i++){
15     dp[i][0] = (4 * dp[i-1][0] + dp[i-1][1]) % mod;
16     dp[i][1] = (dp[i-1][0] + 2 * dp[i-1][1]) % mod;
17 }
18 cout << (dp[n][0] + dp[n][1]) % mod << endl;
19 }

```

3.18 counting numbers

```

1 /*
2 區間 DP - O(digits*10)
3 您的任務是計算在區間 a 到 b 之間，沒有任何相鄰兩位數字相同的整數的個數。
4
5 定義 dp[pos][prev_digit][is_tight][is_started] 為從位置 pos 到結尾的有效整數數量。
6 其中 prev_digit 是位置 pos-1 的數字。
7 is_tight 表示是否受原數字前綴的限制。
8 is_started 表示是否已開始構成有效數字（用以避免計入前導零）。
9 */
10
11 // dp[pos][prev_digit][is_tight][is_started]
12 int dp[20][10][2][2];
13 string s;
14
15 int f(int pos, int prev_digit, bool is_tight, bool is_started) {
16     if (pos == (int)s.size()) {
17         return 1;
18     }
19     if (dp[pos][prev_digit][is_tight][is_started] != -1) {
20         return dp[pos][prev_digit][is_tight][is_started];
21     }
22
23     int ans = 0;
24     int max_d = is_tight ? (s[pos] - '0') : 9;
25     for (int d = 0; d <= max_d; ++d) {
26         if (is_started && d == prev_digit) {
27             continue;
28         }
29
30         bool new_is_started = is_started || (d > 0);
31         bool new_is_tight = is_tight && (d == max_d);
32         ans += f(pos + 1, d, new_is_tight, new_is_started);
33     }
34 }

```

```

35 return dp[pos][prev_digit][is_tight][is_started] = ans;
36 }
37
38 int count(int x) {
39     if (x < 0) return 0;
40     s = to_string(x);
41     memset(dp, -1, sizeof(dp));
42     return f(0, 0, true, false);
43 }
44
45 void solve() {
46     int a, b;
47     cin >> a >> b;
48
49     int ca = count(a - 1);
50     int cb = count(b);
51     cout << cb - ca << "\n";
52 }

```

4 Data Structure

4.1 undo disjoint set

```

1 struct DisjointSet {
2     // save() is like recursive
3     // undo() is like return
4     int n, fa[MXN], sz[MXN];
5     vector<pair<int*, int*>> h;
6     vector<int> sp;
7     void init(int tn) {
8         n = tn;
9         for (int i = 0; i < n; i++) sz[fa[i]] = i + 1;
10        sp.clear(); h.clear();
11    }
12    void assign(int *k, int v) {
13        h.PB({k, *k});
14        *k = v;
15    }
16    void save() { sp.PB(SZ(h)); }
17    void undo() {
18        assert(!sp.empty());
19        int last = sp.back(); sp.pop_back();
20        while (SZ(h) != last) {
21            auto x = h.back(); h.pop_back();
22            *x.F = x.S;
23        }
24    }
25    int f(int x) {
26        while (fa[x] != x) x = fa[x];
27        return x;
28    }
29    void uni(int x, int y) {
30        x = f(x); y = f(y);
31        if (x == y) return;
32        if (sz[x] < sz[y]) swap(x, y);
33        assign(&sz[x], sz[x] + sz[y]);
34        assign(&fa[y], x);
35    }
36 } djs;

```

4.2 segment tree range update (lazy propagation)

```

1 // segment tree
2 // range query & range modify
3 class SGT {
4     using value_t = int;
5     using node_t = pair<value_t, int>;
6
7     int n;
8     vector<node_t> t;
9     vector<optional<value_t>> lz;
10    // [ tv+1 : tv+2*(tm-tl) ) -> Left subtree
11    int left(int tv) { return tv + 1; }
12    int right(int tv, int tl, int tm) {
13        return tv + 2 * (tm - tl);
14    }
15    // ** differ from case to case **
16    // query is "max" and modify is "add" here
17    node_t merge(const node_t& x, const node_t& y) { // associative function
18        return max(x, y);
19    }
20    void update(int tv, int len, const value_t& x) {
21        if (!lz[tv]) lz[tv] = x;
22        else lz[tv] = lz[tv].value() + x;
23        t[tv].fi = t[tv].fi + x;
24    }
25    // *****
26    void build(const vector<value_t>& v, int tv, int tl, int tr) {
27        if (tr - tl > 1) {
28            int tm = (tl + tr) / 2;
29            build(v, left(tv), tl, tm);
30            build(v, right(tv, tl, tm), tm, tr);
31            t[tv] = merge(t[left(tv)], t[right(tv, tl, tm)]);
32        } else t[tv] = {v[tl], tl};
33    }
34    void push(int tv, int tl, int tr) { // lazy propagation
35        if (!lz[tv]) return;
36        int tm = (tl + tr) / 2;
37        update(left(tv), tm - tl, lz[tv].value());
38        update(right(tv, tl, tm), tr - tm, lz[tv].value());
39        lz[tv].reset();
40    }
41    void set(int p, const value_t& x, int tv, int tl, int tr) {
42        if (tr - tl > 1) {
43            push(tv, tl, tr);
44            int tm = (tl + tr) / 2;
45            if (p < tm) set(p, x, left(tv), tl, tm);
46            else set(p, x, right(tv, tl, tm), tm, tr);
47            t[tv] = merge(t[left(tv)], t[right(tv, tl, tm)]);
48        } else t[tv].fi = x;
49    }

```

```

50 }
51 void rmodify(int l, int r, const value_t& x, int tv, int tl, int tr) {
52     if (!(l == tl && r == tr)) {
53         push(tv, tl, tr);
54         int tm = (tl + tr) / 2;
55         if (r <= tm) rmodify(l, r, x, left(tv), tl, tm);
56         else if (l >= tm) rmodify(l, r, x, right(tv, tl, tm), tm, tr);
57         else rmodify(l, tm, x, left(tv), tl, tm), rmodify(tm, r, x, right(tv, tl, tm), tm, tr);
58         t[tv] = merge(t[left(tv)], t[right(tv, tl, tm)]);
59     } else update(tv, tr - tl, x);
60 }
61 node_t rquery(int l, int r, int tv, int tl, int tr) {
62     if (l == tl && r == tr) return t[tv];
63     push(tv, tl, tr);
64     int tm = (tl + tr) / 2;
65     if (r <= tm) return rquery(l, r, left(tv), tl, tm);
66     else if (l >= tm) return rquery(l, r, right(tv, tl, tm), tm, tr);
67     else return merge(rquery(l, tm, left(tv), tl, tm), rquery(tm, r, right(tv, tl, tm), tm, tr));
68 }
69 public:
70 explicit SGT(const vector<value_t>& v) : n{v.size()}, t(2 * n - 1, lz(2 * n - 1) { build(v, 0, 0, n); } } {}
71 void set(int p, const value_t& x) { set(p, x, 0, 0, n); }
72 void rmodify(int l, int r, const value_t& x) { rmodify(l, r, x, 0, 0, n); }
73 // [l:r)
74 node_t rquery(int l, int r) { return rquery(l, r, 0, 0, n); } // [l:r)
75 };
76
77 int main() {
78     vector<long long> a = {1, 5, 2, 4, 3};
79     SGT st(a);
80
81     auto [val, idx] = st.rquery(0, 5);
82     cout << "Initial max: " << val << " at " << idx << "\n"; // (5, 1)
83
84     st.rmodify(1, 4, 3); // add 3 to indices 1..3
85     tie(val, idx) = st.rquery(0, 5);
86     cout << "After add: " << val << " at " << idx << "\n"; // (8, 1)
87
88     st.set(2, 10); // set a[2] = 10
89     tie(val, idx) = st.rquery(0, 5);
90     cout << "After set: " << val << " at " << idx << "\n"; // (10, 2)
91 }

```


4.3 lazy propagation

```

1 // Lazy Propagation Segment Tree supporting
  range add, range set, and range sum
  queries
2
3 int n;
4 int sz;
5 struct Node {
6     int sum = 0;
7     int lazy_add = 0;
8     int lazy_set = 0;
9     bool has_set = false;
10 };
11 vector<Node> tree;
12
13 void build(const vector<int>& a) {
14     tree.resize(sz << 1);
15     for (int i = 0; i < sz; ++i) {
16         tree[i + sz].sum = i < n ? a[i] : 0;
17     }
18     for (int i = sz - 1; i >= 1; --i) {
19         tree[i].sum = tree[i << 1].sum +
20             tree[i << 1 | 1].sum;
21     }
22
23 void apply_set(int idx, int l, int r, int
24     val) {
25     tree[idx].sum = (r - l + 1) * val;
26     tree[idx].lazy_set = val;
27     tree[idx].lazy_add = 0;
28     tree[idx].has_set = true;
29 }
30
31 void apply_add(int idx, int l, int r, int
32     val) {
33     if (tree[idx].has_set) {
34         tree[idx].lazy_set += val;
35     }
36     else {
37         tree[idx].lazy_add += val;
38     }
39     tree[idx].sum += (r - l + 1) * val;
40 }
41
42 void push(int idx, int l, int r) {
43     if (l == r) return;
44
45     int mid = l + (r - l) / 2;
46     int left = idx << 1, right = idx << 1 |
47         1;
48
49     if (tree[idx].has_set) {
50         // propagate the set operation
51         apply_set(left, l, mid, tree[idx].
52             lazy_set);
53         apply_set(right, mid + 1, r, tree[
54             idx].lazy_set);
55         tree[idx].has_set = false;
56         tree[idx].lazy_set = 0;
57         tree[idx].lazy_add = 0;
58     }
59     else if (tree[idx].lazy_add != 0) {
60         // propagate the add operation

```

```

56     apply_add(left, l, mid, tree[idx].
57         lazy_add);
58     apply_add(right, mid + 1, r, tree[
59         idx].lazy_add);
60     tree[idx].lazy_add = 0;
61 }
62
63 void range_set(int idx, int l, int r, int ql
64     , int qr, int val) {
65     if (qr < l || ql > r) return;
66     if (ql <= l && r <= qr) {
67         apply_set(idx, l, r, val);
68         return;
69     }
70     push(idx, l, r);
71     int mid = (l + r) / 2;
72     range_set(idx << 1, l, mid, ql, qr, val);
73     range_set(idx << 1 | 1, mid + 1, r, ql,
74         qr, val);
75     tree[idx].sum = tree[idx << 1].sum +
76         tree[idx << 1 | 1].sum;
77 }
78
79 void range_add(int idx, int l, int r, int ql
80     , int qr, int val) {
81     if (ql > r || qr < l) return;
82     if (ql <= l && r <= qr) {
83         apply_add(idx, l, r, val);
84         return;
85     }
86     push(idx, l, r);
87     int mid = (l + r) / 2;
88     range_add(idx << 1, l, mid, ql, qr, val);
89     range_add(idx << 1 | 1, mid + 1, r, ql,
90         qr, val);
91     tree[idx].sum = tree[idx << 1].sum +
92         tree[idx << 1 | 1].sum;
93 }
94
95 int range_sum(int idx, int l, int r, int ql,
96     int qr) {
97     if (qr < l || ql > r) return 0;
98     if (ql <= l && r <= qr) return tree[idx
99         ].sum;
100     push(idx, l, r);
101     int mid = (l + r) / 2;
102     return range_sum(idx << 1, l, mid, ql,
103         qr) +
104         range_sum(idx << 1 | 1, mid + 1,
105             r, ql, qr);
106 }
107
108 void solve() {
109     int q;
110     cin >> n >> q;
111     vector<int> a(n);
112
113     sz = 1;
114     while (sz < n) sz <= 1;
115     for (int i = 0; i < n; ++i) {

```

```

108     cin >> a[i];
109 }
110
111 build(a);
112
113 while (q--) {
114     int mode, l, r, x;
115     cin >> mode >> l >> r;
116     l--, r--;
117     if (mode == 1) {
118         cin >> x;
119         range_add(1, 0, sz - 1, l, r, x)
120     }
121     else if (mode == 2) {
122         cin >> x;
123         range_set(1, 0, sz - 1, l, r, x)
124     }
125     else {
126         cout << range_sum(1, 0, sz - 1,
127             l, r) << "\n";
128     }
129 }

```

4.4 segment tree prefix sum lower bound

```

1 class SGT {
2     int n;
3     vector<long long> t;
4     int left(int tv) { return tv + 1; }
5     int right(int tv, int tl, int tm) {
6         return tv + 2 * (tm - tl); }
7     void modify(int p, long long x, int tv,
8         int tl, int tr) {
9         if (tr - tl > 1) {
10             int tm = (tl + tr) / 2;
11             if (p < tm) modify(p, x, left(tv),
12                 tl, tm);
13             else modify(p, x, right(tv, tl,
14                 tm), tm, tr);
15             t[tv] = t[left(tv)] + t[right(tv),
16                 tl, tm];
17         } else t[tv] = x;
18     }
19     long long query(int l, int r, int tv,
20         int tl, int tr) {
21         if (l == tl && r == tr) return t[tv];
22         int tm = (tl + tr) / 2;
23         if (r <= tm) return query(l, r, left
24             (tv), tl, tm);
25         else if (l >= tm) return query(l, r,
26             right(tv, tl, tm), tm, tr);
27         else return query(l, tm, left(tv),
28             tl, tm) +
29             query(tm, r, right(tv, tl, tm),
30                 tm, tr);
31     }
32 }
33 public:

```

```

23     explicit SGT(int _n) : n(_n), t(2 * n -
24         1) {}
25     void modify(int p, long long x) { modify
26         (p, x, 0, 0, n); }
27     long long query(int l, int r) { return
28         query(l, r, 0, 0, n); }
29     int ps_lower_bound(long long ps) { //
30         prefix sum lower bound
31         if (ps > t[0]) return n;
32         int tv{0}, tl{0}, tr{n};
33         while (tr - tl > 1) {
34             int tm = (tl + tr) / 2;
35             if (t[left(tv)] >= ps) tv = left
36                 (tv), tr = tm;
37             else ps -= t[left(tv)], tv =
38                 right(tv, tl, tm), tl = tm;
39         }
40         return tl;
41     }
42 };

```

4.5 Fenwick tree (BIT)

```

1 struct Fenwick {
2     int n;
3     vector<int> bit, prefix;
4     Fenwick(int _n) : n(_n), bit(n + 1, 0),
5         prefix(n + 1, 0) {}
6     void update(int i, int delta) {
7         for (; i <= n; i += i & -i) {
8             bit[i] += delta;
9         }
10     }
11     int query(int i) {
12         int ans = 0;
13         for (; i >= 1; i -= i & -i) {
14             ans += bit[i];
15         }
16         return ans;
17     }
18     int range(int l, int r) {
19         return query(r) - query(l - 1);
20     }
21 };
22
23 int main() {
24     int n = 5;
25     vector<int> a = {0, 1, 2, 3, 4, 5}; //
26     1-indexed
27     Fenwick ft(n);
28     for (int i = 1; i <= n; ++i) ft.update(i,
29         a[i]);
30
31     cout << ft.query(3) << "\n"; // 6
32     cout << ft.range(2, 4) << "\n"; // 9
33
34     ft.update(3, 10); //
35     a[3] += 10
36     cout << ft.query(3) << "\n"; // 16
37     cout << ft.range(1, 5) << "\n"; // 25
38 }

```

4.6 BIT - range update + range prefix sum

```

1 int n, q;
2
3 void solve() {
4     cin >> n >> q;
5     Fenwick bit_values(n), bit_count(n);
6
7     while (q--) {
8         int mode, l, r, val;
9         cin >> mode;
10        if (mode == 0) {
11            cin >> l >> r >> val;
12            bit_values.update(l, val);
13            bit_count.update(l, val * (1 - 1));
14            bit_values.update(r + 1, -val);
15            bit_count.update(r + 1, -val * r);
16        }
17        else {
18            cin >> l >> r;
19            int pref_l = bit_values.query(l - 1) * (1 - 1) - bit_count.query(l - 1);
20            int pref_r = bit_values.query(r) * r - bit_count.query(r);
21            cout << pref_r - pref_l << "\n";
22        }
23    }
24 }

```

4.7 Order Statistics Tree

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 using namespace __gnu_pbds;
3
4 template <class T>
5 using Tree =
6     tree<T, null_type, less<T>, rb_tree_tag,
7     tree_order_statistics_node_update>;
8
9 int main() {
10     int n;
11     cin >> n;
12     Tree<int> ist;
13     vector<int> p(n);
14     for (int i = 0; i < n; i++) {
15         ist.insert(i);
16         cin >> p[i];
17     }
18     for (int i = 0; i < n; i++) {
19         int ind;
20         cin >> ind;
21         ind--;
22         int pos = *ist.find_by_order(ind);
23         ist.erase(pos);
24         cout << p[pos] << (i == n - 1 ? '\n' : ' ');
25     }
26 }

```

4.8 Trie (Prefix tree)

```

1 struct trie {
2     int n = 0;
3     trie *a[2];
4     trie() {
5         a[0] = a[1] = nullptr;
6     }
7
8     void insert(int k) {
9         trie *curr = this;
10        for (int i = 63; i >= 0; --i) {
11            bool bit = (k & (1LL << i)) > 0;
12            if (curr->a[bit] == nullptr) {
13                curr->a[bit] = new trie();
14            }
15            curr = curr->a[bit];
16            curr->n++;
17        }
18    }
19
20    void erase(int k) {
21        trie *curr = this;
22        for (int i = 63; i >= 0; --i) {
23            bool bit = (k & (1LL << i)) > 0;
24            curr = curr->a[bit];
25            curr->n--;
26        }
27    }
28
29    int query(int k) {
30        trie *curr = this;
31        int x = 0;
32        for (int i = 63; i >= 0; --i) {
33            x ^= (1LL << i) & k;
34            x ^= 1LL << i;
35            bool bit = (k & (1LL << i)) == 0;
36            if (curr->a[bit] == nullptr || curr->a[bit]->n == 0) {
37                x ^= 1LL << i, bit ^= 1;
38                curr = curr->a[bit];
39            }
40            return x;
41        }
42    };
43 }

```

4.9 segment tree

```

1 // Segment tree
2 template<typename value_t, class merge_t>
3 class SGT {
4     int n;
5     vector<value_t> t;
6     value_t defa;
7     merge_t merge;
8 public:

```

```

9     explicit SGT(int _n, value_t _defa,
10        const merge_t& _merge = merge_t{})
11        : n{_n}, t(2 * n), defa{_defa},
12        merge{_merge} {}
13
14    void modify(int p, const value_t& x) {
15        for (t[p += n] = x; p > 1; p >>= 1)
16            t[p >> 1] = merge(t[p], t[p ^ 1]);
17    }
18
19    value_t query(int l, int r) { return query(l, r, defa); }
20
21    value_t query(int l, int r, value_t init) {
22        for (l += n, r += n; l < r; l >>= 1, r >>= 1) {
23            if (l & 1) init = merge(init, t[l++]);
24            if (r & 1) init = merge(init, t[--r]);
25        }
26        return init;
27    }
28
29    // Custom merge for range minimum + index
30    struct MergeMin {
31        pair<int, int> operator()(const pair<int, int>& a,
32            const pair<int, int>& b) const {
33            if (a.first != b.first) return (a.first < b.first) ? a : b;
34            return (a.second < b.second) ? a : b;
35            ; // tie-break on index
36        }
37    };
38
39    int main() {
40        int n = 6;
41        SGT<pair<int, int>, MergeMin> tree(n, {INT_MAX, -1});
42
43        vector<int> a = {5, 3, 6, 1, 4, 2};
44        for (int i = 0; i < n; ++i)
45            tree.modify(i, {a[i], i});
46
47        auto [min_val, min_idx] = tree.query(1, 5); // range [1, 5]
48        cout << "Min value in [1, 5]: " << min_val << ", at index " << min_idx << '\n';
49    }
50
51    return 0;
52 }

```

4.10 BIT range update point query

```

1 // Fenwick Tree (Binary Indexed Tree) for
2 // Range Updates and Point Queries

```

```

1 template<typename T>
2 class BIT {
3     #define ALL(x) begin(x), end(x)
4 private:
5     vector<T> arr;
6     int n;
7     inline int lowbit(int x) { return x & (-x); }
8     void addInternal(int s, T v) {
9         while (s > 0) {
10             arr[s] += v;
11             s -= lowbit(s);
12         }
13     }
14 public:
15     void init(int n_) {
16         n = n_;
17         arr.resize(n + 1);
18         fill(ALL(arr), 0);
19     }
20     void add(int l, int r, T v) {
21         // add v to interval (l, r], 1-based
22         addInternal(l, -v);
23         addInternal(r, v);
24     }
25     T query(int x) {
26         // value at index x
27         T res = 0;
28         while (x <= n) {
29             res += arr[x];
30             x += lowbit(x);
31         }
32         return res;
33     }
34     #undef ALL
35 };
36
37 int main() {
38     BIT<int> bit;
39     bit.init(5);
40
41     // add +3 to indices 1..3
42     bit.add(0, 3, 3);
43
44     // add +2 to indices 3..5
45     bit.add(2, 5, 2);
46
47     cout << "Value at 1 = " << bit.query(1) << "\n"; // expect 3
48     cout << "Value at 3 = " << bit.query(3) << "\n"; // expect 3+2=5
49     cout << "Value at 5 = " << bit.query(5) << "\n"; // expect 2
50 }

```

4.11 DSU remove node find prev next one

```

1 // previous/next one
2 class PvNx {
3     vector<int> pa, sz, mn, mx;

```

```

4 int find(int x) { // collapsing find
5     return pa[x] == -1 ? x : pa[x] =
        find(pa[x]);
6 }
7 void unionn(int x, int y) { // weighted
    union
8     auto rx{find(x)}, ry{find(y)};
9     if (rx == ry) return ;
10    if (sz[rx] < sz[ry]) swap(rx, ry);
11    pa[ry] = rx, sz[rx] += sz[ry], mn[rx]
        = min(mn[rx], mn[ry]), mx[rx]
        = max(mx[rx], mx[ry]);
12 }
13 public:
14     explicit PnX(int n) : pa(n + 1, -1), sz
        (n + 1, 1), mn(n + 1) { iota(mn.
        begin(), mn.end(), 0), mx = mn; }
15 void remove(int i) { unionn(i, i + 1); }
16 int prev(int i) { return mn[find(i)] -
    1; }
17 int next(int i) {
18     int j{mx[find(i)]};
19     if (i == j) j = mx[find(j + 1)];
20     return j;
21 }
22 bool exist(int i) { return i == mx[find(
    i)]; }
23 };

```

4.12 DSU

```

1 // fast disjoint set union
2 class DSU {
3     vector<int> pa, sz;
4 public:
5     explicit DSU(int n) : pa(n, -1), sz(n,
        1) {}
6     int find(int x) { // collapsing find
7         return pa[x] == -1 ? x : pa[x] =
            find(pa[x]);
8     }
9     void unite(int x, int y) { // weighted
        union
10        auto rx{find(x)}, ry{find(y)};
11        if (rx == ry) return ;
12        if (sz[rx] < sz[ry]) swap(rx, ry);
13        pa[ry] = rx, sz[rx] += sz[ry];
14    }
15 };

```

5 Geometry

5.1 cross. product

```

1 // If cross(a,b,c) > 0, c is to the left of
    line ab
2 // If cross(a,b,c) < 0, c is to the right of
    line ab

```

```

3 // If cross(a,b,c) = 0, a, b, c are
    collinear
4
5 // point struct version
6 // 2D Point structure
7 struct Point {
8     double x, y;
9 };
10
11 // Cross product (scalar in 2D)
12 double cross(const Point& a, const Point& b,
    const Point& c) {
13     // Computes (b - a) x (c - a)
14     return (b.x - a.x) * (c.y - a.y) -
        (b.y - a.y) * (c.x - a.x);
15 }
16
17 // std::complex version
18 typedef std::complex<double> point;
19 #define x real()
20 #define y imag()
21
22 double cross(const point &a, const point &b)
    {
23     return (std::conj(a) * b).imag();
24 }
25

```

5.2 Check Point in Polygon

```

1 /**
2  We can cast a ray from the point P going in
    any fixed direction (people usually go
    to the right).
3  If the point is located on the outside of
    the polygon the ray will intersect its
    edges an even number of times.
4  If the point is on the inside of the polygon
    then it will intersect the edge an odd
    number of times.
5  */
6
7 struct Point {
8     int x, y;
9 };
10
11 int cross(const Point& a, const Point& b,
    const Point& c) {
12     // return (c - a) x (c - b)
13     int ans = (c.x - a.x) * (c.y - b.y) - (c
        .x - b.x) * (c.y - a.y);
14     if (ans == 0) return 0;
15     return ans > 0 ? 1 : -1;
16 }
17
18 bool intersect(const Point& a, const Point&
    b, const Point& c, const Point& d) {
19     int c1 = cross(a, b, c);
20     int c2 = cross(a, b, d);
21     int c3 = cross(c, d, a);
22     int c4 = cross(c, d, b);
23
24     return (c1 * c2 < 0 && c3 * c4 < 0);
25 }

```

```

26
27 bool onSegment(const Point& a, const Point&
    b, const Point& c) {
28     return (cross(a, b, c) == 0 &&
        min(a.x, b.x) <= c.x && c.x <=
        max(a.x, b.x) &&
        min(a.y, b.y) <= c.y && c.y <=
        max(a.y, b.y));
29 }
30
31 void solve() {
32     int n, m;
33     cin >> n >> m;
34     vector<Point> poly(n);
35
36     for (int i = 0; i < n; ++i) {
37         int a, b;
38         cin >> a >> b;
39         poly[i] = {a, b};
40     }
41
42     // set a point at outside point to form
    the ray
43     Point outside = {(int)2e9 + 7, (int)2e9
        + 13};
44     for (int i = 0; i < m; ++i) {
45         int a, b;
46         cin >> a >> b;
47         bool found = false;
48         Point p = {a, b};
49
50         int cnt = 0;
51         if (intersect(poly.back(), poly[0],
            p, outside)) {
52             cnt++;
53         }
54         if (onSegment(poly.back(), poly[0],
            p)) {
55             cout << "BOUNDARY\n";
56             found = true;
57         }
58         for (int j = 0; j < n - 1 && !found;
            ++j) {
59             if (intersect(poly[j], poly[j +
                1], p, outside)) {
60                 cnt++;
61             }
62             if (onSegment(poly[j], poly[j +
                1], p)) {
63                 cout << "BOUNDARY\n";
64                 found = true;
65             }
66         }
67         if (found) continue;
68         if (cnt & 1) {
69             cout << "INSIDE\n";
70         }
71         else {
72             cout << "OUTSIDE\n";
73         }
74     }
75 }
76
77 }

```

5.3 Point

```

1 /**
2  * Author: Ulf Lundstrom
3  * Date: 2009-02-26
4  * License: CC0
5  * Source: My head with inspiration from
    tinyKACTL
6  * Description: Class to handle points in
    the plane.
7  * T can be e.g. double or long long. (
    Avoid int.)
8  * Status: Works fine, used a lot
9  */
10 #pragma once
11
12 template <class T> int sgn(T x) { return (x
    > 0) - (x < 0); }
13 template<class T>
14 struct Point {
15     typedef Point P;
16     T x, y;
17     explicit Point(T x=0, T y=0) : x(x), y(y)
        {}
18     bool operator<(P p) const { return tie(x,y)
        < tie(p.x,p.y); }
19     bool operator==(P p) const { return tie(x,
        y)==tie(p.x,p.y); }
20     P operator+(P p) const { return P(x+p.x, y
        +p.y); }
21     P operator-(P p) const { return P(x-p.x, y
        -p.y); }
22     P operator*(T d) const { return P(x*d, y*d
        ); }
23     P operator/(T d) const { return P(x/d, y/d
        ); }
24     T dot(P p) const { return x*p.x + y*p.y; }
25     T cross(P p) const { return x*p.y - y*p.x; }
26     T cross(P a, P b) const { return (a-*this)
        .cross(b-*this); }
27     T dist2() const { return x*x + y*y; }
28     double dist() const { return sqrt((double)
        dist2()); }
29     // angle to x-axis in interval [-pi, pi]
30     double angle() const { return atan2(y, x); }
31     P unit() const { return *this/dist(); } //
    makes dist()=1
32     P perp() const { return P(-y, x); } //
    rotates +90 degrees
33     P normal() const { return perp().unit(); }
34     // returns point rotated 'a' radians ccw
    around the origin
35     P rotate(double a) const {
36         return P(x*cos(a)-y*sin(a),x*sin(a)+y*
            cos(a)); }
37     friend ostream& operator<<(ostream& os, P
        p) {
38         return os << "(" << p.x << ", " << p.y <<
            ")"; }
39 };

```


5.4 Polygon Lattice Points

```

1  /*
2  Pick's Theorem:
3  For a simple polygon with integer
4  coordinates, the area A, the number of
5  interior lattice points a
6  and the number of boundary lattice points b
7  satisfy:
8   $A = a + b/2 - 1$ 
9  */
10 /*
11 The number of intersections of a line with
12 lattice points is the greatest common
13 divisor of
14  $P1.x - P2.x$  and  $P1.y - P2.y$ 
15 */
16 int countOnSegment(const Point& a, const
17 Point& b) {
18     int dx = abs(a.x - b.x);
19     int dy = abs(a.y - b.y);
20     return __gcd(dx, dy);
21 }
22
23 void solve() {
24     int n;
25     cin >> n;
26     vector<Point> poly(n);
27
28     for (int i = 0; i < n; ++i) {
29         cin >> poly[i].x >> poly[i].y;
30     }
31
32     // b: boundary points
33     int b = countOnSegment(poly.back(), poly
34     [0]);
35     for (int i = 0; i < n - 1; ++i) {
36         b += countOnSegment(poly[i], poly[i
37         + 1]);
38     }
39
40     int area2 = abs(polygonArea2(poly));
41     // a: interior points
42     int a = (area2 + 2 - b) / 2;
43     cout << a << " " << b << "\n";
44 }

```

5.5 line intersect

```

1  // 2D Point structure
2  struct Point {
3      double x, y;
4  };
5
6  // Cross product (scalar in 2D)
7  double cross(const Point& a, const Point& b,
8  const Point& c) {
9      // Computes (b - a) x (c - a)
10     return (b.x - a.x) * (c.y - a.y) -
11            (b.y - a.y) * (c.x - a.x);
12 }

```

```

13 // Check if value is between two others (
14 // with endpoints allowed)
15 bool between(double a, double b, double x) {
16     return min(a, b) <= x && x <= max(a, b);
17 }
18
19 // Check if point c lies on segment ab
20 bool onSegment(const Point& a, const Point&
21 b, const Point& c) {
22     return cross(a, b, c) == 0 &&
23            between(a.x, b.x, c.x) &&
24            between(a.y, b.y, c.y);
25 }
26
27 // Main intersection check
28 bool segmentsIntersect(const Point& A, const
29 Point& B,
30 const Point& C, const
31 Point& D) {
32     double c1 = cross(A, B, C);
33     double c2 = cross(A, B, D);
34     double c3 = cross(C, D, A);
35     double c4 = cross(C, D, B);
36
37     // General case
38     if (c1 * c2 < 0 && c3 * c4 < 0) return
39     true;
40
41     // Special cases: collinear +
42     // overlapping
43     if (c1 == 0 && onSegment(A, B, C))
44         return true;
45     if (c2 == 0 && onSegment(A, B, D))
46         return true;
47     if (c3 == 0 && onSegment(C, D, A))
48         return true;
49     if (c4 == 0 && onSegment(C, D, B))
50         return true;
51
52     return false;
53 }
54
55 int main() {
56     Point A{0, 0}, B{4, 4};
57     Point C{0, 4}, D{4, 0};
58
59     if (segmentsIntersect(A, B, C, D))
60         cout << "Segments intersect\n";
61     else
62         cout << "Segments do not intersect\n";
63
64     return 0;
65 }

```

5.6 Polygon area

```

1  /**
2  * Author: Ulf Lundstrom
3  * Date: 2009-03-21
4  * License: CC0
5  * Source: tinyKACTL

```

```

6  * Description: Returns twice the signed
7  * area of a polygon.
8  * Clockwise enumeration gives negative
9  * area. Watch out for overflow if using
10 int as T!
11 * Status: Stress-tested and tested on
12 kattis:polygonarea
13 */
14 #pragma once
15 #include "Point.h"
16
17 template<class T>
18 T polygonArea2(vector<Point<T>>& v) {
19     T a = v.back().cross(v[0]);
20     for (int i = 0; i < (int)v.size() - 1;
21         ++i) {
22         a += v[i].cross(v[i+1]);
23     }
24     return a;
25 }
26
27 int main() {
28     // Example: square with vertices (0,0),
29     // (0,1), (1,1), (1,0)
30     vector<Point<long long>> poly = {
31         {0,0}, {0,1}, {1,1}, {1,0}
32     };
33
34     long long area2 = polygonArea2(poly);
35     cout << "Twice signed area = " << area2
36     << "\n";
37     cout << "Absolute area = " << abs(area2)
38     / 2.0 << "\n";
39
40     return 0;
41 }

```

5.7 using std::complex

```

1  #include <iostream>
2  #include <complex>
3  #include <cmath>
4
5  typedef std::complex<double> point;
6  #define x real()
7  #define y imag()
8
9  constexpr double PI = std::acos(-1.0);
10 // conj(a) = a 的共軛 · reflection of a
11 // across x-axis
12
13 // Basic operations
14 double dot(const point &a, const point &b) {
15     return (std::conj(a) * b).real(); }
16 double cross(const point &a, const point &b) {
17     return (std::conj(a) * b).imag(); }
18
19 // Projections / reflections / geometry
20 // helpers
21 point project_onto_vector(const point &p,
22 const point &v) {
23     return v * (dot(p, v) / std::norm(v));

```

```

19 }
20
21 point project_onto_line(const point &p,
22 const point &a, const point &b) {
23     return a + (b - a) * (dot(p - a, b - a)
24     / std::norm(b - a));
25 }
26
27 point reflect_across_line(const point &p,
28 const point &a, const point &b) {
29     return a + std::conj((p - a) / (b - a))
30     * (b - a);
31 }
32
33 point intersection(const point &a, const
34 point &b, const point &p, const point &q) {
35     double c1 = cross(p - a, b - a), c2 =
36     cross(q - a, b - a);
37     return (c1 * q - c2 * p) / (c1 - c2); //
38     undefined if parallel (divide by
39     zero)
40 }
41
42 double angle_between(const point &a, const
43 point &b) {
44     // angle of vector b - a
45     return std::arg(b - a);
46 }
47
48 double angle_ABC(const point &a, const point
49 &b, const point &c) {
50     double r = std::remainder(std::arg(a - b)
51     - std::arg(c - b), 2.0 * PI);
52     return std::abs(r);
53 }
54
55 int main() {
56     // sample
57     point a = 2.0; // (2,0)
58     point b(3.0, 7.0); // (3,7)
59     std::cout << a << ' ' << b << '\n'; //
60     (2,0) (3,7)
61     std::cout << a + b << '\n'; //
62     (5,7)
63
64     // usage examples
65     point p1(3, 2), p2(2, -7);
66     std::cout << "p1 + p2 = " << p1 + p2 <<
67     '\n'; // (5,-5)
68     std::cout << "p1 - p2 = " << p1 - p2 <<
69     '\n'; // (1,9)
70     std::cout << "3.0 * p1 = " << 3.0 * p1
71     << '\n'; // (9,6)
72     std::cout << "p1 / 5.0 = " << p1 / 5.0
73     << '\n'; // (0.6,0.4)
74
75     // dot / cross via complex
76     std::cout << "dot(p1,p2) = " << dot(p1,
77     p2) << '\n';
78     std::cout << "cross(p1,p2) = " << cross(
79     p1, p2) << '\n';
80
81     // distances, angle, rotation, polar/
82     cartesian

```

```

63 std::cout << "squared dist = " << std::
    norm(p1 - p2) << '\n';
64 std::cout << "euclid dist = " << std::
    abs(p1 - p2) << '\n';
65 std::cout << "angle p1->p2 = " <<
    angle_between(p1, p2) << '\n';
66 std::cout << "angle ABC = " << angle_ABC
    (point(1,0), point(0,0), point(0,1))
    << '\n';

67 // project / reflect / intersect
    examples
68 point v(1, 1);
69 point proj = project_onto_vector(p1, v);
70 std::cout << "project p1 onto v = " <<
    proj << '\n';

71 point lineA(0,0), lineB(2,0);
72 std::cout << "project p1 onto line = "
    << project_onto_line(p1, lineA,
    lineB) << '\n';
73 std::cout << "reflect p1 across line = "
    << reflect_across_line(p1, lineA,
    lineB) << '\n';

74 // intersection example
75 point r1a(0,0), r1b(1,1);
76 point r2a(0,1), r2b(1,0);
77 std::cout << "intersection = " <<
    intersection(r1a, r1b, r2a, r2b) <<
    '\n';

78 // polar/cartesian and rotation
79 point polar_pt = std::polar(5.0, PI/4);
80 std::cout << "polar(5,PI/4) = " <<
    polar_pt << '\n';
81 point rotated = p1 * std::polar(1.0, PI
    /2); // rotate p1 by 90 degrees
    about origin
82 std::cout << "rotate p1 by 90deg = " <<
    rotated << '\n';

83 return 0;
84 }

```

5.8 point distance from a line

```

1 // calculate area using cross product and
    divide by base length
2 double point_line_distance(const point &p,
    const point &a, const point &b) {
3     // area = 0.5 * |cross(b - a, p - a)|
4     // base = |b - a|
5     // height = 2 * area / base = |cross(b -
    a, p - a)| / |b - a|
6     std::abs(cross(b - a, p - a)) / std::abs
    (b - a);
7 }

```

6 Graph

6.1 Prim

```

1 // Prim's algorithm
2 vector<tuple<int, int, long long>> Prim(
    const vector<vector<pair<int, long long
    >>& adj) {
3     const auto& n = adj.size();
4     vector<tuple<int, int, long long>> mst
    {};

5     vector<bool> found(n, false);
6     using ti = tuple<long long, int, int>;
7     priority_queue<ti, vector<ti>, greater<
    ti>> pq{};
8     found[0] = true;
9     for (auto& [v, w] : adj[0]) pq.emplace(w
    , 0, v);

10    for (int i = 0; i < n - 1; ++i) {
11        int mn, u, v;
12        do {
13            tie(mn, u, v) = pq.top(), pq.pop
    ();
14        } while (found[v]);
15        found[v] = true; mst.emplace_back(u,
    v, mn);
16        for (auto& [x, w] : adj[v]) pq.
    emplace(w, v, x);
17    }
18    return mst;
19 }
20 }
21 }

```

6.2 Eulerian cycle

```

1 // Eulerian cycle in an undirected graph
2 vector<int> euler_cycle(vector<vector<pair<
    int, int>>& adj, int w = 0) {
3     int n{adj.size()}, m{};
4     for (int v{0}; v < n; ++v) m += adj[v].
    size();
5     m /= 2;

6     vector<int> res{};
7     stack<pair<int, int>> stk{};
8     stk.emplace(w, -1);
9     vector<int> nxt(n);
10    vector<bool> used(m);
11    while (!stk.empty()) {
12        auto [u, i]{stk.top()};
13        while (nxt[u] < adj[u].size() && used
    [adj[u][nxt[u].second]] ++nxt[u]
14        );
15        if (nxt[u] < adj[u].size()) {
16            auto [v, j]{adj[u][nxt[u]]};
17            ++nxt[u], used[j] = true, stk.
    emplace(v, j);
18        } else {
19            res.push_back(u);
20            stk.pop();
21        }
22    }
23    return res;
24 }

```

```

20     if (i != -1) res.push_back(i);
21     stk.pop();
22 }
23 }
24 return res;
25 }

26 int main() {
27     int n = 4; // number of vertices
28     vector<vector<pair<int, int>>> adj(n);

29     // Add edges with edge IDs
30     int eid = 0;
31     auto add_edge = [&](int u, int v) {
32         adj[u].push_back({v, eid});
33         adj[v].push_back({u, eid});
34         ++eid;
35     };

36     add_edge(0, 1);
37     add_edge(1, 2);
38     add_edge(2, 3);
39     add_edge(3, 0);

40     vector<int> cycle = euler_cycle(adj, 0);

41     cout << "Euler cycle (edge IDs in order)
    : ";
42     for (int id : cycle) cout << id << " ";
43     cout << "\n";

44     return 0;
45 }

```

6.3 maximum weight bipartite matching

```

1 // maximum weight matching in a weighted
    bipartite graph
2 // Hungarian Algorithm - O(V^3)
3 class bipartiteGraph {
4     int n;
5     vector<long long> lx, ly;
6     vector<bool> vx, vy;
7     queue<int> qu{}; // only X's vertices
8     vector<int> py;
9     vector<long long> dy; // smallest (lx[x]
    + ly[y] - w[x][y])
10    vector<int> pdy; // & which x
11    void relax(int x) {
12        for (int y{0}; y < n; ++y)
13            if (lx[x] + ly[y] - w[x][y] < dy
    [y])
14                dy[y] = lx[x] + ly[y] - w[x
    ][y], pdy[y] = x;
15    }
16    void augment(int y) {
17        while (y != -1) {
18            int x{py[y]}, yy{mx[x]};
19            mx[x] = y, my[y] = x;
20            y = yy;
21        }
22    }
23 }

```

```

22 }
23 bool bfs() {
24     while (!qu.empty()) {
25         int x{qu.front()}; qu.pop();
26         for (int y{0}; y < n; ++y) {
27             if (!vy[y] && lx[x] + ly[y]
    == w[x][y]) {
28                 vy[y] = true, py[y] = x;
29                 if (my[y] == -1) return
    augment(y), true;
30                 int xx{my[y]};
31                 qu.push(x), vx[xx] =
    true, relax(xx);
32             }
33         }
34     }
35     return false;
36 }
37 void relabel() {
38     long long d{numeric_limits<long long
    >::max()};
39     for (int x{0}; x < n; ++x) if (!vy[x]
    ) d = min(d, dy[x]);
40     for (int x{0}; x < n; ++x) if (vx[x]
    ) lx[x] -= d;
41     for (int y{0}; y < n; ++y) if (vy[y]
    ) ly[y] += d;
42     for (int y{0}; y < n; ++y) if (!vy[y]
    ) dy[y] -= d;
43 }
44 bool expand() { // expand one layer with
    new equality edges
45     for (int y{0}; y < n; ++y) {
46         if (!vy[y] && dy[y] == 0) {
47             vy[y] = true, py[y] = pdy[y]
    ;
48             if (my[y] == -1) return
    augment(y), true;
49             int xx{my[y]};
50             qu.push(xx), vx[xx] = true,
    relax(xx);
51         }
52     }
53     return false;
54 }
55 public:
56     vector<vector<long long>> w;
57     vector<int> mx, my;
58     bipartiteGraph(int n) : n{n}, lx(n),
    ly(n), vx(n), vy(n), py(n), dy(n),
    pdy(n),
59     w(n, vector<long long>(n, 0)), mx(n,
    -1), my(n, -1) {}
60     long long Hungarian() {
61         for (int i{0}; i < n; ++i) {
62             lx[i] = ly[i] = 0;
63             for (int j{0}; j < n; ++j)
64                 lx[i] = max(lx[i], w[i][j]);
65             mx[i] = my[i] = -1;
66         }
67         for (int x{0}; x < n; ++x) {
68             fill(vx.begin(), vx.end(), false
    );
69             fill(vy.begin(), vy.end(), false
    );
70         }
71     }

```

```

71     fill(dy.begin(), dy.end(),
72           numeric_limits<long long>::
73           max());
74     qu = {}, qu.push(x), vx[x] =
75     true, relax(x);
76     while (!bfs()) {
77         relabel();
78         if (expand()) break;
79     }
80     long long weight{0};
81     for (int x{0}; x < n; ++x) weight +=
82     w[x][mx[x]];
83     return weight;
84 };
85
86 // usage example:
87 int main() {
88     int n = 4;
89     bipartiteGraph g(n);
90     long long mat[4][4] = {
91         {9, 2, 7, 8},
92         {6, 4, 3, 7},
93         {5, 8, 1, 8},
94         {7, 6, 9, 4}
95     };
96     for (int i = 0; i < n; ++i)
97         for (int j = 0; j < n; ++j)
98             g.w[i][j] = mat[i][j];
99
100     long long maxWeight = g.Hungarian();
101     std::cout << "Maximum weight: " <<
102     maxWeight << "\n";
103     for (int i = 0; i < n; ++i)
104         std::cout << "X " << i << " matched
105         with Y " << g.mx[i] << "\n";
106 }

```

6.4 maximum flow

```

1 // minimum cut maximum flow
2 // relabel-to-front algorithm
3 // (an implementation of generic push-relabel
4 // algorithm)
5
6 // handle the capacity, c, of a vertex, v.
7 // Add a new vertex, v', and an edge v-v'
8 // with capacity c.
9 template<typename T> //requires
10 signed_integral<T>
11 void handle_vertex_capacity(int u, T c, int&
12 n, vector<tuple<int, int, T>>& e) {
13     int w{n++};
14     for (auto& [_u, _v, _c] : e) if (_u == u
15         ) _u = w;
16     e.emplace_back(u, w, c);
17 }
18
19 template<typename T> //requires
20 signed_integral<T>

```

```

21 class flowNetwork {
22     int n, s, t;
23     vector<int> h; // height label
24     vector<T> e; // excess
25     vector<vector<int>>> adj; // adjacent
26     list
27     vector<vector<T>>> c, r;
28     vector<size_t> p; // record the position
29     processed
30     list<int> lst; // topological sort of
31     admissible network
32     void push(int u, int v) {
33         T f{min(e[u], r[u][v])};
34         r[u][v] -= f, r[v][u] += f;
35         e[u] -= f, e[v] += f;
36     }
37     void relabel(int u) {
38         int mn{numeric_limits<int>::max()};
39         for (auto& v : adj[u])
40             if (r[u][v]) mn = min(mn, h[v]);
41         h[u] = mn + 1;
42     }
43     void init() {
44         fill(h.begin(), h.end(), 0);
45         fill(e.begin(), e.end(), 0);
46         lst.clear();
47         for (int u{0}; u < n; ++u) if (u !=
48             s && u != t) lst.push_back(u);
49         fill(p.begin(), p.end(), 0);
50         r = c;
51     }
52     void preflow() {
53         h[s] = n;
54         for (auto& v : adj[s]) {
55             e[v] += r[s][v], e[s] -= r[s][v];
56             r[v][s] += r[s][v], r[s][v] = 0;
57         }
58     }
59     bool discharge(int u) {
60         bool flag{false};
61         while (e[u]) {
62             for (; p[u] < adj[u].size(); ++p
63                 [u]) {
64                 int v{adj[u][p[u]]};
65                 if (r[u][v] && h[u] == h[v]
66                     + 1) push(u, v);
67                 if (!e[u]) break;
68             }
69             if (e[u]) relabel(u), p[u] = 0,
70                 flag = true;
71         }
72         return flag; // return if relabel or
73         not
74     }
75     public:
76     flowNetwork(int _n) : n{_n}, s{0}, t{n -
77         1}, h(n), e(n), adj(n), c(n, vector
78         <T>(n)), p(n) {}
79     void set_st(int _s, int _t) {
80         s = _s, t = _t;
81     }
82     void add_edge(int u, int v, T _c) {
83         if (!c[u][v] && !c[v][u]) adj[u].
84             push_back(v), adj[v].push_back(u)
85             ;
86     }

```

```

87         c[u][v] += _c;
88     }
89     T max_flow() {
90         assert(s != t);
91         init();
92         preflow();
93         auto it{lst.begin()};
94         while (it != lst.end()) {
95             if (discharge(*it)) { // relabel
96                 -to-front
97                 lst.push_front(*it), lst.
98                 erase(it);
99                 it = lst.begin();
100             }
101             it = next(it);
102         }
103         return e[t];
104     };
105 };
106
107 int main() {
108     using T = int;
109
110     // Build an example graph with a vertex
111     capacity on node 2.
112     int n = 4; // nodes: 0 (source), 1, 2, 3
113     (sink)
114     vector<tuple<int, int, T>> edges = {
115         {0, 1, 3},
116         {0, 2, 2},
117         {1, 2, 5},
118         {1, 3, 2},
119         {2, 3, 3}
120     };
121
122     // Enforce vertex capacity of 2 on node
123     2.
124     handle_vertex_capacity<T>(2, 2, n, edges
125         );
126
127     // Build the network and compute max
128     flow.
129     flowNetwork<T> net(n);
130     net.set_st(0, 3);
131     for (auto [u, v, cap] : edges) net.
132         add_edge(u, v, cap);
133
134     cout << "Max flow = " << net.max_flow()
135         << '\n';
136 }

```

6.5 maximum cardinality bipartite matching

```

1 class BipartiteMatching {
2     vector<bool> vis;
3     vector<int> mx{}, my{};

```

```

4     bool dfs(const vector<vector<int>>& adj,
5             int x) {
6         for (int y : adj[x]) {
7             if (vis[y]) continue;
8             vis[y] = true;
9             if (my[y] == -1 || dfs(adj, my[y]
10                 )) {
11                 mx[x] = y, my[y] = x;
12                 return true;
13             }
14         }
15         return false;
16     }
17     public:
18     pair<vector<int>, vector<int>> operator
19     (const vector<vector<int>>& adj,
20         int ny) {
21         vis.assign(ny, false);
22         mx.assign((int)adj.size(), -1);
23         my.assign(ny, -1);
24         for (int x = 0; x < (int)adj.size();
25             ++x) {
26             fill(vis.begin(), vis.end(),
27                 false);
28             dfs(adj, x);
29         }
30         return {mx, my};
31     };
32
33     // Usage example:
34     int main() {
35         // Left partition has 4 nodes (0..3),
36         // right partition has 4 nodes (0..3).
37         vector<vector<int>> adj(4);
38         adj[0] = {0, 1};
39         adj[1] = {1, 2};
40         adj[2] = {2};
41         adj[3] = {2, 3};
42
43         BipartiteMatching bm;
44         auto res = bm(adj, 4);
45         const auto& mx = res.first;
46
47         cout << "Matched pairs (Left -> right):\n";
48         for (int i = 0; i < (int)mx.size(); ++i)
49             if (mx[i] != -1)
50                 cout << i << " -> " << mx[i] <<
51                 "\n";
52     }

```

6.6 Floyd-Warshall

```

1 // Floyd-Warshall algorithm
2 template<typename T>
3 vector<vector<optional<T>>> Floyd_Warshall(
4     const vector<vector<optional<T>>>& adj)
5 {
6     const auto& n{adj.size()};
7     auto d{adj};
8     for (int i{0}; i < n; ++i) d[i][i] = 0;

```

```

8   for (int k{0}; k < n; ++k)
9       for (int i{0}; i < n; ++i)
10          for (int j{0}; j < n; ++j) {
11              if (!d[i][k] || !d[k][j])
12                  continue; // no value
13              if (!d[i][j] || d[i][j] > d[
14                  i][k].value() + d[k][j].
15                  value())
16                  d[i][j] = d[i][k].value
17                      () + d[k][j].value()
18                      ;
19          }
20      }
21      return d;
22  }

```

6.7 MST

```

1  // Kruskal' s algorithm
2  vector<tuple<int, int, long long>> Kruskal(
3      int n, vector<tuple<long long, int, int
4      >>& edges) {
5      vector<tuple<int, int, long long>> mst
6      {};
7      DSU dsu{n};
8      sort(edges.begin(), edges.end());
9      for (auto& [w, u, v] : edges)
10         if (dsu.find(u) != dsu.find(v))
11             dsu.unionn(u, v, mst.
12                 emplace_back(u, v, w));
13     return mst;
14 }

```

6.8 Hopcroft–Karp

```

1  // maximum cardinality bipartite matching in
2  // unweighted bipartite graph
3  // Hopcroft–Karp algorithm - O(EV)
4  class bipartiteGraph {
5      int nx, ny;
6      vector<int> dx, dy;
7      bool dfs(int y) {
8          for (auto& x : adjy[y]) {
9              if (dx[x] + 1 != dy[y]) continue
10             ;
11             if (mx[x] == -1 || dfs(mx[x])) {
12                 my[y] = x, mx[x] = y;
13                 dy[y] = -1; // augmented -
14                 // remove from BFS forest
15                 return true;
16             }
17         }
18         dy[y] = -1; // no augmenting path -
19         // remove from BFS forest
20         return false;
21     }
22     bool bfs() {
23         fill(dx.begin(), dx.end(), -1);
24         fill(dy.begin(), dy.end(), -1);
25     }

```

```

26     queue<int> qu{}, qu2{};
27     for (int x{0}; x < nx; ++x) // use
28         all unmatched x's as roots
29         if (mx[x] == -1) qu.push(x), dx[
30             x] = 0;
31     bool found{false};
32     while (!qu.empty() && !found) { //
33         stop at the level found
34         while (!qu.empty()) {
35             auto x{qu.front()}; qu.pop()
36             ;
37             for (auto& y : adjx[x])
38                 if (dy[y] == -1 && my[y]
39                     != x) {
40                     dy[y] = dx[x] + 1;
41                     if (my[y] == -1)
42                         found = true;
43                     else
44                         qu2.push(my[y]),
45                         dx[my[y]] = dy
46                         [y] + 1;
47                 }
48             }
49             qu.swap(qu2);
50         }
51         return found;
52     }
53     vector<vector<int>> adjx, adjy;
54     public:
55     vector<int> mx, my;
56     bipartiteGraph(int _nx, int _ny) : nx{
57         _nx}, ny{_ny}, dx(nx), dy(ny)
58         , adjx(nx), adjy(ny), mx(nx, -1), my
59         (ny, -1) {}
60     void addEdge(int x, int y) {
61         adjx[x].push_back(y), adjy[y].
62         push_back(x);
63     }
64     int Hopcroft_Karp() {
65         int c{0};
66         while (bfs()) {
67             for (int y{0}; y < ny; ++y)
68                 if (my[y] == -1 && dy[y] !=
69                     -1) c += dfs(y);
70         }
71         return c;
72     }
73     // Usage example:
74     int main() {
75         // Left partition has 4 nodes (0..3),
76         // right partition has 4 nodes (0..3).
77         bipartiteGraph bg(4, 4);
78         bg.addEdge(0, 0);
79         bg.addEdge(0, 1);
80         bg.addEdge(1, 1);
81         bg.addEdge(1, 2);
82         bg.addEdge(2, 2);
83         bg.addEdge(3, 2);
84         bg.addEdge(3, 3);
85     }
86     int max_matching = bg.Hopcroft_Karp();
87     cout << "Maximum matching size: " <<
88         max_matching << "\n";
89 }

```

```

70     cout << "Matched pairs (left -> right):\n";
71     for (int i = 0; i < (int)bg.mx.size();
72         ++i)
73         if (bg.mx[i] != -1)
74             cout << i << " -> " << bg.mx[i]
75             << "\n";
76 }

```

6.9 Dijkstra

```

1  // Dijkstra algorithm
2  template<typename T>
3  vector<optional<T>> Dijkstra(const vector<
4      vector<pair<int, T>>& adj, int s) {
5      const auto& n{adj.size()};
6      vector<optional<T>> d(n, nullopt);
7      d[s] = 0;
8      vector<bool> found(n, false);
9      using pq_t = pair<T, int>;
10     priority_queue<pq_t, vector<pq_t>,
11         greater<pq_t>> pq{};
12     pq.emplace(0, s);
13     while (!pq.empty()) {
14         auto [_, u]{pq.top()}; pq.pop();
15         if (found[u]) continue;
16         found[u] = true;
17         for (auto& [v, w] : adj[u])
18             if (!d[v] || d[v] > d[u].value()
19                 + w) {
20                 d[v] = d[u].value() + w;
21                 pq.emplace(d[v].value(), v);
22             }
23     }
24     return d;
25 }

```

6.10 Dinic

```

1  int n, s, t, cnt, inf = 1e17;
2  vector<vector<int>> adj;
3  int level[100000], ptr[100000], rev[100000],
4  to[100000], cap[100000];
5  void add(int a, int b, int c) {
6      int id = cnt++, rid = cnt++;
7      to[id] = b;
8      to[rid] = a;
9      cap[id] = c;
10     cap[rid] = 0;
11     rev[id] = rid;
12     rev[rid] = id;
13     adj[a].push_back(id);
14     adj[b].push_back(rid);
15 }
16 bool bfs() {
17     memset(level, -1, sizeof(level));
18     queue<int> q;

```

```

19     q.push(s);
20     level[s] = 0;
21     while(!q.empty()) {
22         int curr = q.front(); q.pop();
23         for(int id: adj[curr]) {
24             int next = to[id];
25             if(cap[id] > 0 && level[next] ==
26                 -1) {
27                 level[next] = level[curr] +
28                     1;
29                 q.push(next);
30             }
31         }
32     }
33     return level[t] != -1;
34 }
35 int dfs(int curr, int flow) {
36     if(curr == t || flow == 0) return flow;
37     for(ptr[curr] < adj[curr].size(); ptr[
38         curr]++) {
39         int id = adj[curr][ptr[curr]];
40         int next = to[id];
41         if(cap[id] > 0 && level[next] ==
42             level[curr] + 1) {
43             int temp = dfs(next, min(flow,
44                 cap[id]));
45             if(temp == 0) continue;
46             int rid = rev[id];
47             cap[id] -= temp;
48             cap[rid] += temp;
49             return temp;
50         }
51     }
52     return 0;
53 }
54 int maxflow() {
55     int flow = 0;
56     while(bfs()) {
57         memset(ptr, 0, sizeof(ptr));
58         while(true) {
59             int temp = dfs(s, inf);
60             if(temp == 0) break;
61             flow += temp;
62         }
63     }
64     return flow;
65 }

```

6.11 maximum cardinality matching

```

1  // maximum matching
2  // Edmonds' Blossom algorithm
3  // O(VEa(E,V))
4  class graph {
5      int n;
6      vector<int> p, d, a, c1, c2;
7      // (alternating tree), (unvisited: -1,
8      // even: 0, odd: 1), (auxiliary for lca
9      //), (cross edge)

```

```

8  int label{0};
9  /* DSU */
10 vector<int> dsu_p, dsu_sz, dsu_b;
11 void dsu_reset() {
12     fill(dsu_p.begin(), dsu_p.end(), -1)
13     ;
14     fill(dsu_sz.begin(), dsu_sz.end(), 1);
15     iota(dsu_b.begin(), dsu_b.end(), 0);
16 }
17 int find(int x) { // collapsing find
18     return dsu_p[x] == -1 ? x : dsu_p[x]
19     = find(dsu_p[x]);
20 }
21 int base(int x) { return dsu_b[find(x)];
22 }
23 void contract(int x, int y) { //
24     weighted union
25     auto rx{find(x)}, ry{find(y)};
26     if (rx == ry) return ;
27     auto b{dsu_b[rx]}; // treat x's base
28     as new base
29     if (dsu_sz[rx] < dsu_sz[ry]) swap(rx
30     , ry);
31     dsu_p[ry] = rx, dsu_sz[rx] += dsu_sz
32     [ry], dsu_b[rx] = b;
33 }
34 /*******/
35 queue<int> qu{}; // only even vertices
36 void handle_one_side(int x, int y, int b
37 ) {
38     for (int v{base(x)}; v != b; v =
39     base(p[v])) {
40         c1[v] = x, c2[v] = y, contract(b
41         , v);
42         if (d[v] == 1) qu.push(v); //
43         odd vertex -> even vertex
44     }
45 }
46 int lca(int u, int v) {
47     ++label; // use next number in order
48     not to clear a
49     while (true) {
50         if (a[u] == label) return u;
51         a[u] = label;
52         if (p[u] != -1) u = base(p[u]);
53         swap(u, v);
54     }
55 }
56 void augment(int r, int y) {
57     if (r == y) return ;
58     if (d[y] == 0) { // even vertex ->
59     odd vertex
60     // check d[y] == 0 instead of d[
61     p[y]] == 1, so m[y], y is
62     in the blossom
63     augment(m[y], m[c1[y]]);
64     augment(r, m[c2[y]]);
65     m[c1[y]] = c2[y], m[c2[y]] = c1[
66     y];
67 } else {
68     int x{p[y]};
69     augment(r, m[x]);
70     m[x] = y, m[y] = x;
71 }
72 }

```

```

57 bool bfs(int r) {
58     dsu_reset();
59     fill(d.begin(), d.end(), -1);
60
61     qu = {}, qu.push(r), d[r] = 0;
62     while (!qu.empty()) {
63         int x{qu.front()}; qu.pop();
64         for (auto& y : adj[x]) {
65             if (base(x) == base(y))
66                 continue;
67             if (d[y] == -1) {
68                 p[y] = x, d[y] = 1;
69                 if (m[y] == -1) { //
70                 augmenting path
71                 augment(-1, y);
72                 return true;
73             } else {
74                 p[m[y]] = y, d[m[y]]
75                 = 0;
76                 qu.push(m[y]);
77             }
78             } else if (d[base(y)] == 0)
79             { // blossom
80                 int b{lca(base(x), base(
81                 y))};
82                 handle_one_side(x, y, b)
83                 ;
84                 handle_one_side(y, x, b)
85                 ;
86             }
87         }
88     }
89     return false;
90 }
91 public:
92 vector<vector<int>> adj;
93 vector<int> m;
94 explicit graph(int _n) : n{_n}, p(n, -1)
95 , d(n), a(n), c1(n), c2(n), dsu_p(n)
96 , dsu_sz(n), dsu_b(n), adj(n), m(n,
97 -1) {}
98 int Edmonds() {
99     int c{0};
100     for (int v{0}; v < n; ++v)
101         if (m[v] == -1 && bfs(v)) ++c;
102     return c;
103 }
104 };
105 // usage example:
106 int main() {
107     int n = 5;
108     graph g(n);
109     auto add = [&](int u, int v) {
110         g.adj[u].push_back(v);
111         g.adj[v].push_back(u);
112     };
113     // sample graph
114     add(0, 1);
115     add(0, 2);
116     add(1, 2); // triangle (0,1,2)
117     add(2, 3);
118     add(3, 4);
119
120     int match_size = g.Edmonds();

```

```

113 cout << "Maximum matching size: " <<
114     match_size << "\n";
115 for (int i = 0; i < n; ++i)
116     if (g.m[i] != -1 && i < g.m[i])
117         cout << i << " - " << g.m[i] <<
118         "\n";
119 }

```

6.12 topological sort

```

1 // topological sort 1
2 optional<vector<int>> top_sort(vector<vector
3 <int>>& adj) {
4     vector<int> res{};
5     int n{static_cast<int>(adj.size())};
6     vector<int> cnt(n, 0); // predecessor
7     count
8     for (int u = 0; u < n; ++u)
9         for (auto& v : adj[u]) ++cnt[v];
10
11     queue<int> qu{};
12     for (int u = 0; u < n; ++u) if (!cnt[u])
13         qu.push(u);
14     while (!qu.empty()) {
15         auto u = qu.front();
16         qu.pop();
17         res.push_back(u);
18
19         for (auto& v : adj[u])
20             if (--cnt[v] == 0) qu.push(v);
21     }
22     if (res.size() != adj.size()) return
23     nullopt;
24     return res;

```

6.13 Floyd's algorithm

```

1 // Floyd's Cycle-Finding Algorithm
2 // Given a function succ(x) that returns the
3 // successor of x,
4 a = succ(x);
5 b = succ(succ(x));
6 while (a != b) {
7     a = succ(a);
8     b = succ(succ(b));
9 }
10 // Find the first element in the cycle
11 a = x;
12 while (a != b) {
13     a = succ(a);
14     b = succ(b);
15 }
16 first = a;
17 // Find the length of the cycle
18 length = 1;
19 b = succ(a);
20 while (b != first) {
21     ++length;
22     b = succ(b);
23 }

```

```

21 while (a != b) {
22     b = succ(b);
23     length = length + 1;
24 }
25 // Finding the starting point of the cycle
26 from any node
27 int n;
28 vector<int> adj, res;
29 vector<vector<int>> radj;
30
31 void fill_radj(int u) {
32     for (int v : radj[u]) {
33         if (res[v] == -1) {
34             res[v] = res[u];
35             fill_radj(v);
36         }
37     }
38 }
39
40 void floyd(int x) {
41     int y = x;
42     do { // find a cycle using Floyd's
43     algorithm
44     x = adj[x];
45     y = adj[adj[y]];
46     } while (y != x);
47     do { // set res[x] = x for all x along
48     cycle
49     res[x] = x;
50     x = adj[x];
51     } while (y != x);
52     do { // set res'es for all x not along
53     cycle
54     fill_radj(x);
55     x = adj[x];
56     } while (y != x);
57 }
58
59 void solve() {
60     cin >> n;
61     adj.resize(n);
62     radj.assign(n, {});
63     res.assign(n, -1);
64
65     for (auto& e : adj) {
66         cin >> e;
67         e--;
68     }
69
70     for (int i = 0; i < n; ++i) {
71         radj[adj[i]].push_back(i);
72     }
73
74     for (int i = 0; i < n; ++i) {
75         if (res[i] == -1) {
76             floyd(i);
77         }
78     }
79
80     for (int i = 0; i < n; ++i) {
81         cout << res[i] + 1 << " ";
82     }
83     cout << "\n";
84 }

```


6.14 stable matching

```

1 // Stable Marriage Problem (stable matching)
2 // Gale-Shapley algorithm
3 class bipartiteGraph {
4 public:
5     int n;
6     vector<vector<int>> px, py;
7     vector<int> mx, my;
8
9     bipartiteGraph(int n) : n(n), px(n,
10        vector<int>(n)), py(n, vector<int>(n)),
11        mx(n, -1), my(n, -1) {}
12     void Gale_Shapley() {
13         queue<int> qu{};
14         vector<priority_queue<pair<int, int>>> pq(n);
15         for (int x{0}; x < n; ++x) {
16             qu.push(x);
17             for (int y{0}; y < n; ++y) pq[x]
18                 .emplace(px[x][y], y);
19         }
20         while (!qu.empty()) {
21             auto x{qu.front()}; qu.pop();
22
23             int y;
24             do {
25                 y = pq[x].top().second; pq[x]
26                     .pop();
27             } while (my[y] != -1 && py[y][x]
28                 <= py[y][my[y]]);
29             if (my[y] != -1) mx[my[y]] = -1,
30                 qu.push(my[y]); // y
31                 // prefers x to my[y]
32             mx[x] = y, my[y] = x;
33         }
34     };
35
36     int main() {
37         int n = 3;
38         bipartiteGraph g(n);
39
40         // Men preferences (from most to Least
41         // preferred)
42         vector<vector<int>> men = {
43             {0, 1, 2},
44             {1, 2, 0},
45             {1, 0, 2}
46         };
47
48         // Women preferences (from most to Least
49         // preferred)
50         vector<vector<int>> women = {
51             {1, 0, 2},
52             {2, 1, 0},
53             {0, 1, 2}
54         };
55
56         // Convert orders to scores (higher is
57         // better)
58         for (int x = 0; x < n; ++x)
59             for (int pos = 0; pos < n; ++pos)

```

```

53         g.px[x][men[x][pos]] = n - pos;
54
55         for (int y = 0; y < n; ++y)
56             for (int pos = 0; pos < n; ++pos)
57                 g.py[y][women[y][pos]] = n - pos;
58
59         g.Gale_Shapley();
60
61         // Output matches: man x is matched to
62         // woman g.mx[x]
63         for (int x = 0; x < n; ++x)
64             cout << x << " " << g.mx[x] << '\n';
65
66         return 0;
67     }
68 }

```

6.15 Bellman-Ford

```

1 // Bellman-Ford algorithm
2 template<typename T>
3 optional<vector<optional<T>>> Bellman_Ford(
4     const vector<vector<pair<int, T>>>& adj,
5     int s) {
6     const auto& n{adj.size()};
7     vector<optional<T>> d(n, nullopt);
8     d[s] = 0;
9
10    queue<int> qu{};
11    vector<bool> in(n, false), in2(n, false);
12
13    qu.push(s), in[s] = true;
14    for (int i{0}; i < n; ++i) { // at most
15        // n-1 edges
16        while (!qu.empty()) {
17            int u{qu.front()};
18            qu.pop(), in[u] = false;
19            for (auto& [v, w] : adj[u])
20                if (!d[v] || d[v] > d[u].
21                    value() + w) { // relax
22                    d[v] = d[u].value() + w;
23                    if (!in2[v]) qu2.push(v),
24                        in2[v] = true;
25                }
26            }
27        qu.swap(qu2), in.swap(in2);
28    }
29
30    if (qu.empty()) return d;
31    return nullopt; // if negative cycle
32 }

```

7 Language

7.1 CNF

```

1 #define MAXN 55
2 struct CNF{

```

```

3     int s,x,y;//s->xy | s->x, if y==1
4     int cost;
5     CNF(){}
6     CNF(int s,int x,int y,int c):s(s),x(x),y(y),cost(c){}
7 };
8
9 int state;//規則數量
10 map<char,int> rule;//每個字元對應到的規則
11 //小寫字母為終端字符
12 vector<CNF> cnf;
13 void init(){
14     state=0;
15     rule.clear();
16     cnf.clear();
17 }
18
19 void add_to_cnf(char s,const string &p,int cost){
20     //加入一個s -> <p>的文法，代價為cost
21     if(rule.find(s)==rule.end())rule[s]=state++;
22     for(auto c:p){if(rule.find(c)==rule.end())
23         rule[c]=state++;
24     }
25     if(p.size()==1){
26         cnf.push_back(CNF(rule[s],rule[p[0]],-1,cost));
27     }else{
28         int left=rule[s];
29         int sz=p.size();
30         for(int i=0;i<sz-2;++i){
31             cnf.push_back(CNF(left,rule[p[i]],state,0));
32             left=state++;
33         }
34         cnf.push_back(CNF(left,rule[p[sz-2]],rule[p[sz-1]],cost));
35     }
36 }
37
38 vector<long long> dp[MAXN][MAXN];
39 vector<bool> neg_INF[MAXN][MAXN]; //如果花費
40 //是負的可能會有無限小的情形
41 void relax(int l,int r,const CNF &c,long long cost,bool neg_c=0){
42     if(!neg_INF[l][r][c.s]&&(neg_INF[l][r][c.x]||cost<dp[l][r][c.s])){
43         if(neg_c||neg_INF[l][r][c.x]){
44             dp[l][r][c.s]=0;
45             neg_INF[l][r][c.s]=true;
46         }else dp[l][r][c.s]=cost;
47     }
48 }
49
50 void bellman(int l,int r,int n){
51     for(int k=1;k<=state;++k)
52         for(auto c:cnf)
53             if(c.y==1)relax(l,r,c,dp[l][r][c.x]+c.cost,k=n);
54 }
55
56 void cyk(const vector<int> &tok){
57     for(int i=0;i<(int)tok.size();++i){
58         for(int j=0;j<(int)tok.size();++j){
59             dp[i][j]=vector<long long>(state+1,INT_MAX);
60             neg_INF[i][j]=vector<bool>(state+1,false);
61         }
62     }
63     dp[i][i][tok[i]]=0;
64 }

```

```

54     bellman(i,i,tok.size());
55 }
56 for(int r=1;r<(int)tok.size();++r){
57     for(int l=r-1;l>=0;--l){
58         for(int k=l;k<r;++k)
59             for(auto c:cnf)
60                 if(~c.y)relax(l,r,c,dp[l][k][c.x]+dp[k+1][r][c.y]+c.cost);
61     }
62     bellman(l,r,tok.size());
63 }
64 }

```

8 Number Theory

8.1 Linear Sieve

```

1 // Calculate the smallest divisor of
2 // integers in [2, maxn] in O(maxn)
3 vector<int> min_div{[]};
4 constexpr int maxn = 400000 + 10;
5
6 vector<int> v(maxn), p;
7
8 for (int i = 2; i < maxn; ++i) {
9     if (!v[i]) {
10         v[i] = i;
11         p.push_back(i);
12     }
13     for (int j = 0; p[j] * i < maxn; ++j) {
14         v[p[j] * i] = p[j];
15         if (p[j] == v[i]) break;
16     }
17 }
18 return v;
19 }();

```

8.2 C(n,m)

```

1 ll Cnm(ll n, ll m) {
2     if (m > n / 2) m = n - m;
3     ll r{1};
4     for (ll i{1}, j{n}; i <= m; ++i, --j) r
5         *= j, r /= i;
6     return r;
7 }

```

8.3 Euler Totient 1 to n

```

1 void phi_1_to_n(int n) {
2     vector<int> phi(n + 1);
3     for (int i = 0; i <= n; i++)
4         phi[i] = i;
5 }

```

```

5   for (int i = 2; i <= n; i++) {
6       if (phi[i] == i) {
7           for (int j = i; j <= n; j += i)
8               phi[j] -= phi[j] / i;
9       }
10  }
11  }
12  }

```

8.4 derangement (Principle of Inclusion-Exclusion)

```

1  // 1. Principle of Inclusion-Exclusion
2  // n! = n! * Σ (from k=0 to n) [((-1)^k) / (k!)]
3
4  mint c = 1;
5  for (int i = 1; i <= n; i++) {
6      c = (c * i) + (i % 2 == 1 ? -1 : 1);
7      cout << c.val() << ' ';
8  }

```

8.5 matrix template (with fast power)

```

1  template<class T> struct Matrix {
2      T **mat; int a, b;
3
4      Matrix() : a(0), b(0) {}
5      Matrix(int a_, int b_) : a(a_), b(b_) {
6          int i, j;
7          mat = new T*[a];
8          for (i = 0; i < a; ++i) {
9              mat[i] = new T[b];
10             for (j = 0; j < b; ++j) {
11                 mat[i][j] = 0;
12             }
13         }
14     }
15     Matrix(const vector<vector<T>>& vt) {
16         int i, j;
17
18         *this = Matrix((int)vt.size(), (int)
19             vt[0].size());
20         for (i = 0; i < a; ++i) {
21             for (j = 0; j < b; ++j) {
22                 mat[i][j] = vt[i][j];
23             }
24         }
25     }
26     Matrix operator*(const Matrix& m) {
27         int i, j, k;
28
29         assert(b == m.a);
30         Matrix r(a, m.b);
31         for (i = 0; i < a; ++i) {
32             for (j = 0; j < m.b; ++j) {
33                 for (k = 0; k < b; ++k) {

```

```

33             r.mat[i][j] += mul(mat[i]
34                 ][k], m.mat[k][j]);
35             r.mat[i][j] %= MOD;
36         }
37     }
38     return r;
39 }
40 Matrix& operator*=(const Matrix& m) {
41     return *this = (*this) * m;
42 }
43 friend Matrix power(Matrix m, long long
44     p) {
45     int i;
46
47     assert(m.a == m.b);
48     Matrix r(m.a, m.b);
49     for (i = 0; i < m.a; ++i) {
50         r.mat[i][i] = 1;
51     }
52     for (; p > 0; p >= 1, m *= m) {
53         if (p & 1) {
54             r *= m;
55         }
56     }
57     return r;
58 }
59
60 int main() {
61     Matrix<int> mat(adj);
62     mat = power(mat, k); // mat^k
63     cout << mat.mat[i][j];
64 }

```

8.6 Sieve of Eratosthenes (with big num)

```

1  const int MX = 100000;
2  bool np[MX + 1];
3  vector<int> prime_numbers;
4
5  int init = []() {
6      np[0] = np[1] = true;
7      for (int i = 2; i <= MX; i++) {
8          if (!np[i]) {
9              prime_numbers.push_back(i);
10             for (int j = i; j <= MX / i; j
11                 ++i) { // 避免溢出的写法
12                 np[i * j] = true;
13             }
14         }
15     }
16     return 0;
17 }();
18 bool is_prime(long long n) {
19     if (n <= MX) {
20         return !np[n];
21     }
22     for (long long p : prime_numbers) {
23         if (p * p > n) {

```

```

24         break;
25     }
26     if (n % p == 0) {
27         return false;
28     }
29 }
30 return true;
31 }

```

8.7 mod inv

```

1  // Modular inverse when mod is prime
2  long long mod_inverse(long long a, long long
3      mod) {
4      return power_mod(a, mod - 2, mod); //
5      Fermat's Little Theorem
6  }

```

8.8 derangement (DP)

```

1  // 2. DP
2  // !n = (n - 1) * (!n - 1) + !(n - 2)),
3  // with !0 = 1, !1 = 0
4
5  mint a = 1, b = 0;
6  cout << 0 << ' ';
7
8  for (int i = 2; i <= n; i++) {
9      mint c = (i - 1) * (a + b);
10     cout << c.val() << ' ';
11     a = b;
12     b = c;
13 }

```

8.9 Euler Totient

```

1  int phi(int n) {
2      int result = n;
3      for (int i = 2; i * i <= n; i++) {
4          if (n % i == 0) {
5              while (n % i == 0)
6                  n /= i;
7              result -= result / i;
8          }
9      }
10     if (n > 1)
11         result -= result / n;
12     return result;
13 }

```

8.10 fast power

```

1  /* Iterative Function to calculate (a^b) %
2     mod in O(log b) */
3  long long power_mod(long long a, long long b
4      , long long mod) {
5      long long res{1};
6      while (b > 0) {
7          if (b & 1) res = res * a % mod;
8          b >>= 1;
9          a = (a * a) % mod;
10     }
11     return res;
12 }

```

8.11 first and second mex

```

1  // Calculate first and second MEX
2  pair<int, int> calculate_mexes(vector<int>&
3      nums) {
4      int n = nums.size();
5      vector<bool> seen(n + 2, false);
6
7      for (int num : nums) {
8          if (num >= 0 && num < seen.size()) {
9              seen[num] = true;
10          }
11      }
12
13     int first_mex = -1;
14     int second_mex = -1;
15
16     for (int i = 0; i < seen.size(); ++i) {
17         if (!seen[i]) {
18             if (first_mex == -1) {
19                 first_mex = i;
20             } else {
21                 second_mex = i;
22                 break;
23             }
24         }
25     }
26
27     return {first_mex, second_mex};
28 }

```

8.12 Catalan Number

```

1  // Catalan Number
2  // C(n) = 1/(n+1) * (2n choose n) = (2n)! /
3     ((n+1)! * n!)
4
5  ll catalan(int n) {
6      return fact[2 * n] * invf[n + 1] % mod *
7         invf[n] % mod;
8  }

```

8.13 Chinese Remainder Theorem

```

1 // coprime (p^k)
2 pair<ll, ll> CRT(const vector<pair<ll, ll>&
  congruences) {
3     ll M = 1, sol = 0;
4     for (auto& [m, a] : congruences) M *= m;
5     for (auto& [m, a] : congruences) {
6         ll x = M / m, y = MI(x, m);
7         sol = MA(sol, MM(MM(a, x, M), y, M),
8             M);
9     }
10    return {M, sol};
11 }
12 // example usage
13 int main() {
14     // x ≡ 2 (mod 3)
15     // x ≡ 3 (mod 5)
16     // x ≡ 2 (mod 7)
17     vector<pair<ll, ll>> congruences = {{3,
18         2}, {5, 3}, {7, 2}};
19     auto [M, sol] = CRT(congruences);
20     cout << "M: " << M << ", x: " << sol <<
21     endl; // M: 105, x: 23
22 }

```

8.14 mod inv (not prime)

```

1 /* exists when a and mod are coprime */
2 /* but mod is not prime */
3 long long MI(long long a, long long mod) {
4     return power_mod(a, euler_phi(mod) - 1,
5         mod);
6 }

```

8.15 mod inv (not coprime)

```

1 /* a and mod are not coprime */
2 long long MI(long long a, long long mod) {
3     long long d, x, y;
4     extEcu(a, mod, d, x, y);
5     return d == 1 ? (x + mod) % mod : -1;
6 }

```

8.16 Mint

```

1 using ll = long long;
2 const ll PM{1000000007};
3 ll MC(ll a) { return (a % PM + PM) % PM; }
4 // for negative a
5 ll MA(ll a, ll b) { return (a + b) % PM; }
6 // (a + b) % PM
7 ll MS(ll a, ll b) { return (a - b + PM) % PM; }
8 // (a - b) % PM

```

```

6 ll MM(ll a, ll b) { return (a * b) % PM; }
7 // (a * b) % PM
8 ll MP(ll a, ll b) { // (a ^ b) % PM
9     ll res{1};
10    do {
11        if (b & 1) res = MM(res, a);
12        while (a = MM(a, a), b >= 1);
13        return res;
14    }
15    ll MI(ll a) { return MP(a, PM - 2); } //
16    modular inverse of a % PM
17 ll MD(ll a, ll b) { return MM(a, MI(b)); }
18 // (a / b) % PM

```

8.17 inclusion-exclusion principle

```

1 /*
2 inclusion-exclusion principle:  $U A = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots$ 
3
4 if  $f(n) = \sum g(d)$  for all  $d|n$ 
5 then  $g(n) = \sum \mu(d) * f(n/d)$  for all  $d|n$ 
6
7 where  $\mu(d)$  is the mobius function defined as :
8
9  $|U A| = \sum \mu(d) * |A_d|$ 
10 where  $A_d = \{x \in U : d|x\}$ 
11  $\mu(d) = 1$  if  $d$  is a product of even number of
12 distinct primes
13  $\mu(d) = -1$  if  $d$  is a product of odd number of
14 distinct primes
15  $\mu(d) = 0$  if  $d$  has a squared prime factor
16 */
17 //
18 problem: In number theory we call an integer
19 square-free if it is not divisible by a
20 perfect square, except 1. You have to
21 count them!
22 square-free number: An integer  $n$  is square-
23 free if no  $p^2$  divides  $n$ .
24
25 /*
26 const int VALMAX = 1e7;
27
28 int mobius[VALMAX];
29
30 int main() {
31     int test_num;
32     cin >> test_num;
33
34     mobius[1] = -1;
35     for (int i = 1; i < VALMAX; i++) {
36         if (mobius[i]) {
37             mobius[i] = -mobius[i];
38             for (int j = 2 * i; j < VALMAX; j += i)
39                 mobius[j] += mobius[i];
40         }
41     }
42
43     for (int t = 0; t < test_num; t++) {
44         long long n;

```

```

38 long long ans = 0;
39 cin >> n;
40 for (int i = 1; 1LL * i * i <= n; i++) {
41     ans += mobius[i] * n / ((long long)i *
42         i);
43 }
44 cout << ans << '\n';
45 }
46
47 /*
48 problem: Given  $N$  cows ( $2 \leq N \leq 50,000$ ),
49 each listing five favorite ice cream
50 flavors,
51 where two cows are compatible if they share
52 at least one flavor,
53 determine the number of cow pairs that are
54 not compatible.
55
56 ans =  $n * (n - 1) / 2 - \sum |A_i| + \sum |A_i \cap A_j| - \sum |A_i \cap A_j \cap A_k| + \dots$ 
57 where  $A_i$  is the set of cows that like
58 flavor  $i$ 
59 */
60 void solve() {
61     int n;
62     cin >> n;
63     vector<array<int, 5>> vc(n);
64
65     map<vector<int>, int> subsets;
66     for (int i = 0; i < n; ++i) {
67         for (int j = 0; j < 5; ++j) {
68             cin >> vc[i][j];
69         }
70
71         sort(vc[i].begin(), vc[i].end());
72         for (int mask = 1; mask < (1<<5); ++
73             mask) {
74             vector<int> subset;
75             for (int b = 0; b < 5; ++b) {
76                 if (mask & (1<<b)) {
77                     subset.push_back(vc[i][b]);
78                 }
79             }
80             subsets[subset]++;
81         }
82     }
83
84     int ans = n * (n - 1) / 2;
85     for (const auto& [subset, freq] :
86         subsets) {
87         ans -= (subset.size() % 2 == 1 ? 1 :
88             -1) * freq * (freq - 1) / 2;
89     }
90     cout << ans << '\n';
91 }

```

8.18 C(n,k) mod inverse

```

1 fac[0] = 1;
2 for (int i = 1; i <= n; ++i) {

```

```

3     fac[i] = fac[i - 1] * i % MOD;
4 }
5
6 inv_fac[n] = power_mod(fac[n], MOD - 2, MOD);
7
8 for (int i = n - 1; i >= 0; --i) {
9     inv_fac[i] = inv_fac[i + 1] * (i + 1) %
10     MOD;
11 }
12
13 //  $C(n, k) = fac[n] * inv_fac[k] * inv_fac[n - k];$ 

```

8.19 Linear Diophantine 丟番圖

```

1 // ax + by = c
2 // x0 = x * (c / gcd(a, b))
3 // y0 = y * (c / gcd(a, b))
4 // x = x0 + k * (b / gcd(a, b))
5 // y = y0 - k * (a / gcd(a, b))
6 // k is any integer
7
8 tll extendedEuclid(ll a, ll b) {
9     if (b == 0) return {a, 1, 0};
10    ll d, x1, y1;
11    tie(d, x1, y1) = extendedEuclid(b, a % b);
12    return {d, y1, x1 - (a / b) * y1};
13 }
14
15 pll linearDiophantine(ll a, ll b, ll c) {
16     ll d, x, y;
17     tie(d, x, y) = extendedEuclid(a, b);
18     if (c % d != 0) return {-1, -1}; // no
19     solution
20     x *= c / d;
21     y *= c / d;
22     return {x, y};
23 }
24 // Example usage
25 int main() {
26     ll a = 15, b = 10, c = 35;
27     pll res = linearDiophantine(a, b, c);
28     if (res.first == -1 && res.second == -1)
29         cout << "No solution exists" << endl;
30     else {
31         cout << "One solution is x = " <<
32         res.first << ", y = " <<
33         res.second << endl;
34     }
35     return 0;
36 }
37 ax + by = c

```

8.20 擴展歐基里德

```

1 /* solve x, y for ax + by = gcd(a, b) = g */
2 template<typename T>
3 void extEcu(T a, T b, T &g, T &x, T &y) {
4     if (b) extEcu(b, a % b, g, y, x), y -= x
5         * (a / b);
6     else g = a, x = 1, y = 0;
7 }

```

8.21 Sieve of Eratosthenes

```

1 void sieve(vector<int>& primes) {
2     vector<int> is_prime(INF + 1, 0);
3     is_prime[0] = 1;
4     is_prime[1] = 1;
5
6     int sq = sqrt(INF);
7
8     for (int i = 2; i <= sq; ++i) {
9         if (!is_prime[i]) {
10             primes.push_back(i);
11             for (int j = i * i; j <= INF; j
12                 += i) {
13                 is_prime[j] = 1;
14             }
15         }
16     }
17 }

```

8.22 builtin

```

1 // __builtin_ctz
2 // 這個函數作用是返回輸入數二進制表示從最低
3 // 位開始 (右起) 的連續的0的個數; 如果傳入0
4 // 則行為未定義。
5
6 int __builtin_ctz (unsigned int x)
7 Returns the number of trailing 0-bits in x,
8 starting at the least significant bit
9 position. If x is 0, the result is
10 undefined.
11
12 int __builtin_ctzl (unsigned long)
13 Similar to __builtin_ctz, except the
14 argument type is unsigned long.
15
16 int __builtin_ctzll (unsigned long long)
17 Similar to __builtin_ctz, except the
18 argument type is unsigned long long.
19
20 // __builtin_clz
21 // 這個函數作用是返回輸入數二進制表示從最高
22 // 位開始 (左起) 的連續的0的個數; 如果傳入0
23 // 則行為未定義。
24
25 int __builtin_clz (unsigned int x)
26 Returns the number of leading 0-bits in x,
27 starting at the most significant bit
28 position. If x is 0, the result is
29 undefined.

```

```

19 int __builtin_clzl (unsigned long)
20 Similar to __builtin_clz, except the
21 argument type is unsigned long.
22
23 int __builtin_clzll (unsigned long long)
24 Similar to __builtin_clz, except the
25 argument type is unsigned long long.
26
27 // __builtin_ffs
28 // 這個函數作用是返回輸入數二進制表示的最低
29 // 非0位的下標。下標從1開始計數; 如果傳入0
30 // 則返回0
31
32 int __builtin_ffs (unsigned int x)
33 Returns one plus the index of the least
34 significant 1-bit of x, or if x is zero,
35 returns zero.
36
37 int __builtin_ffsl (unsigned long)
38 Similar to __builtin_ffs, except the
39 argument type is unsigned long.
40
41 int __builtin_ffsll (unsigned long long)
42 Similar to __builtin_ffs, except the
43 argument type is unsigned long long.
44
45 // __builtin_popcount
46 // 這個函數作用是返回輸入數二進制表示中1的個
47 // 數。
48
49 int __builtin_popcount (unsigned int x)
50 Returns the number of 1-bits in x.
51
52 int __builtin_popcountl (unsigned long)
53 Similar to __builtin_popcount, except the
54 argument type is unsigned long.
55
56 int __builtin_popcountll (unsigned long long)
57 Similar to __builtin_popcount, except the
58 argument type is unsigned long long.
59
60 // __builtin_parity
61 // 這個函數作用是返回輸入數二進制表示中1的個
62 // 數的奇偶性。1表示奇數個1。0表示偶數個1。
63
64 int __builtin_parity (unsigned int x)
65 Returns the parity of x, i.e. the number of
66 1-bits in x modulo 2.
67
68 int __builtin_parityl (unsigned long)
69 Similar to __builtin_parity, except the
70 argument type is unsigned long.
71
72 int __builtin_parityll (unsigned long long)
73 Similar to __builtin_parity, except the
74 argument type is unsigned long long.

```

8.23 C(n,k) DP

```

1 long long binomial(long long n, long long k,
2 long long p) {
3     // dp[i][j] = iCj
4     vector<vector<long long>> dp(n + 1,
5         vector<long long>(k + 1, 0));
6
7     for (int i = 0; i <= n; ++i) {
8         dp[i][0] = 1;
9         if (i <= k) dp[i][i] = 1;
10    }
11
12    for (int i = 0; i <= n; ++i) {
13        for (int j = 1; j <= min(i, k); ++j) {
14            if (i != j) {
15                dp[i][j] = (dp[i - 1][j - 1]
16                    + dp[i - 1][j]) % p;
17            }
18        }
19    }
20
21    return dp[n][k];
22 }

```

9 Range Queries

9.1 subarray maximum sum queries

```

1 // CSES: Subarray Sum Queries
2
3 const int maxn = 2e5;
4 int n = 0;
5 int t[maxn << 1];
6
7 void build() {
8     for (int i = 0; i < n; ++i) {
9         t[i + n] = 0;
10    }
11    for (int i = n - 1; i >= 1; --i) {
12        t[i] = t[i << 1] + t[i << 1 | 1];
13    }
14 }
15
16 void update(int idx, int val) {
17     idx += n;
18     if (t[idx] == val) return;
19     t[idx] = val;
20     for (idx >= 1; idx >= 1; idx >= 1) {
21         t[idx] = t[idx << 1] + t[idx << 1 |
22             1];
23     }
24 }
25
26 int query(int l, int r) {
27     int ans = 0;
28     for (l += n, r += n; l < r; l >= 1, r
29         >= 1) {
30         if (l & 1) ans += t[l++];

```

```

29     if (r & 1) ans += t[--r];
30 }
31 return ans;
32 }
33
34 void solve() {
35     int q;
36     cin >> q;
37     n = 0;
38     vector<pii> op(q);
39     map<int, int> mp;
40     vector<int> to_val;
41
42     memset(t, 0, sizeof(t));
43
44     for (int i = 0; i < q; ++i) {
45         char c;
46         cin >> c >> op[i].second;
47         if (c == 'I') {
48             op[i].first = 0;
49             mp[op[i].second] = 1;
50         }
51         else if (c == 'D') {
52             op[i].first = 1;
53             mp[op[i].second] = 1;
54         }
55         else if (c == 'K') {
56             op[i].first = 2;
57         }
58         else {
59             op[i].first = 3;
60         }
61     }
62
63     for (auto& x : mp) {
64         x.second = n;
65         to_val.push_back(x.first);
66         n++;
67     }
68
69     build();
70
71     for (pii o : op) {
72         if (o.first == 0) {
73             int idx = mp[o.second];
74             update(idx, 1);
75         }
76         if (o.first == 1) {
77             int idx = mp[o.second];
78             update(idx, 0);
79         }
80         if (o.first == 2) {
81             int k = o.second;
82             int ans = -1;
83             int l = 0, r = n - 1;
84             while (l < r) {
85                 int m = l + (r - l) / 2;
86                 int rk = query(0, m + 1);
87                 if (rk >= k) {
88                     ans = m;
89                     r = m - 1;
90                 }
91                 else {
92                     l = m + 1;
93                 }
94             }

```

```

95     if (ans == -1) cout << "invalid\n";
96     else cout << to_val[ans] << "\n";
97     ;
98 }
99 if (o.first == 3) {
100     auto it = mp.lower_bound(o.second);
101     if (it == mp.begin() && (it == mp.end() || o.second <= it->first)) {
102         cout << "0\n";
103     }
104     else if (it == mp.end()) {
105         cout << query(0, n) << "\n";
106     }
107     else {
108         cout << query(0, it->second) << "\n";
109     }
110 }
111 }
112 }

```

```

35 void solve() {
36     cin >> n >> m;
37     vector<int> a(n);
38
39     for (int i = 0; i < n; ++i) {
40         cin >> a[i];
41     }
42
43     while (sz < n) sz <= 1;
44     memset(t, 0, sizeof(t));
45     build(a);
46
47     int r;
48     while (m--) {
49         cin >> r;
50         int idx = query(r);
51         cout << idx << " ";
52         if (idx) {
53             a[idx - 1] -= r;
54             update(idx - 1, a[idx - 1]);
55         }
56     }
57     cout << "\n";
58 }
59 }

```

9.2 find first equal or greater

```

1 // CSE: Hotel Queries
2
3 int n, m;
4 int sz = 1;
5 const int maxn = 2e6;
6 int t[maxn << 1];
7
8 void build(const vector<int>& a) {
9     for (int i = 0; i < n; ++i) {
10         t[i + sz] = a[i];
11     }
12     for (int i = sz - 1; i >= 1; --i) {
13         t[i] = max(t[i << 1], t[i << 1 | 1]);
14     }
15 }
16
17 void update(int i, int val) {
18     i += sz;
19     t[i] = val;
20     for (i >= 1; i >= 1; i >= 1) {
21         t[i] = max(t[i << 1], t[i << 1 | 1]);
22     }
23 }
24
25 int query(int r) {
26     if (t[1] < r) return 0;
27
28     int i = 1;
29     while (i < sz) {
30         if (t[i << 1] >= r) i = i << 1;
31         else i = i << 1 | 1;
32     }
33     return i - sz + 1;
34 }

```

9.3 find k-th and count less than

```

1 /*
2 SPOJ: ORDERSET - Order statistic set
3
4 In this problem, you have to maintain a
5 dynamic set of numbers which support the
6 two fundamental operations
7
8 INSERT(S,x): if x is not in S, insert x into S
9 DELETE(S,x): if x is in S, delete x from S
10 and the two type of queries
11
12 K-TH(S) : return the k-th smallest element of S
13 COUNT(S,x): return the number of elements of S smaller than x
14
15 */
16 const int maxn = 2e5;
17 int n = 0;
18 int t[maxn << 1];
19
20 void build() {
21     for (int i = 0; i < n; ++i) {
22         t[i + n] = 0;
23     }
24     for (int i = n - 1; i >= 1; --i) {
25         t[i] = t[i << 1] + t[i << 1 | 1];
26     }
27 }
28
29 void update(int idx, int val) {
30     idx += n;
31     if (t[idx] == val) return;
32     t[idx] = val;

```

```

31     for (idx >= 1; idx >= 1; idx >= 1) {
32         t[idx] = t[idx << 1] + t[idx << 1 | 1];
33     }
34 }
35
36 int query(int l, int r) {
37     int ans = 0;
38     for (l += n, r += n; l < r; l >= 1, r >= 1) {
39         if (l & 1) ans += t[l++];
40         if (r & 1) ans += t[--r];
41     }
42     return ans;
43 }
44
45 void solve() {
46     int q;
47     cin >> q;
48     n = 0;
49     vector<pii> op(q);
50     map<int, int> mp;
51     vector<int> to_val;
52
53     memset(t, 0, sizeof(t));
54
55     for (int i = 0; i < q; ++i) {
56         char c;
57         cin >> c >> op[i].second;
58         if (c == 'I') {
59             op[i].first = 0;
60             mp[op[i].second] = 1;
61         }
62         else if (c == 'D') {
63             op[i].first = 1;
64             mp[op[i].second] = 1;
65         }
66         else if (c == 'K') {
67             op[i].first = 2;
68         }
69         else {
70             op[i].first = 3;
71         }
72     }
73
74     for (auto& x : mp) {
75         x.second = n;
76         to_val.push_back(x.first);
77         n++;
78     }
79
80     build();
81
82     for (pii o : op) {
83         if (o.first == 0) {
84             int idx = mp[o.second];
85             update(idx, 1);
86         }
87         if (o.first == 1) {
88             int idx = mp[o.second];
89             update(idx, 0);
90         }
91         if (o.first == 2) {
92             int k = o.second;
93             int ans = -1;
94             int l = 0, r = n - 1;

```

```

95     while (l <= r) {
96         int m = l + (r - l) / 2;
97         int rk = query(0, m + 1);
98         //printf("(m, rk): %lld, %lld\n", m, rk);
99         if (rk >= k) {
100             ans = m;
101             r = m - 1;
102         }
103         else {
104             l = m + 1;
105         }
106     }
107     if (ans == -1) cout << "invalid\n";
108     else cout << to_val[ans] << "\n";
109     ;
110 }
111 if (o.first == 3) {
112     auto it = mp.lower_bound(o.second);
113     if (it == mp.begin() && (it == mp.end() || o.second <= it->first)) {
114         cout << "0\n";
115     }
116     else if (it == mp.end()) {
117         cout << query(0, n) << "\n";
118     }
119     else {
120         cout << query(0, it->second) << "\n";
121     }
122 }
123 }
124 }

```

10 String

10.1 Z

```

1 // 计算并返回 z 数组，其中 z[i] = LCP(s[i:], s)
2 vector<int> calc_z(const string& s) {
3     int n = s.size();
4     vector<int> z(n);
5     int box_l = 0, box_r = 0;
6     for (int i = 1; i < n; i++) {
7         if (i <= box_r) {
8             z[i] = min(z[i - box_l], box_r - i + 1);
9         }
10        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
11            box_l = i;
12            box_r = i + z[i];
13            z[i]++;
14        }
15    }
16    z[0] = n;

```



```
17     return z;
18 }
```

10.2 manacher

```
1 class Solution {
2 public:
3     int countSubstrings(string s) {
4         int l1 = s.size(), l2 = l1 * 2 + 1;
5         string ch = "#";
6         for(char c: s) {
7             ch = ch + c + "#";
8         }
9
10        int c = 0, r = 0, cnt = 0;
11        vector<int> p(l2);
12        for(int i = 0; i < l2; i++) {
13            p[i] = (i < r)? min(p[2 * c - i], r - i): 1;
14            while(i + p[i] < l2 && i - p[i]
15                >= 0 && ch[i + p[i]] == ch[i - p[i]]) p[i]++;
16            if(i + p[i] > r) {
17                r = i + p[i];
18                c = i;
19            }
20            int l = p[i] - 1;
21            if(l % 2 == 0) cnt += 1 / 2;
22            else cnt += 1 / 2 + 1;
23        }
24        return cnt;
25    };
26 }
```

10.3 AC 自動機

```
1 template<char L='a',char R='z'>
2 class ac_automaton{
3     struct joe{
4         int next[R-L+1], fail, efl, ed, cnt_dp, vis;
5         joe():ed(0), cnt_dp(0), vis(0){
6             for(int i=0; i<=R-L; ++i) next[i]=0;
7         }
8     };
9     public:
10    std::vector<joe> S;
11    std::vector<int> q;
12    int qs, qe, vt;
13    ac_automaton():S(1), qs(0), qe(0), vt(0){}
14    void clear(){
15        q.clear();
16        S.resize(1);
17        for(int i=0; i<=R-L; ++i) S[0].next[i]=0;
18        S[0].cnt_dp=S[0].vis=qs=qe=vt=0;
19    }
20    void insert(const char *s){
21        int o=0;
22        for(int i=0, id; s[i]; ++i){
23            id=s[i]-L;
24            if(!S[o].next[id]){
```

```
25        S.push_back(joe());
26        S[o].next[id]=S.size()-1;
27    }
28    o=S[o].next[id];
29 }
30 ++S[o].ed;
31 }
32 void build_fail(){
33     S[0].fail=S[0].efl=-1;
34     q.clear();
35     q.push_back(0);
36     ++qe;
37     while(qs!=qe){
38         int pa=q[qs++], id, t;
39         for(int i=0; i<=R-L; ++i){
40             t=S[pa].next[i];
41             if(!t) continue;
42             id=S[pa].fail;
43             while(~id && S[id].next[i]) id=S[id].fail;
44             S[t].fail=~id?S[id].next[i]:0;
45             S[t].efl=S[S[t].fail].ed?S[t].fail:S[S[t].fail].efl;
46             q.push_back(t);
47             ++qe;
48         }
49     }
50 }
51 /*DP出每個前綴在字串s出現的次數並傳回所有
52 字串被s匹配成功的次數O(N*M)*/
53 int match_0(const char *s){
54     int ans=0, id, p=0, t;
55     for(i=0; s[i]; ++i){
56         id=s[i]-L;
57         while(!S[p].next[id] && p) p=S[p].fail;
58         if(!S[p].next[id]) continue;
59         p=S[p].next[id];
60         ++S[p].cnt_dp; /*匹配成功則它所有後綴都
61 可以被匹配(DP計算)*/
62     }
63     for(i=qe-1; i>=0; --i){
64         ans+=S[q[i]].cnt_dp*S[q[i]].ed;
65         if(~S[q[i]].fail) S[S[q[i]].fail].cnt_dp+=S[q[i]].cnt_dp;
66     }
67     return ans;
68 }
69 /*多串匹配走efl邊並傳回所有字串被s匹配成功
70 的次數O(N*M^1.5)*/
71 int match_1(const char *s) const{
72     int ans=0, id, p=0, t;
73     for(int i=0; s[i]; ++i){
74         id=s[i]-L;
75         while(!S[p].next[id] && p) p=S[p].fail;
76         if(!S[p].next[id]) continue;
77         p=S[p].next[id];
78         if(S[p].ed) ans+=S[p].ed;
79         for(t=S[p].efl; ~t; t=S[t].efl){
80             ans+=S[t].ed; /*因為都走efl邊所以保證
81 匹配成功*/
82         }
83     }
84     return ans;
85 }
```

```
82 /*枚舉(s的子串串nA)的所有相異字串各恰一次
83 並傳回次數O(N*M^(1/3))*/
84 int match_2(const char *s){
85     int ans=0, id, p=0, t;
86     ++vt;
87     /*把戳記vt+=1，只要vt沒溢位，所有S[p].vis==vt就會變成false
88     這種利用vt的方法可以O(1)歸零vis陣列*/
89     for(int i=0; s[i]; ++i){
90         id=s[i]-L;
91         while(!S[p].next[id] && p) p=S[p].fail;
92         if(!S[p].next[id]) continue;
93         p=S[p].next[id];
94         if(S[p].ed && S[p].vis!=vt){
95             S[p].vis=vt;
96             ans+=S[p].ed;
97         }
98         for(t=S[p].efl; ~t && S[t].vis!=vt; t=S[t].efl){
99             S[t].vis=vt;
100             ans+=S[t].ed; /*因為都走efl邊所以保證
101 匹配成功*/
102         }
103     }
104     /*把AC自動機變成真的自動機*/
105     void evolution(){
106         for(qs=1; qs!=qe;){
107             int p=q[qs++];
108             for(int i=0; i<=R-L; ++i)
109                 if(S[p].next[i]==0) S[p].next[i]=S[S[p].fail].next[i];
110         }
111     }
112 }
```

10.4 KMP

```
1 // 在文本串 text 中查找模式串 pattern，返回
2 所有成功匹配的位置 (pattern[0] 在 text
3 中的下标)
4 vector<int> kmp(const string& text, const
5 string& pattern) {
6     int m = pattern.size();
7     vector<int> pi(m);
8     int cnt = 0;
9     for (int i = 1; i < m; i++) {
10         char b = pattern[i];
11         while (cnt && pattern[cnt] != b) {
12             cnt = pi[cnt - 1];
13         }
14         if (pattern[cnt] == b) {
15             cnt++;
16         }
17         pi[i] = cnt;
18     }
19     vector<int> pos;
20     cnt = 0;
21     for (int i = 0; i < text.size(); i++) {
```

```
20     char b = text[i];
21     while (cnt && pattern[cnt] != b) {
22         cnt = pi[cnt - 1];
23     }
24     if (pattern[cnt] == b) {
25         cnt++;
26     }
27     if (cnt == m) {
28         pos.push_back(i - m + 1);
29         cnt = pi[cnt - 1];
30     }
31     }
32     return pos;
33 }
```

10.5 suffix array lcp

```
1 #define radix_sort(x,y){\
2     for(i=0; i<A; ++i) c[i]=0;\
3     for(i=0; i<n; ++i) c[x[y[i]]]++;\
4     for(i=1; i<A; ++i) c[i]+=c[i-1];\
5     for(i=n-1; ~i; --i) sa[--c[x[y[i]]]]=y[i];\
6 }
7 #define AC(r,a,b)\
8     r[a]!=(r[b]||a+k>n||r[a+k]!=(r[b+k]
9 void suffix_array_lcp(const char *s, int n, int *
10 sa, int *rank, int *tmp, int *c){
11     int A='z'+1, i, k, id=0;
12     for(i=0; i<n; ++i) rank[tmp[i]=i]=s[i];
13     radix_sort(rank, tmp);
14     for(k=1; id<n-1; k<=1){
15         for(id=0, i=n-k; i<n; ++i) tmp[id++] = i;
16         for(i=0; i<n; ++i)
17             if(sa[i]>=k) tmp[id++] = sa[i]-k;
18         radix_sort(rank, tmp);
19         swap(rank, tmp);
20         for(rank[sa[0]]=id=0, i=1; i<n; ++i)
21             rank[sa[i]]=id+=AC(tmp, sa[i-1], sa[i]);
22         A=id+1;
23     }
24 }
25 //h:高度數組 sa:後綴數組 rank:排名
26 void suffix_array_lcp(const char *s, int len,
27 int *h, int *sa, int *rank){
28     for(int i=0; i<len; ++i) rank[sa[i]]=i;
29     for(int i=0, k=0; i<len; ++i){
30         if(rank[i]==0) continue;
31         if(k--<0)
32             while(s[i+k]==s[sa[rank[i]-1]+k]) ++k;
33         h[rank[i]]=k;
34 }
```

10.6 hash

```
1 #define MAXN 1000000
2 #define mod 1073676287
3 /*mod 必須要是質數*/
```

```

4 typedef long long T;
5 char s[MAXN+5];
6 T h[MAXN+5]; /*hash 陣列*/
7 T h_base[MAXN+5]; /*h_base[n]=(prime^n)%mod*/
8 void hash_init(int len, T prime){
9     h_base[0]=1;
10    for(int i=1; i<=len; ++i){
11        h[i]=(h[i-1]*prime+s[i-1])%mod;
12        h_base[i]=(h_base[i-1]*prime)%mod;
13    }
14 }
15 T get_hash(int l, int r){ /*閉區間寫法，設編號
    為 0 ~ len-1*/
16    return (h[r+1]-(h[l]*h_base[r-l+1])%mod+
    mod)%mod;
17 }

```

10.7 minimal string rotation

```

1 int min_string_rotation(const string &s){
2     int n=s.size(), i=0, j=1, k=0;
3     while(i<n&&j<n&&k<n){
4         int t=s[(i+k)%n]-s[(j+k)%n];
5         ++k;
6         if(t){
7             if(t>0)i+=k;
8             else j+=k;
9             if(i==j)++j;
10            k=0;
11        }
12    }
13    return min(i, j); //最小循環表示法起始位置
14 }

```

10.8 reverseBWT

```

1 const int MAXN = 305, MAXC = 'Z';
2 int ranks[MAXN], tots[MAXN], first[MAXC];
3 void rankBWT(const string &bw){
4     memset(ranks, 0, sizeof(int)*bw.size());
5     memset(tots, 0, sizeof(tots));
6     for(size_t i=0; i<bw.size(); ++i)
7         ranks[i] = tots[ bw[i] ]++;
8 }
9 void firstCol(){
10    memset(first, 0, sizeof(first));
11    int totc = 0;
12    for(int c='A'; c<='Z'; ++c){
13        if(!tots[c]) continue;
14        first[c] = totc;
15        totc += tots[c];
16    }
17 }
18 string reverseBwt(string bw, int begin){
19     rankBWT(bw), firstCol();
20     int i = begin; //原字串最後一個元素的位置
21     string res;
22     do{
23         char c = bw[i];

```

```

24         res = c + res;
25         i = first[ int(c) ] + ranks[i];
26     } while (i != begin );
27     return res;
28 }

```

11 Tree

11.1 Euler tour+RMQ

```

1 // Euler Tour Technique
2 class LCA {
3     const vector<vector<int>>& adj;
4     int n;
5     vector<int> d, first, euler{}, log2{};
6     vector<vector<int>> st{};
7     void dfs(int u, int w = -1, int dep = 0)
8     {
9         d[u] = dep;
10        first[u] = euler.size();
11        euler.push_back(u);
12        for (auto& v : adj[u]) {
13            if (v == w) continue;
14            dfs(v, u, dep + 1);
15            euler.push_back(u);
16        }
17    public:
18        LCA(const vector<vector<int>>& _adj, int
19            root) : adj{ _adj }, n{ adj.size() }, d
20            (n), first(n) {
21            dfs(root);
22
23            int tn{euler.size()};
24            log2.resize(tn + 1);
25            log2[1] = 0;
26            for (int i{2}; i <= tn; ++i) log2[i]
27                = log2[i / 2] + 1;
28
29            st.assign(tn, vector<int>(log2[tn] +
30                1));
31            for (int i{tn - 1}; i >= 0; --i) {
32                st[i][0] = euler[i];
33                for (int j{1}; i + (1 << j) <=
34                    tn; ++j) {
35                    auto& x{st[i][j - 1]};
36                    auto& y{st[i + (1 << j) - 1]
37                        ][j - 1]};
38                    st[i][j] = d[x] <= d[y] ? x
39                        : y;
40                }
41            }
42            int operator()(int u, int v) {
43                int l{first[u]}, r{first[v]};
44                if (l > r) swap(l, r);
45                ++r; // make the interval left
46                    closed right open
47
48                int j{log2[r - l]};
49                auto& x{st[l][j]};

```

```

43         auto& y{st[r - (1 << j)][j]};
44         return d[x] <= d[y] ? x : y;
45     }
46 };
47
48 int main() {
49     int n, q;
50     cin >> n >> q;
51     vector<vector<int>> adj(n);
52
53     for (int i = 1; i < n; ++i) {
54         int u;
55         cin >> u;
56         u--;
57         adj[u].pb(i);
58         adj[i].pb(u);
59     }
60
61     LCA lca(adj, 0);
62
63     while (q--) {
64         int u, v;
65         cin >> u >> v;
66         u--, v--;
67         cout << lca(u, v) + 1 << "\n";
68     }
69     return 0;
70 }

```

11.2 HLD template

```

1 struct HLD {
2     int n, cur = 0;
3     vector<int> sz, top, dep, par, tin, tout
4         , seq;
5     vector<vector<int>> adj;
6     HLD(int n = 0) {
7         init(n);
8     }
9     void init(int n_) {
10        n = n_;
11        sz.assign(n, 1);
12        top.assign(n, -1);
13        dep.assign(n, 0);
14        par.assign(n, -1);
15        tin.assign(n, 0);
16        tout.assign(n, 0);
17        seq.assign(n, 0);
18        adj.resize(n, {});
19    }
20    void add_edge(int u, int v) { adj[u].
21        push_back(v), adj[v].push_back(u); }
22    void build(int root = 0) {
23        top[root] = root, dep[root] = 0, par
24            [root] = -1;
25        dfs1(root), dfs2(root);
26    }
27    void dfs1(int u) {
28        if (auto it = find(adj[u].begin(),
29            adj[u].end(), par[u]); it != adj
30            [u].end()) {
31            adj[u].erase(it);
32        }
33    }

```

```

28     for (auto &v : adj[u]) {
29         par[v] = u;
30         dep[v] = dep[u] + 1;
31         dfs1(v);
32         sz[u] += sz[v];
33         if (sz[v] > sz[adj[u][0]]) {
34             swap(v, adj[u][0]);
35         }
36     }
37     void dfs2(int u) {
38         tin[u] = cur++;
39         seq[tin[u]] = u;
40         for (auto v : adj[u]) {
41             top[v] = v == adj[u][0] ? top[u]
42                 : v;
43             dfs2(v);
44         }
45         tout[u] = cur;
46     }
47     int lca(int u, int v) {
48         while (top[u] != top[v]) {
49             if (dep[top[u]] > dep[top[v]]) {
50                 u = par[top[u]];
51             } else {
52                 v = par[top[v]];
53             }
54         }
55         return dep[u] < dep[v] ? u : v;
56     }
57     int dist(int u, int v) { return dep[u] +
58         dep[v] - 2 * dep[lca(u, v)]; }
59     int jump(int u, int k) {
60         if (dep[u] < k) { return -1; }
61         int d = dep[u] - k;
62         while (dep[top[u]] > d) { u = par[
63             top[u]]; }
64         return seq[tin[u] - dep[u] + d];
65     }
66     // u is v's ancestor
67     bool is_ancestor(int u, int v) { return
68         tin[u] <= tin[v] && tin[v] < tout[u]
69         ]; }
70     // root's parent is itself
71     int rooted_parent(int r, int u) {
72         if (r == u) { return u; }
73         if (is_ancestor(r, u)) { return par[
74             u]; }
75         auto it = upper_bound(adj[u].begin(),
76             adj[u].end(), r, [&](int x,
77                 int y) {
78                     return tin[x] < tin[y];
79                 }) - 1;
80         return *it;
81     }
82     // rooted at u, v's subtree size
83     int rooted_size(int r, int u) {
84         if (r == u) { return n; }
85         if (is_ancestor(u, r)) { return sz[u]
86             ]; }
87         return n - sz[rooted_parent(r, u)];
88     }
89     int rooted_lca(int r, int a, int b) {
90         return lca(a, b) ^ lca(a, r) ^ lca(b
91             , r); }
92 };

```

```

82 int main() {
83     ios::sync_with_stdio(false);
84     cin.tie(nullptr);
85
86     int n = 7;
87     HLD hld(n);
88
89     // build this tree:
90     //      0
91     //    / \
92     //   1   2
93     //  / \   \
94     // 3   4   5
95     //    |   /
96     //    6
97     hld.add_edge(0, 1);
98     hld.add_edge(0, 2);
99     hld.add_edge(1, 3);
100    hld.add_edge(1, 4);
101    hld.add_edge(2, 5);
102    hld.add_edge(5, 6);
103
104    hld.build(0); // root at node 0
105
106    cout << "LCA(3,4) = " << hld.lca(3,4) <<
107           "\n"; // expected 1
108    cout << "LCA(3,6) = " << hld.lca(3,6) <<
109           "\n"; // expected 0
110    cout << "Distance(3,4) = " << hld.dist
111           (3,4) << "\n"; // expected 2
112    cout << "Distance(3,6) = " << hld.dist
113           (3,6) << "\n"; // expected 4
114    cout << "Is 0 ancestor of 6? " << (hld.
115           is_ancestor(0,6) ? "yes" : "no") <<
116           "\n"; // yes
117
118    return 0;
119 }

```

11.3 dist between u-v in tree

```

1 int dist(int a, int b) {
2     int c = lca(a, b);
3     return depth[a] + depth[b] - 2 * depth[c];
4 }

```

11.4 all longest path dfs

```

1 // all Longest path (generalization of the
2   tree diameter problem)
3 vector<tuple<int, int, int>> dp{};
4 // [mx1, x, mx2] the path of mx1 goes
5   through x
6 int dfs1(int u, int w = -1) {
7     int mx{0};
8     for (auto& v : adj[u])
9         if (v != w) {
10             auto l{1 + dfs1(v, u)};
11             mx = max(mx, l);
12         }
13     return mx;
14 }

```

```

10 auto& [mx1, x, mx2]{dp[u]};
11 if (l >= mx1) mx2 = mx1, mx1 = l;
12     , x = v;
13 else if (l > mx2) mx2 = l;
14 }
15 return mx;
16 }
17 void dfs2(int u, int w = -1) {
18     if (w != -1) {
19         int tmx;
20         { // calculate the Longest path
21             through parent
22             auto& [mx1, x, mx2]{dp[w]};
23             if (x != u) tmx = mx1 + 1;
24             else tmx = mx2 + 1;
25         }
26         { // update the path
27             auto& [mx1, x, mx2]{dp[u]};
28             if (tmx >= mx1) mx2 = mx1, mx1 =
29                 tmx, x = w;
30             else if (tmx > mx2) mx2 = tmx;
31         }
32     }
33     for (auto& v : adj[u])
34         if (v != w) dfs2(v, u);
35 }
36 void all_longest_path() {
37     dfs1(0), dfs2(0);
38 }

```

11.5 all longest path top sort

```

1 // all Longest path in DAG
2 // 1. topological sort
3 vector<int> in(n, 0);
4 for (int i = 0; i < m; ++i) {
5     int a, b, w;
6     cin >> a >> b >> w;
7     adj[a].emplace_back(b, w);
8     in[b]++;
9 }
10 vector<int> topo; // sequence of top sort
11 queue<int> q;
12 for (int i = 0; i < n; ++i) {
13     if (in[i] == 0) {
14         q.push(i);
15     }
16 }
17 while (!q.empty()) {
18     int pa = q.front();
19     q.pop();
20     topo.push_back(pa);
21     for (auto& [child, w] : adj[pa]) {
22         in[child]--;
23         if (in[child] == 0) {
24             q.push(child);
25         }
26     }
27 }
28 // all Longest path

```

```

31 vector<int> dist(n, INT_MIN);
32 vector<vector<int>> parents(n);
33 dist[0] = process[0];
34
35 for (int u : topo) {
36     for (auto& [v, w] : adj[u]) {
37         if (dist[v] < dist[u] + process[v] +
38             w) {
39             dist[v] = dist[u] + process[v] +
40                 w;
41             parents[v] = {u};
42         } else if (dist[v] == dist[u] +
43                     process[v] + w) {
44             parents[v].push_back(u);
45         }
46     }
47 }
48 cout << dist[n - 1];

```

11.6 Heavy-light decomposition (HLD)

```

1 const int maxn = 2e5;
2 int n, q;
3 int val[maxn];
4 int id[maxn];
5 int sz[maxn];
6 int parent[maxn];
7 int depth[maxn];
8 int tp[maxn];
9 vector<int> adj[maxn];
10 int timer = 0;
11
12 int dfs_sz(int u, int p) {
13     sz[u] = 1;
14     parent[u] = p;
15
16     // calculate size of each subtree
17     for (int v : adj[u]) {
18         if (v != p) {
19             depth[v] = depth[u] + 1;
20             sz[u] += dfs_sz(v, u);
21         }
22     }
23     return sz[u];
24 }
25
26 void dfs_hld(int u, int p, int top, SGT<int,
27     MergeMax>& tree) {
28     id[u] = timer++;
29     tp[u] = top;
30     tree.update(id[u], val[u]);
31
32     // find the heavy child
33     int h_v = -1, h_sz = -1;
34     for (int v : adj[u]) {
35         if (v != p) {
36             if (h_sz < sz[v]) {
37                 h_sz = sz[v];
38                 h_v = v;
39             }
40         }
41     }
42     dfs_hld(h_v, u, top, tree);
43 }

```

```

38 }
39 }
40 }
41 if (h_v == -1) {
42     return;
43 }
44 // continue the chain
45 dfs_hld(h_v, u, top, tree);
46 // start a new chain for each light
47   child
48 for (int v : adj[u]) {
49     if (v != p && v != h_v) {
50         dfs_hld(v, u, v, tree);
51     }
52 }
53 }
54
55 int path(int x, int y, SGT<int, MergeMax>&
56     tree) {
57     int ans = 0;
58     // move x and y to their top nodes until
59     // they are in the same chain
60     while (tp[x] != tp[y]) {
61         if (depth[tp[x]] < depth[tp[y]])
62             swap(x, y);
63         ans = max(ans, tree.query(id[tp[x]],
64             id[x] + 1));
65         x = parent[tp[x]];
66     }
67     if (depth[x] > depth[y]) swap(x, y);
68     ans = max(ans, tree.query(id[x], id[y] +
69         1));
70     return ans;
71 }
72
73 void solve() {
74     cin >> n >> q;
75     SGT<int, MergeMax> tree(n, -1);
76
77     for (int i = 0; i < n; ++i) {
78         cin >> val[i];
79     }
80     for (int i = 0; i < n - 1; ++i) {
81         int a, b;
82         cin >> a >> b;
83         a--, b--;
84         adj[a].push_back(b);
85         adj[b].push_back(a);
86     }
87     dfs_sz(0, 0);
88     dfs_hld(0, 0, 0, tree);
89
90     cerr << "tp: ";
91     for (int i = 0; i < n; ++i) {
92         cerr << tp[i] << " ";
93     }
94     cerr << "\n";
95
96     while (q--) {
97         int mode;
98         cin >> mode;
99         if (mode == 1) {
100             int s, x;
101             cin >> s >> x;
102         }
103     }

```

```

98     s--;
99     val[s] = x;
100     tree.update(id[s], val[s]);
101 }
102 else {
103     int a, b;
104     cin >> a >> b;
105     cout << path(a - 1, b - 1, tree)
106         << " ";
107 }
108 }

```

11.7 binary lifting

```

1 // binary lifting
2 class LCA {
3     const vector<vector<int>>& adj;
4     int n;
5     vector<int> d;
6     vector<int> log2;
7     vector<vector<int>> an{};
8     void dfs(int u, int w = -1, int dep = 0)
9     {
10         d[u] = dep;
11         an[u][0] = w;
12         for (int i{1}; i <= log2[n - 1] &&
13             an[u][i - 1] != -1; ++i)
14             an[u][i] = an[an[u][i - 1]][i - 1]; // 走 2^(i-1) 再走 2^(i-1) 步
15
16         // 因為計算 an 會用到祖先的資訊，所以先計算再繼續往下
17         for (auto& v : adj[u]) {
18             if (v == w) continue; // parent
19             dfs(v, u, dep + 1);
20         }
21     }
22 public:
23     LCA(const vector<vector<int>>& _adj, int root)
24     : adj{_adj}, n{adj.size()}, d(n, log2(n)) {
25         log2[1] = 0;
26         for (int i{2}; i < log2.size(); ++i)
27             log2[i] = log2[i / 2] + 1;
28         an.assign(n, vector<int>(log2[n - 1] + 1, -1));
29         dfs(root);
30     }
31     int operator()(int u, int v) {
32         if (d[u] > d[v]) swap(u, v);
33
34         for (int i{log2[d[v] - d[u]]}; i >= 0; --i)
35             if (d[v] - d[u] >= (1 << i)) v = an[v][i];
36         // v 先走到跟 u 同高度
37         if (u == v) return u;
38         for (int i{log2[d[u]]}; i >= 0; --i)

```

```

37         if (an[u][i] != an[v][i]) u = an[u][i], v = an[v][i];
38         // u, v 一起走到 Lca(u, v) 的下方
39
40         return an[u][0];
41         // 回傳 Lca(u, v)
42     }
43 };
44
45 int main() {
46     int n, q;
47     cin >> n >> q;
48     vector<vector<int>> adj(n);
49
50     for (int i = 1; i < n; ++i) {
51         int u;
52         cin >> u;
53         u--;
54         adj[u].pb(i);
55         adj[i].pb(u);
56     }
57
58     // adj, root
59     LCA lca(adj, 0);
60
61     while (q--) {
62         int u, v;
63         cin >> u >> v;
64         u--, v--;
65         cout << lca(u, v) + 1 << "\\n";
66     }
67     return 0;
68 }

```

11.8 get kth ancestor

```

1 void dfs(int u, int p = -1) {
2     up[0][u] = (p == -1 ? u : p);
3     for (int v : adj[u]) {
4         if (v != p) {
5             dfs(v, u);
6         }
7     }
8 }
9
10 int get_kth_ancestor(int x, int k) {
11     for (int i = 0; i <= LOG; ++i) {
12         if (k & (1LL << i)) {
13             x = up[i][x];
14         }
15     }
16     return (x == 0 ? -1 : x);
17 }
18
19 void solve() {
20     int n, q;
21     cin >> n >> q;
22
23     adj[0].push_back(1);
24     for (int i = 2; i <= n; ++i) {
25         int u;
26         cin >> u;

```

```

27         adj[u].push_back(i);
28     }
29     dfs(0, -1);
30
31     for (int k = 1; k <= LOG; ++k) {
32         for (int v = 1; v <= n; ++v) {
33             up[k][v] = up[k - 1][up[k - 1][v]];
34         }
35     }
36
37     while (q--) {
38         int x, k;
39         cin >> x >> k;
40         cout << get_kth_ancestor(x, k) << "\\n";
41     }
42 }
43 }

```

11.9 check z on path u-v

```

1 bool on_path(int z, int u, int v) {
2     return dist(u, z) + dist(z, v) == dist(u, v);
3 }

```

11.10 binary jumping

```

1 const int LOG = 32;
2 const int maxn = 2e5 + 2;
3
4 int depth[maxn];
5 int up[LOG][maxn];
6 vector<int> adj[maxn];
7
8 void dfs(int u, int p = -1) {
9     up[0][u] = (p == -1 ? u : p);
10    for (int v : adj[u]) {
11        if (v != p) {
12            depth[v] = depth[u] + 1;
13            dfs(v, u);
14        }
15    }
16 }
17
18 int lca(int a, int b) {
19     if (depth[a] < depth[b]) swap(a, b);
20     int diff = depth[a] - depth[b];
21     for (int k = 0; diff; ++k) {
22         if (diff & 1) a = up[k][a];
23         diff >>= 1;
24     }
25     if (a == b) return a;
26     for (int k = LOG - 1; k >= 0; --k) {
27         if (up[k][a] != up[k][b]) {
28             a = up[k][a];
29             b = up[k][b];
30         }
31     }

```

```

32     return up[0][a];
33 }
34
35 void solve() {
36     ios::sync_with_stdio(false);
37     cin.tie(nullptr);
38
39     int n, q;
40     cin >> n >> q;
41
42     for (int i = 2; i <= n; ++i) {
43         int u;
44         cin >> u;
45         adj[u].push_back(i);
46     }
47     dfs(1, -1); // root = 1
48
49     for (int k = 1; k <= LOG; ++k) {
50         for (int v = 1; v <= n; ++v) {
51             up[k][v] = up[k - 1][up[k - 1][v]];
52         }
53     }
54
55     while (q--) {
56         int a, b;
57         cin >> a >> b;
58         cout << lca(a, b) << "\\n";
59     }
60 }
61 }

```

11.11 tree diameter

```

1 int diam = 0;
2
3 void dfs(int u, int p = -1) {
4     int mx = 0;
5     for (int v : adj[u]) {
6         if (v != p) {
7             int len = 1 + dfs(v, u);
8             diam = max(diam, mx + len);
9             mx = max(mx, len);
10        }
11    }
12    return mx;
13 }

```

11.12 all longest path

```

1 int fir[maxn]; // Length of the longest downward path from u into its subtree.
2 int sec[maxn]; // Length of the second longest downward path from u
3 int res[maxn];
4
5 void dfs1(int u, int p) {
6     for (int v : adj[u]) {
7         if (v != p) {
8             dfs1(v, u);

```

```

9      if (fir[v] + 1 > fir[u]) {
10          sec[u] = fir[u];
11          fir[u] = fir[v] + 1;
12      }
13      else if (fir[v] + 1 > sec[u]) {
14          sec[u] = fir[v] + 1;
15      }
16  }
17  }
18  }
19  // to_p: the best path length that comes
20  // from the parent's side
21  void dfs2(int u, int p, int to_p) {
22      res[u] = max(to_p, fir[u]);
23  }
24  for (int v : adj[u]) {
25      if (v != p) {
26          if (fir[v] + 1 == fir[u]) {
27              dfs2(v, u, max(to_p, sec[u])
28                  + 1);
29          }
30          else {
31              dfs2(v, u, res[u] + 1);
32          }
33      }
34  }
35  }
36  // usage
37  dfs1(1, 0);
38  dfs2(1, 0, 0);
39  // Now res[i] is the maximum distance from
40  // node i to any other node
41  for (int i = 1; i <= n; i++) {
42      cout << res[i] << " ";
43  }

```

11.13 tree diameter (len,end)

```

1  array<int, 2> dfs(int u, int w = -1) {
2      array<int, 2> mx{0, u}; // {length,
3      // farthest leaf}
4      for (auto& v : adj[u]) {
5          if (v == w) continue;
6          auto [len, leaf]{dfs(v, u)};
7          mx = max(mx, {len + 1, leaf});
8      }
9      return mx;
10 }
11
12 array<int, 3> tree_diameter(int a = 0) {
13     auto b{dfs(a)[1]}; // farthest node
14     // from 'a'
15     auto [l, c]{dfs(b)}; // farthest node
16     // from 'b'
17     return {l, b, c}; // {diameter
18     // length, endpoint1, endpoint2}
19 }

```

11.14 minimum distance in functional graph

```

1  // Given a functional graph (each node has
2  // exactly one outgoing edge), answer
3  // queries about the minimum distance
4  // between two nodes, or determine that
5  // it's impossible to get from one to the
6  // other.
7  // There are three cases:
8  // 1. Both in Tree
9  // 2. Both in Cycle
10 // 3. One in Tree, one in Cycle
11
12 int main() {
13     int planet_num;
14     int query_num;
15     std::cin >> planet_num >> query_num;
16     vector<int> next(planet_num);
17     vector<vector<int>> before(planet_num);
18     for (int p = 0; p < planet_num; p++) {
19         std::cin >> next[p];
20         next[p]--;
21         before[next[p]].push_back(p);
22     }
23
24     /*
25     * -2 = We haven't even got to processing
26     * this planet yet.
27     * -1 = This node is part of a tree.
28     * >= 0: the ID of the cycle the planet
29     * belongs to.
30     */
31     vector<int> cycle_id(planet_num, -2);
32     // Each map, given a planet #, returns the
33     // index of that planet in the
34     // cycle.
35     vector<std::map<int, int>> cycles;
36     for (int p = 0; p < planet_num; p++) {
37         if (cycle_id[p] != -2) { continue; }
38         vector<int> path{p};
39         int at = p;
40         while (cycle_id[next[at]] == -2) {
41             at = next[at];
42             cycle_id[at] = -3; // Leave
43             // breadcrumbs for this iteration
44             path.push_back(at);
45         }
46         std::map<int, int> cycle;
47         bool in_cycle = false;
48         for (int i : path) {
49             in_cycle = in_cycle || i == next[at];
50             if (in_cycle) { cycle[i] = cycle.size()
51                             (); }
52             cycle_id[i] = in_cycle ? cycles.size()
53                             : -1;
54         }
55         cycles.push_back(cycle);
56     }
57
58     /*
59     * Precalculate the distance from each
60     * planet to its cycle with BFS.
61     */
62 }

```

```

63 * (cyc_dist[p] = 0) if p is part of a
64 * cycle.
65 */
66 vector<int> cyc_dist(planet_num);
67 for (int p = 0; p < planet_num; p++) {
68     // Check if this planet is part of a
69     // cycle.
70     if (cycle_id[next[p]] == -1 || cycle_id[
71         p] != -1) { continue; }
72     cyc_dist[p] = 1;
73     vector<int> stack(before[p]);
74     while (!stack.empty()) {
75         int curr = stack.back();
76         stack.pop_back();
77         cyc_dist[curr] = cyc_dist[next[curr]]
78             + 1;
79         stack.insert(stack.end(), before[curr]
80             .begin(), before[curr].end());
81     }
82 }
83
84 // Initialize the binary jumping arrays.
85 int log2 = std::ceil(std::log2(planet_num)
86 );
87 vector<vector<int>> pow2_ends(planet_num,
88     vector<int>(log2 + 1));
89 for (int p = 0; p < planet_num; p++) {
90     pow2_ends[p][0] = next[p];
91 }
92 for (int i = 1; i <= log2; i++) {
93     for (int p = 0; p < planet_num; p++) {
94         pow2_ends[p][i] = pow2_ends[pow2_ends[
95             p][i - 1]][i - 1];
96     }
97 }
98
99 /*
100 * Given a starting planet & dist, returns
101 * the planet we end up at
102 * if we use the teleporter dist times.
103 */
104 auto advance = [&](int pos, int dist) {
105     for (int pow = log2; pow >= 0; pow--) {
106         if ((dist & (1 << pow)) != 0) { pos =
107             pow2_ends[pos][pow]; }
108     }
109     return pos;
110 };
111
112 for (int q = 0; q < query_num; q++) {
113     int u, v; // going from u to v
114     std::cin >> u >> v;
115     u--;
116     v--;
117     if (cycle_id[pow2_ends[u][log2]] !=
118         cycle_id[pow2_ends[v][log2]]) {
119         cout << -1 << '\n';
120         continue;
121     }
122     if (cycle_id[u] != -1 || cycle_id[v] !=
123         -1) {
124         if (cycle_id[v] == -1 && cycle_id[u]
125             != -1) {
126             cout << -1 << '\n';
127             continue;
128         }
129         // Handle the 2nd & 3rd cases at the
130         // same time.
131     }
132 }

```

```

133 int dist = cyc_dist[u];
134 int u_cyc = advance(u, cyc_dist[u]);
135 // The root of u's tree
136
137 std::map<int, int> &cyc = cycles[
138     cycle_id[u_cyc]]; // u and v's
139 // cycle
140 int u_ind = cyc[u_cyc];
141 int v_ind = cyc[v];
142 int diff = u_ind <= v_ind ? v_ind -
143     u_ind : cyc.size() - (u_ind -
144     v_ind);
145 cout << dist + diff << '\n';
146 } else {
147     if (cyc_dist[v] > cyc_dist[u]) {
148         cout << -1 << '\n';
149         continue;
150     }
151     int diff = cyc_dist[u] - cyc_dist[v];
152     cout << (advance(u, diff) == v ? diff
153         : -1) << '\n';
154 }
155 }
156 }

```

11.15 union length of two tree paths

```

1  /*
2  Problem: Given a tree and queries (a,b,c,d),
3  find the size of the union of nodes on
4  paths a+b and c+d.
5
6  Let P1 be the nodes on the path a+b and P2
7  the nodes on c+d.
8  We want |P1 ∩ P2| = |P1| + |P2| - |P1 ∪ P2|.
9  |P1| = dist(a,b) + 1, |P2| = dist(c,d) + 1.
10 So the only hard part is |P1 ∩ P2|.
11
12 Key facts for trees:
13
14 The intersection of two simple paths is
15 itself either empty or a simple path.
16
17 Any endpoint of that intersection must be
18 one of the LCAs among endpoint pairs of
19 the two paths.
20
21 Concretely the candidate nodes are
22 lca(a,b), lca(c,d), lca(a,c), lca(a,d), lca(
23     b,c), lca(b,d)
24
25 A node z lies on path u+v iff dist(u,z) +
26     dist(z,v) == dist(u,v).
27
28 So for each query:
29
30 compute the 6 candidate LCAs,
31
32 keep those candidates that lie on both paths
33 (test with distances),
34
35

```



```

26 if none → intersection size = 0; if one →
    intersection size = 1;
27 otherwise take the two kept candidates with
    greatest depth (they are the endpoints
    of the intersection)
28 and intersection size = dist(p,q) + 1.
29 */
30
31 const int LOG = 32;
32
33 const int maxn = 1e5;
34 int depth[maxn];
35 vector<int> adj[maxn];
36 int up[LOG + 1][maxn];
37 int n, q;
38
39 void dfs(int u, int p = -1) {
40     up[0][u] = (p == -1 ? u : p);
41     for (int v : adj[u]) {
42         if (v != p) {
43             depth[v] = depth[u] + 1;
44             dfs(v, u);
45         }
46     }
47 }
48
49 void solve() {
50     cin >> n >> q;
51     for (int i = 0; i < n - 1; ++i) {
52         int a, b;
53         cin >> a >> b;
54         a--, b--;
55         adj[a].push_back(b);
56         adj[b].push_back(a);
57     }
58     memset(depth, 0, sizeof(depth));
59     dfs(0, -1);
60
61     for (int k = 1; k < LOG; ++k) {
62         for (int v = 0; v < n; ++v) {
63             up[k][v] = up[k - 1][up[k - 1][v]
64                 ];
65         }
66     }
67
68     while (q--) {
69         int a, b, c, d;
70         cin >> a >> b >> c >> d;
71         a--, b--, c--, d--;
72         int len1 = dist(a, b) + 1;
73         int len2 = dist(c, d) + 1;
74
75         array<int, 6> cand = {
76             lca(a, b), lca(c, d),
77             lca(a, c), lca(a, d),
78             lca(b, c), lca(b, d)
79         };
80
81         vector<int> good;
82         for (int z : cand) {
83             if (on_path(z, a, b) && on_path(
84                 z, c, d)) {
85                 good.push_back(z);
86             }
87         }
88     }
89 }

```

```

87 int inter = 0;
88 if (good.size() == 1) {
89     inter = 1;
90 }
91 else if (good.size() > 1) {
92     sort(good.begin(), good.end(),
93         [&](int x, int y) {
94             return depth[x] > depth[y];
95         });
96     int p = good[0], qn = good[1];
97     inter = dist(p, qn) + 1;
98 }
99 cout << (len1 + len2 - inter) << "\n";
100 }

```

11.16 Centroid decomposition

```

1 // Centroid Decomposition
2 // Problem: Given a tree with N nodes (1-
    indexed) and M queries. Each query is
    one of the following two types:
3 // 1 v: Paint node v red.
4 // 2 v: Print the minimal distance between
    node v and any red node.
5 // Initially, only node 1 is red.
6 // Constraints: 1 ≤ N, M ≤ 10^5
7 // Time complexity: O((N + M) Log N)
8
9 vector<vector<int>> adj;
10 vector<int> subtree_size;
11 // min_dist[v] = the minimal distance
    between v and a red node
12 vector<int> min_dist;
13 vector<bool> is_removed;
14 vector<vector<pii>> ancestors;
15
16 int get_subtree_size(int u, int p = -1) {
17     subtree_size[u] = 1;
18     for (int v : adj[u]) {
19         if (v == p || is_removed[v])
20             continue;
21         subtree_size[u] += get_subtree_size(
22             v, u);
23     }
24     return subtree_size[u];
25 }
26
27 int get_centroid(int u, int tree_size, int p
    = -1) {
28     for (int v : adj[u]) {
29         if (v == p || is_removed[v])
30             continue;
31         if (subtree_size[v] * 2 > tree_size)
32             return get_centroid(v, tree_size
33                 , u);
34     }
35     return u;
36 }

```

```

35 // Calculate the distance between current `u
    ` and the `centroid` it belongs.
36 void get_dist(int u, int centroid, int p =
    -1, int cur_dist = 1) {
37     for (int v : adj[u]) {
38         if (v == p || is_removed[v])
39             continue;
40         cur_dist++;
41         get_dist(v, centroid, u, cur_dist);
42         cur_dist--;
43     }
44     ancestors[u].emplace_back(centroid,
45         cur_dist);
46 }
47
48 void build_centroid_decomp(int u = 0) {
49     int centroid = get_centroid(u,
50         get_subtree_size(u));
51
52     // For all nodes in the subtree rooted
53     // at `centroid`, calculate their
54     // distance to the centroid
55     for (int v : adj[centroid]) {
56         if (is_removed[v]) continue;
57         get_dist(v, centroid, centroid);
58     }
59
60     is_removed[centroid] = true;
61     for (int v : adj[centroid]) {
62         if (is_removed[v]) continue;
63         build_centroid_decomp(v);
64     }
65 }
66
67 // Paint `node` red by updating all of its
68 // ancestors' minimal distances to a red
69 // node
70 void paint(int u) {
71     for (auto &[ancestor, dist] : ancestors[
72         u]) {
73         min_dist[ancestor] = min(min_dist[
74             ancestor], dist);
75     }
76     min_dist[u] = 0;
77 }
78
79 // Print the minimal distance between `u` to
80 // a red node.
81 void query(int u) {
82     int ans = min_dist[u];
83     for (auto &[ancestor, dist] : ancestors[
84         u]) {
85         if (!dist) continue;
86         ans = min(ans, dist + min_dist[
87             ancestor]);
88     }
89     cout << ans << "\n";
90 }
91
92 void solve() {
93     int N, M;
94     cin >> N >> M;
95
96     adj.assign(N, vector<int>());
97     for (int i = 0; i < N - 1; ++i) {
98         int a, b;

```

```

87     cin >> a >> b;
88     a--, b--;
89     adj[a].push_back(b);
90     adj[b].push_back(a);
91 }
92
93 subtree_size.assign(N, 0);
94 ancestors.assign(N, vector<pii>());
95 is_removed.assign(N, false);
96 build_centroid_decomp();
97
98 min_dist.assign(N, INF);
99 paint(0);
100 for (int i = 0; i < M; ++i) {
101     int t, v;
102     cin >> t >> v;
103     v--;
104     if (t == 1) {
105         paint(v);
106     }
107     else {
108         query(v);
109     }
110 }
111 }

```

12 default

12.1 debug

```

1 #ifndef DEBUG
2 #define dbg(...) {\
3     fprintf(stderr, "%s - %d : (%s) = ",
4         __PRETTY_FUNCTION__, __LINE__, #
5         __VA_ARGS__); \
6     _DO(__VA_ARGS__); \
7 }
8 template<typename I> void _DO(I&&x){cerr<<x
9     <<endl;}
10 template<typename I, typename...T> void _DO(I
11     &&x,T&&...tail){cerr<<x<<" ";_DO(tail
12     ...);}
13 #else
14 #define dbg(...)
15 #endif

```

12.2 template

```

1 // alias g++='g++ -std=c++14 -fsanitize=
2 // undefined -Wall -Wextra -Wshadow -D
3 // LOCAL'
4
5 #include <bits/stdc++.h>
6 using namespace std;
7
8 #ifndef LOCAL
9 #include "../debug.h"
10 #else

```

```

9 #pragma GCC optimize("O3,unroll-loops")
10 #pragma GCC target("avx2,bmi,bmi2,lzcnt,
    popcnt")
11 #define debug(...) ((void)0)
12 #define orange(...) ((void)0)
13 #endif
14
15 #define int long long
16 #define pii pair<int, int>
17 #define ff first
18 #define ss second
19 #define pb push_back
20 #define SPEEDY ios_base::sync_with_stdio(
    false); cin.tie(0); cout.tie(0);
21
22 void solve() {
23
24 }
25
26 signed main() {
27     SPEEDY;
28
29     return 0;
30 }

```

12.3 vimrc

```

1 set nu
2 set ts=4
3 set shiftwidth=4
4 set autoindent smartindent
5 set cindent
6
7 imap jk <ESC>
8 inoremap {<CR> {<CR><ESC>ko

```

13 other

13.1 Nim game

```

1 | a1^a2^a3^...^an != 0 ? A win : B win

```

13.2 找小於 n 所有出現的 1 數量

```

1 | current == 0 higher * factor
2 | current == 1 higher * factor + lower + 1
3 | other current (higher + 1) * factor

```

14 other language

14.1 python heap

```

1 import heapq
2
3 heap = [7,1,2,2]
4 heapq.heapify(heap)
5 print(heap) # [1, 2, 2, 7]
6 heapq.heappush(heap, 5)
7 print(heap) # [1, 2, 2, 7, 5]
8 print(heapq.heappop(heap)) # 1
9 print(heap) # [2, 2, 5, 7]

```

14.2 java

14.2.1 文件操作

```

1 import java.io.*;
2 import java.util.*;
3 import java.math.*;
4 import java.text.*;
5
6 public class Main{
7
8     public static void main(String args[]){
9         throws FileNotFoundException,
10            IOException
11         Scanner sc = new Scanner(new FileReader(
12            "a.in"));
13         PrintWriter pw = new PrintWriter(new
14            FileWriter("a.out"));
15         int n,m;
16         n=sc.nextInt();//读入下一个INT
17         m=sc.nextInt();
18
19         for(ci=1; ci<=c; ++ci){
20             pw.println("Case #" + ci + ": easy for
21                output");
22         }
23
24         pw.close();//关闭流并释放，这个很重要，
25            否则是没有输出的
26         sc.close();//关闭流并释放
27     }
28 }

```

14.2.2 优先队列

```

1 PriorityQueue queue = new PriorityQueue( 1,
2     new Comparator(){
3         public int compare( Point a, Point b ){
4             if ( a.x < b.x || a.x == b.x && a.y < b.y )
5                 return -1;
6             else if( a.x == b.x && a.y == b.y )
7                 return 0;
8             else return 1;
9         }
10    });

```

14.2.3 Map

```

1 Map map = new HashMap();
2 map.put("sa", "dd");
3 String str = map.get("sa").toString();
4
5 for(Object obj : map.keySet()){
6     Object value = map.get(obj );
7 }

```

14.2.4 sort

```

1 static class cmp implements Comparator{
2     public int compare(Object o1, Object o2){
3         BigInteger b1=(BigInteger)o1;
4         BigInteger b2=(BigInteger)o2;
5         return b1.compareTo(b2);
6     }
7 }
8 public static void main(String[] args)
9     throws IOException{
10     Scanner cin = new Scanner(System.in);
11     int n;
12     n=cin.nextInt();
13     BigInteger[] seg = new BigInteger[n];
14     for (int i=0;i<n;i++)
15         seg[i]=cin.nextBigInteger();
16     Arrays.sort(seg, new cmp());
17 }

```

14.3 python output

```

1 hello = 'Hello'
2 world = 7122
3 print(f'{hello} {world}') # Hello 7122
4
5 import math
6 print(f'PI is approximately {math.pi:.3f}.')
7 # PI is approximately 3.142.
8
9 print('AAA {} BBB {}'.format('Jin', 'KeLa'
10    ))
11 # AAA Jin BBB "KeLa!"
12
13 hello = 'hello, world\n'
14 hellos = repr(hello)
15 print(hellos) # 'hello, world\n'
16
17 x = 32.5
18 y = 40000
19 print(repr((x, y, ('spam', 'eggs'))))
20 # "(32.5, 40000, ('spam', 'eggs'))"
21
22 x = 7
23 print(eval('3 * x')) # 21

```

14.4 python 大數因數分解

```

1 # 大數因數分解 (使用 Pollard's Rho 與 Miller
2   -Rabin)
3 import sys, random
4 from math import gcd
5
6 # Miller-Rabin 檢定 (機率性質數判定)
7 def is_probable_prime(n, k=12):
8     if n < 2:
9         return False
10    # 先檢查一些小質數
11    small_primes =
12        [2,3,5,7,11,13,17,19,23,29]
13    for p in small_primes:
14        if n % p == 0:
15            return n == p
16    # 把 n-1 寫成 d * 2^s
17    d = n - 1
18    s = 0
19    while d % 2 == 0:
20        d //= 2
21        s += 1
22    # 重複 k 次隨機測試
23    for _ in range(k):
24        a = random.randrange(2, n - 1) # 隨機挑一個測試基數
25        x = pow(a, d, n)
26        if x == 1 or x == n - 1:
27            continue
28        composite = True
29        for __ in range(s - 1):
30            x = pow(x, 2, n)
31            if x == n - 1:
32                composite = False
33                break
34        if composite:
35            return False
36    return True
37
38 # Pollard's Rho 演算法 (找非平凡因數)
39 def pollards_rho(n):
40     if n % 2 == 0:
41         return 2
42     if n % 3 == 0:
43         return 3
44     # 隨機多項式 (x^2 + c) mod n
45     while True:
46         c = random.randrange(1, n-1) # 隨機挑選常數 c
47         x = random.randrange(2, n-1) # 起始點
48         y = x
49         d = 1
50         while d == 1:
51             x = (pow(x, 2, n) + c) % n # x -> f(x)
52             y = (pow(y, 2, n) + c) % n # y -> f(f(y)) · 走兩步
53             d = gcd(abs(x - y), n) # 計算兩者差的 gcd

```

```

53         if d == n:                #
54             失敗就重試
55             break
56         if d > 1 and d < n:        #
57             找到非平凡因數
58             return d
59 # 遞迴分解
60 def factor(n, out):
61     if n == 1:
62         return
63     if is_probable_prime(n):
64         out.append(n)
65     else:
66         d = pollards_rho(n)
67         while d is None or d == n: # 偶爾失
68             敗就重試
69             d = pollards_rho(n)
70         factor(d, out)
71         factor(n // d, out)
72
73 def main():
74     data = sys.stdin.read().strip().split()
75     if not data:
76         return
77     # 每個 token 當作一個數字
78     for token in data:
79         try:
80             n = int(token)
81         except:
82             continue
83         if n <= 1:
84             print(n)
85             continue
86         facs = []
87         factor(n, facs)
88         facs.sort()
89         # 輸出因數
90         print(" ".join(str(x) for x in facs)
91               )
92
93 if __name__ == "__main__":
94     random.seed() # 使用系統時間作為隨機種子
95     main()

```

14.5 decimal

```

1 # 使用 decimal 模組來處理高精度小數運算
2 from decimal import *
3 setcontext(Context(prec=MAX_PREC, Emax=
4 MAX_EMAX, rounding=ROUND_FLOOR))
5 print(Decimal(input()) * Decimal(input()))
6
7 # 將小數轉成分數，方便做近似或理論分析，且可
8 以限制分母大小。
9 from fractions import Fraction
10 Fraction('3.14159').limit_denominator(10).
11     numerator # 22
12
13 # 設定精確度

```

```

11 from decimal import Decimal, getcontext
12
13 # 精確位數設定
14 getcontext().prec = 70
15
16 n = 100
17 # 指定n為高精確度的物件
18 n = Decimal(n)
19 n /= 7
20 print(n)
21
22 # 將小數轉成分數
23 from fractions import Fraction
24
25 n = 1.5654
26 # 建立一個轉換物件
27 n = Fraction(n)
28
29 print(n)

```

14.6 python 大數排序

```

1 # 大數排序
2 # line one n : 多少數字
3 # next line : 依序輸入每行一個
4 # sort : sort + lambda
5 from sys import stdin
6
7 data = stdin.read().splitlines()
8
9 i = 0
10
11 while(i < len(data)):
12     T = int(data[i].strip())
13     i += 1
14     temp = [int(data[i + index].strip()) for
15             index in range(T)]
16     i += T
17     temp = sorted(temp, key = lambda x : (x)
18                 )
19     for ele in temp:
20         print(ele)

```

14.7 python 大數計算 2

```

1 # 單行輸入
2 # format : n1, operation, n2
3 from sys import stdin
4
5 data = stdin.read().splitlines()
6
7 limit = len(data)
8 i = 0
9
10 while(i < limit):
11     a, operation, b = map(str, data[i].split
12                             ())
13     a, b = int(a), int(b)
14     i += 1

```

```

14 if(operation == '+'):
15     print(int(a + b))
16 elif(operation == '-'):
17     print(int(a - b))
18 elif(operation == '*'):
19     print(int(a * b))
20 else:
21     print(int(a // b))

```

14.8 python input

```

1 ans = sum(map(float, input().split()))
2 # input: 1.1 2.2 3.3 4.4 5.5
3 print(ans) # 16.5
4
5 (n, m) = map(int, input().split()) # 300 200
6 print(n * m) # 60000
7
8 Arr = list(map(int, input().split()))
9 # input: 1 2 3 4 5
10 print(Arr) # [1, 2, 3, 4, 5]

```

14.9 python 大數計算

```

1 # 讀取多行輸入
2 # line one first number
3 # line two operation
4 # line three second number
5 from sys import stdin
6
7 data = stdin.read().splitlines()
8
9 limit = len(data)
10 i = 0
11 while(i < limit):
12     a = int(data[i].strip())
13     i += 1
14     operation = data[i].strip()
15     i += 1
16     b = int(data[i].strip())
17     i += 1
18     if(operation == '*'):
19         print(int(a * b))
20     else:
21         print(int(a / b))

```

15 zformula

15.1 formula

15.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

15.1.2 圖論

- 對於平面圖， $F = E - V + C + 1$ ， C 是連通分量數
- 對於平面圖， $E \leq 3V - 6$
- 對於連通圖 G ，最大獨立點集的大小設為 $I(G)$ ，最大匹配大小設為 $M(G)$ ，最小點覆蓋設為 $Cv(G)$ ，最小邊覆蓋設為 $Ce(G)$ 。對於任意連通圖：

$$(a) I(G) + Cv(G) = |V|$$

$$(b) M(G) + Ce(G) = |V|$$

- 對於連通二分圖：

$$(a) I(G) = Cv(G)$$

$$(b) M(G) = Ce(G)$$

- 最大權閉合圖：

$$(a) C(u, v) = \infty, (u, v) \in E$$

$$(b) C(S, v) = W_v, W_v > 0$$

$$(c) C(v, T) = -W_v, W_v < 0$$

$$(d) ans = \sum_{W_v > 0} W_v - flow(S, T)$$

- 最大密度子圖：

$$(a) \text{求 } \max \left(\frac{W_e + W_v}{|V|} \right), e \in E', v \in V'$$

$$(b) U = \sum_{v \in V} 2W_v + \sum_{e \in E} W_e$$

$$(c) C(u, v) = W_{(u, v)}, (u, v) \in E \cdot \text{雙向邊}$$

$$(d) C(S, v) = U, v \in V$$

$$(e) D_u = \sum_{(u, v) \in E} W_{(u, v)}$$

$$(f) C(v, T) = U + 2g - D_v - 2W_v, v \in V$$

$$(g) \text{二分搜 } g:$$

$$l = 0, r = U, eps = 1/n^2$$

$$\text{if}((U \times |V| - flow(S, T))/2 > 0) l = mid$$

$$\text{else } r = mid$$

$$(h) ans = \min_cut(S, T)$$

$$(i) |E| = 0 \text{ 要特殊判斷}$$

- 弦圖：

- 點數大於 3 的環都要有一條弦
- 完美消除序列從後往前依次給每個點染色，給每個點染上可以染的最小顏色
- 最大團大小 = 色數
- 最大獨立集：完美消除序列從前往後能選就選
- 最小團覆蓋：最大獨立集的點和他延伸的邊構成
- 區間圖是弦圖
- 區間圖的完美消除序列：將區間按造又端點由小到大排序
- 區間圖染色：用線段樹做

15.1.3 dinic 特殊圖複雜度

- 單位流： $O\left(\min\left(V^{3/2}, E^{1/2}\right)E\right)$
- 二分圖： $O\left(V^{1/2}E\right)$

15.1.4 0-1 分數規劃

$x_i \in \{0, 1\}$ · x_i 可能會有其他限制 · 求 $\max \left(\frac{\sum B_i x_i}{\sum C_i x_i} \right)$

1. $D(i, g) = B_i - g \times C_i$
2. $f(g) = \sum D(i, g) x_i$
3. $f(g) = 0$ 時 g 為最佳解 · $f(g) < 0$ 沒有意義
4. 因為 $f(g)$ 單調可以二分搜 g
5. 或用 Dinkelbach 通常比較快

```

1 binary_search(){
2   while(r-l>eps){
3     g=(l+r)/2;
4     for(i:所有元素)D[i]=B[i]-g*C[i];//D(i,g)
5     找出一組合法x[i]使f(g)最大;
6     if(f(g)>0) l=g;
7     else r=g;
8   }
9   Ans = r;
10 }
11 Dinkelbach(){
12   g=任意狀態 (通常設為0);
13   do{
14     Ans=g;
15     for(i:所有元素)D[i]=B[i]-g*C[i];//D(i,g)
16     找出一組合法x[i]使f(g)最大;
17     p=0,q=0;
18     for(i:所有元素)
19       if(x[i])p+=B[i],q+=C[i];
20     g=p/q;//更新解 · 注意q=0的情況
21   }while(abs(Ans-g)>EPS);
22   return Ans;
23 }
```

15.1.5 學長公式

1. $\sum_{d|n} \phi(n) = n$
2. $g(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) \times g(n/d)$
3. Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
4. $\gamma = 0.57721566490153286060651209008240243104215$
5. 格雷碼 $= n \oplus (n >> 1)$
6. $SG(A+B) = SG(A) \oplus SG(B)$
7. 選轉矩陣 $M(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

15.1.6 基本數論

1. $\sum_{d|n} \mu(n) = [n == 1]$
2. $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
3. $\sum_{i=1}^n \sum_{j=1}^m \text{互質數量} = \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
4. $\sum_{i=1}^n \sum_{j=1}^n \text{lcm}(i, j) = n \sum_{d|n} d \times \phi(d)$

15.1.7 排組公式

1. k 卡特蘭 $\frac{C_k n}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
2. $H(n, m) \cong x_1 + x_2 \dots + x_n = k, num = C_k^{n+k-1}$
3. Stirling number of 2^{nd} , n 人分 k 組方法數目
 - (a) $S(0, 0) = S(n, n) = 1$
 - (b) $S(n, 0) = 0$
 - (c) $S(n, k) = kS(n-1, k) + S(n-1, k-1)$
4. Bell number, n 人分任意多組方法數目

- (a) $B_0 = 1$
- (b) $B_n = \sum_{i=0}^n S(n, i)$
- (c) $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
- (d) $B_{p+n} \equiv B_n + B_{n+1} \pmod{p}$, p is prime
- (e) $B_p^m \equiv mB_n + B_{n+1} \pmod{p}$, p is prime
- (f) From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$

5. Derangement, 錯排, 沒有人在自己位置上

- (a) $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \dots + (-1)^n \frac{1}{n!})$
- (b) $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
- (c) From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$

6. Binomial Equality

- (a) $\sum_k \binom{r+k}{m+l} \binom{s}{n-k} = \binom{r+s}{m+n}$
- (b) $\sum_k \binom{m+k}{m+l} \binom{s}{n-k} = \binom{l+s}{l-m+n}$
- (c) $\sum_k \binom{l}{m+k} \binom{s+k}{n} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$
- (d) $\sum_{k \leq l} \binom{l-k}{m} \binom{s}{k-n} (-1)^k = (-1)^{l+m} \binom{s-m-1}{m+n+1}$
- (e) $\sum_{0 \leq k \leq l} \binom{l-n-m}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
- (f) $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$
- (g) $\binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k}$
- (h) $\sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{n+1}$
- (i) $\sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$
- (j) $\sum_{k \leq m} \binom{m+r}{k} x^k y^{m-k} = \sum_{k \leq m} \binom{-r}{k} (-x)^k (x+y)^{m-k}$

15.1.8 冪次, 冪次和

1. $a^{b \% P} = a^{b \% \varphi(P) + \varphi(P)}, b \geq \varphi(P)$
2. $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
3. $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
4. $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
5. $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n+1$
6. $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
7. $\sum_{j=0}^m C_j^{m+1} B_j = 0, B_0 = 1$
8. 除了 $B_1 = -1/2$ · 剩下的奇數項都是 0
9. $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

15.1.9 Burnside's lemma

1. $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$
2. $X^g = t^{c(g)}$
3. G 表示有幾種轉法 · X^g 表示在那種轉法下 · 有幾種是會保持對稱的 · t 是顏色數 · $c(g)$ 是循環節不動的面數 ·
4. 正立方體塗三顏色 · 轉 0 有 3^6 個元素不變 · 轉 90 有 6 種 · 每種有 3^3 不變 · 180 有 $3 \times 3^4 \cdot 120(\text{角})$ 有 $8 \times 3^2 \cdot 180(\text{邊})$ 有 6×3^3 · 全部 $\frac{1}{24} (3^6 + 6 \times 3^3 + 3 \times 3^4 + 8 \times 3^2 + 6 \times 3^3) = \frac{57}{57}$

15.1.10 Count on a tree

1. Rooted tree: $s_{n+1} = \frac{1}{n} \sum_{i=1}^n (i \times a_i \times \sum_{j=1}^{\lfloor n/i \rfloor} a_{n+1-i \times j})$
2. Unrooted tree:
 - (a) Odd: $a_n - \sum_{i=1}^{n/2} a_i a_{n-i}$
 - (b) Even: $Odd + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$
3. Spanning Tree

- (a) 完全圖 $n^n - 2$
- (b) 一般圖 (Kirchhoff's theorem) $M[i][i] = \text{degree}(V_i), M[i][j] = -1, \text{if have } E(i, j), 0 \text{ if no edge. delete any one row and col in } A, ans = \det(A)$

15.1.11 循環小數轉分數

1. 若 $x = 0.\bar{a}$ · 則

$$x = \frac{a}{\underbrace{99 \dots 9}_{k \text{ digits}}}$$

其中 a 為循環節 · k 為循環節的位數 ·

2. 例子 :

$$0.\overline{37} = \frac{37}{99}$$

$$0.\overline{5} = \frac{5}{9}$$

15.1.12 循環小數轉分數

1. 純循環小數 : 若 $x = 0.\bar{a}$ · 其中 a 為循環節 · 長度為 k ·

$$x = \frac{a}{\underbrace{99 \dots 9}_{k \text{ digits}}}$$

例 :

$$0.\overline{37} = \frac{37}{99}, \quad 0.\overline{5} = \frac{5}{9}$$

2. 混循環小數 : 若 $x = 0.b\bar{a}$ · 其中 b 為前綴 · 長度 m · a 為循環節 · 長度 k ·

$$x = \frac{(b \cdot 10^k + a) - b}{10^m(10^k - 1)}$$

例 :

$$0.12\overline{3} = \frac{(12 \cdot 10^1 + 3) - 12}{10^2(10^1 - 1)} = \frac{123 - 12}{100 \cdot 9} = \frac{111}{900} = \frac{1}{8}$$

15.1.13 常見級數與組合公式

1. 平方和公式 :

$$1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$4^2 + 6^2 + \dots + (2n)^2 = \frac{(2n)(n+1)(2n+1)}{3}$$

$$1^2 + 3^2 + \dots + (2n+1)^2 = \frac{n(2n-1)(2n+1)}{3}$$

2. 立方和公式 :

$$1^3 + 2^3 + \dots + n^3 = \frac{n^4 + 2n^3 + n^2}{4}$$

3. 四次方和公式 :

$$1^4 + 2^4 + \dots + n^4 = \frac{6n^5 + 15n^4 + 10n^3 - n}{30}$$

4. 五次方和公式 :

$$1^5 + 2^5 + \dots + n^5 = \frac{2n^6 + 6n^5 + 5n^4 - n^2}{12}$$

5. 六次方和公式 :

$$1^6 + 2^6 + \dots + n^6 = \frac{6n^7 + 21n^6 + 21n^5 - 7n^3 + n}{42}$$

6. 七次方和公式 :

$$1^7 + 2^7 + \dots + n^7 = \frac{3n^8 + 12n^7 + 14n^6 - 7n^4 + 2n^2}{24}$$

7. 八次方和公式 :

$$\frac{1^8 + 2^8 + \dots + n^8}{= \frac{10n^9 + 45n^8 + 60n^7 - 42n^5 + 20n^3 - 3n}{90}}$$

8. 九次方和公式 :

$$\frac{1^9 + 2^9 + \dots + n^9}{= \frac{2n^{10} + 10n^9 + 15n^8 - 14n^6 + 10n^4 - 3n^2}{20}}$$

9. 十次方和公式：

$$= \frac{1^{10} + 2^{10} + \cdots + n^{10}}{66} = \frac{6n^{11} + 33n^{10} + 55n^9 - 66n^7 + 66n^5 - 33n^3 + 5n}{66}$$

10. 等比級數：

$$S = a \cdot \frac{r^n - 1}{r - 1}$$

11. 二項式係數恆等式：

$$\binom{n}{0} + \binom{n}{1} + \cdots + \binom{n}{n} = 2^n$$
$$\binom{n}{1} + \binom{n}{2} + \cdots + \binom{n}{n} = 2^n - 1$$
$$\binom{n}{1} + \binom{n}{3} + \cdots + \binom{n}{n-k} = 2^{n-1}$$

12. 分配問題 (玩具分給小孩)：

(a) n 個玩具 · k 位小孩 · 可以有人沒拿到：

$$\binom{n+k-1}{n} = \binom{n+k-1}{k-1}$$

(b) n 個玩具 · k 位小孩 · 每個人至少一個：

$$\binom{n-1}{k-1}$$

15.1.14 位元運算

(a) 位元條件：

$$(x + k) \& (y + k) = 0$$

(b) 加法恆等式 (利用 XOR 與 AND)：

$$a + b = (a \oplus b) + 2 \cdot (a \& b)$$

(c) OR 與 AND 的關係：

$$a | b = a + b - (a \& b)$$

(d) 交換兩數：

$$a = a \oplus b, \quad b = a \oplus b, \quad a = a \oplus b$$

(e) 取得最低位的 1：

$$x \& (-x)$$

(f) 清除最低位的 1：

$$x \& (x - 1)$$

15.1.15 除數與倍數

(a) LCM:

$$\text{lcm}(a, b) = \frac{a \times b}{\text{gcd}(a, b)}$$

(b) LCM (prime factorization):

$$\text{lcm}(a, b) = \prod_{p \in \text{primes}} p^{\max(e_a, e_b)}$$

(c) GCD (prime factorization):

$$\text{gcd}(a, b) = \prod_{p \in \text{primes}} p^{\min(e_a, e_b)}$$

(d) Counting divisors:

$$\text{If } n = \prod_{i=1}^k p_i^{e_i}, \text{ cnt is } \prod_{i=1}^k (e_i + 1)$$

15.1.16 數論公式

13. Bezout's identity：

$$ax + by = \text{gcd}(a, b) \quad (\text{必定存在整數解 } x, y)$$

14. 模指數運算 (冪的冪)：

$$a^{b^c} \equiv a^{\text{power_mod}(b, c, \text{MOD}-1)} \pmod{\text{MOD}}$$

Pythagorean theorem：

$$(n^2 + m^2, 2nm, n^2 - m^2), \quad n > m > 0$$

Wilson's theorem：

$$(p - 1)! \pmod p = p - 1 \quad (p \text{ 為質數})$$

Catalan number：

$$C_n = \frac{1}{n+1} \binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$$

海龍公式：

$$\text{Area} = \sqrt{s(s-a)(s-b)(s-c)}, \quad s = \frac{a+b+c}{2}$$

兩圓相交弦長：

$$\text{Length} = 2 \cdot \sqrt{\frac{(s-a)(s-b)}{s}}, \quad s = \frac{r_1 + r_2 + d}{2}$$

三角形內切圓半徑：

$$r = \frac{2 \cdot \text{Area}}{a + b + c}$$

三角形外接圓半徑：

$$R = \frac{abc}{4 \cdot \text{Area}}$$

三角形重心：

$$G = \left(\frac{x_1 + x_2 + x_3}{3}, \frac{y_1 + y_2 + y_3}{3} \right)$$

三角形垂心：

$$H = \left(\frac{x_1 \tan A + x_2 \tan B + x_3 \tan C}{\tan A + \tan B + \tan C}, \frac{y_1 \tan A + y_2 \tan B + y_3 \tan C}{\tan A + \tan B + \tan C} \right)$$

三角形內心：

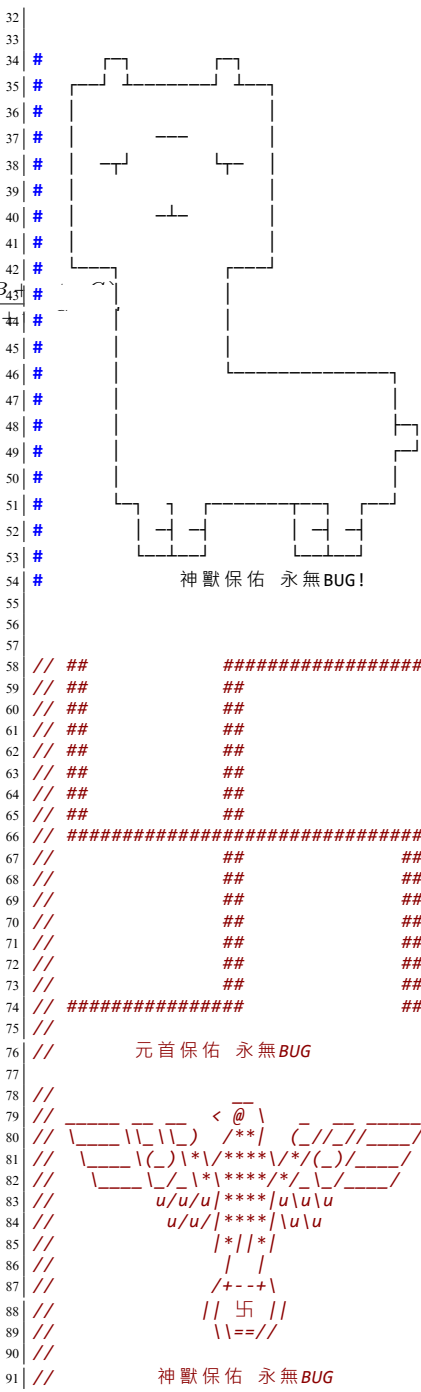
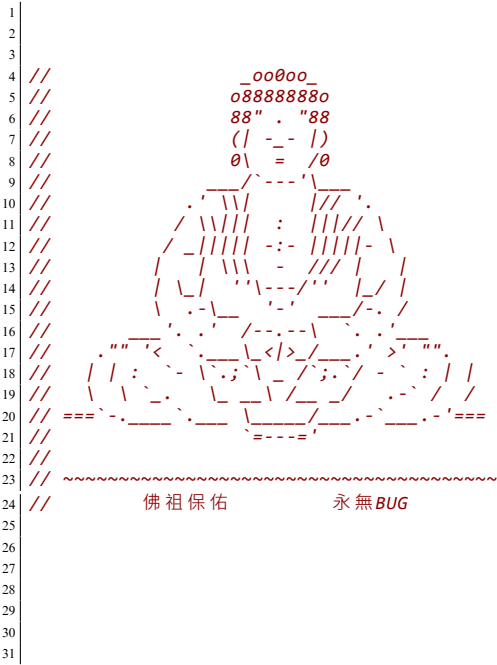
$$I = \left(\frac{ax_1 + bx_2 + cx_3}{a + b + c}, \frac{ay_1 + by_2 + cy_3}{a + b + c} \right)$$

奇 (偶) 長度陣列子數量：

$$n = \frac{(i + 1) * (n - i) + 1}{2}$$

16 Интернационал

16.1 保佑



ACM ICPC Team Reference - BogoSort

Contents

1 Algorithm	1				
1.1 LIS	1				
1.2 LCS	1				
2 Basic	1				
2.1 mod helper function	1				
2.2 self-defined-pq-operator	1				
2.3 generating all subsets	1				
2.4 memset	1				
2.5 submask enumeration	1				
2.6 custom-hash	1				
2.7 stringstream split by comma	1				
3 DP	1				
3.1 deque	1				
3.2 walk	1				
3.3 grouping	2				
3.4 matching	2				
3.5 projects	2				
3.6 stones	2				
3.7 coins	2				
3.8 elevator rides	3				
3.9 slimes	3				
3.10 digit sum	3				
3.11 sushi	3				
3.12 candies	3				
3.13 permutation	4				
3.14 Knasack2	4				
3.15 flowers	4				
3.16 independent set	4				
3.17 counting tower	4				
3.18 counting numbers	5				
4 Data Structure	5				
4.1 undo disjoint set	5				
4.2 segment tree range update (lazy propagation)	5				
4.3 lazy propagation	6				
4.4 segment tree prefix sum lower bound	6				
4.5 Fenwick tree (BIT)	6				
4.6 BIT - range update + range prefix sum	7				
4.7 Order Statistics Tree	7				
4.8 Trie (Prefix tree)	7				
4.9 segment tree	7				
4.10 BIT range update point query	7				
4.11 DSU remove node find prev next one	7				
4.12 DSU	8				
5 Geometry	8				
5.1 cross. product	8				
5.2 Check Point in Polygon	8				
5.3 Point	8				
5.4 Polygon Lattice Points	9				
5.5 line intersect	9				
5.6 Polygon area	9				
5.7 using std::complex	9				
5.8 point distance from a line	10				
6 Graph	10				
6.1 Prim	10				
6.2 Eulerian cycle	10				
6.3 maximum weight bipartite matching	10				
6.4 maximum flow	11				
6.5 maximum cardinality bipartite matching	11				
6.6 Floyd-Warshall	11				
6.7 MST	12				
6.8 Hopcroft-Karp	12				
6.9 Dijkstra	12				
6.10 Dinic	12				
6.11 maximum cardinality matching	12				
6.12 topological sort	13				
6.13 Floyd's algorithm	13				
6.14 stable matching	14				
6.15 Bellman-Ford	14				
7 Language	14				
7.1 CNF	14				
8 Number Theory	14				
8.1 Linear Sieve	14				
8.2 C(n,m)	14				
8.3 Euler Totient 1 to n	14				
8.4 derangement (Principle of Inclusion-Exclusion)	15				
8.5 matrix template (with fast power)	15				
8.6 Sieve of Eratosthenes (with big num)	15				
8.7 mod inv	15				
8.8 derangement (DP)	15				
8.9 Euler Totient	15				
8.10 fast power	15				
8.11 first and second mex	15				
8.12 Catalan Number	15				
8.13 Chinese Remainder Theorem	16				
8.14 mod inv (not prime)	16				
8.15 mod inv (not coprime)	16				
8.16 Mint	16				
8.17 inclusion-exclusion principle	16				
8.18 C(n,k) mod inverse	16				
8.19 Linear Diophantine 丢番圖	16				
8.20 擴展歐基里德	16				
8.21 Sieve of Eratosthenes	17				
8.22 builtin	17				
8.23 C(n,k) DP	17				
9 Range Queries	17				
9.1 subarray maximum sum queries	17				
9.2 find first equal or greater	18				
9.3 find k-th and count less than	18				
10 String	18				
10.1 Z	18				
10.2 manacher	19				
10.3 AC 自動機	19				
10.4 KMP	19				
10.5 suffix array lcp	19				
10.6 hash	19				
10.7 minimal string rotation	20				
10.8 reverseBWT	20				
11 Tree	20				
11.1 Euler tour+RMQ	20				
11.2 HLD template	20				
11.3 dist between u-v in tree	21				
11.4 all longest path dfs	21				
11.5 all longest path top sort	21				
11.6 Heavy-light decomposition (HLD)	21				
11.7 binary lifting	22				
11.8 get kth ancestor	22				
11.9 check z on path u-v	22				
11.10 binary jumping	22				
11.11 tree diameter	22				
11.12 all longest path	22				
11.13 tree diameter (len,end)	23				
11.14 minimum distance in functional graph	23				
11.15 union length of two tree paths	23				
11.16 Centroid decomposition	24				
12 default	24				
12.1 debug	24				
12.2 template	24				
12.3 vimrc	25				
13 other	25				
13.1 Nim game	25				
13.2 找小於 n 所有出現的 1 數量	25				
14 other language	25				
14.1 python heap	25				
14.2 java	25				
14.2.1 文件操作	25				
14.2.2 优先队列	25				
14.2.3 Map	25				
14.2.4 sort	25				
14.3 python output	25				
14.4 python 大數因數分解	25				
14.5 decimal	26				
14.6 python 大數排序	26				
14.7 python 大數計算 2	26				
14.8 python input	26				
14.9 python 大數計算	26				

15	zformula	26	15.1.4	0-1 分數規劃	27	15.1.9	Burnside’s lemma . . .	27	15.1.14	位元運算	28		
	15.1	formula	26	15.1.5	學長公式	27	15.1.10	Count on a tree	27	15.1.15	除數與倍數	28	
		15.1.1	Pick 公式	26	15.1.6	基本數論	27	15.1.11	循環小數轉分數 . . .	27	15.1.16	數論公式	28
		15.1.2	圖論	26	15.1.7	排組公式	27	15.1.12	循環小數轉分數 . . .	27	16	Интернационал	28
		15.1.3	dinic 特殊圖複雜度 . .	26	15.1.8	冪次, 冪次和	27	15.1.13	常見級數與組合公式 .	27	16.1	保佑	28

ACM ICPC Judge Test - BogoSort

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6     const size_t KB = 1024;
7     const size_t MB = KB * 1024;
8     const size_t GB = MB * 1024;
9
10    size_t block_size, bound;
11    void stack_size_dfs(size_t depth = 1) {

```

```

12        if (depth >= bound)
13            return;
14        int8_t ptr[block_size]; // 若無法編譯將
15            block_size 改成常數
16        memset(ptr, 'a', block_size);
17        cout << depth << endl;
18        stack_size_dfs(depth + 1);
19    }
20    void stack_size_and_runtime_error(size_t
21        block_size, size_t bound = 1024) {
22        system_test::block_size = block_size;
23        system_test::bound = bound;
24        stack_size_dfs();
25    }
26    double speed(int iter_num) {
27        const int block_size = 1024;
28        volatile int A[block_size];
29        auto begin = chrono::high_resolution_clock
30            ::now();
31        while (iter_num--)
32            for (int j = 0; j < block_size; ++j)
33                A[j] += j;
34        auto end = chrono::high_resolution_clock::
35            now();
36        chrono::duration<double> diff = end -
37            begin;

```

```

38        return diff.count();
39    }
40    void runtime_error_1() {
41        // Segmentation fault
42        int *ptr = nullptr;
43        *(ptr + 7122) = 7122;
44    }
45    void runtime_error_2() {
46        // Segmentation fault
47        int *ptr = (int *)memset;
48        *ptr = 7122;
49    }
50    void runtime_error_3() {
51        // munmap_chunk(): invalid pointer
52        int *ptr = (int *)memset;
53        delete ptr;
54    }
55    void runtime_error_4() {
56        // free(): invalid pointer
57        int *ptr = new int[7122];
58        ptr += 1;
59        delete[] ptr;
60    }
61    }
62

```

```

63    void runtime_error_5() {
64        // maybe illegal instruction
65        int a = 7122, b = 0;
66        cout << (a / b) << endl;
67    }
68    void runtime_error_6() {
69        // floating point exception
70        volatile int a = 7122, b = 0;
71        cout << (a / b) << endl;
72    }
73    void runtime_error_7() {
74        // call to abort.
75        assert(false);
76    }
77    } // namespace system_test
78
79    #include <sys/resource.h>
80    void print_stack_limit() { // only work in
81        Linux
82        struct rlimit l;
83        getrlimit(RLIMIT_STACK, &l);
84        cout << "stack_size = " << l.rlim_cur << "
85            byte" << endl;
86    }
87

```